

Bork in Action

Język, kompilator i mapa repozytorium

Na podstawie kodu Bork

2026

- [1 Przedmowa](#)
- [2 O tej książce](#)
- [3 Rozdział 1. Pierwszy program i sposób jego uruchomienia](#)
- [4 Rozdział 2. Po co Borkowi regiony pamięci](#)
- [5 Rozdział 3. Z czego składa się plik źródłowy](#)
- [6 Rozdział 4. Typy danych i ich znaczenie](#)
- [7 Rozdział 5. Funkcje, parametry i wywołania](#)
- [8 Rozdział 6. Warunki, pętle i powrót z funkcji](#)
- [9 Rozdział 7. Tablice o znanej z góry długości](#)
- [10 Rozdział 8. Kopiowanie, współdzielenie i przeniesienie wartości](#)
- [11 Rozdział 9. Jak Bork przechowuje napisy w pamięci](#)
- [12 Rozdział 10. Komunikaty kompilatora i błędy podczas działania](#)
- [13 Rozdział 11. Funkcje wbudowane i sposób pisania programów](#)
- [14 Rozdział 12. Od pliku źródłowego do gotowego programu](#)
- [15 Rozdział 13. Jak kompilator czyta tekst programu](#)
- [16 Rozdział 14. Jak kompilator sprawdza własność nazw](#)
- [17 Rozdział 15. Jak kompilator sprawdza typy](#)
- [18 Rozdział 16. Jak kompilator wybiera miejsce na napis](#)
- [19 Rozdział 17. Jak powstaje kod maszynowy](#)
- [20 Rozdział 18. Komunikaty błędów, polecenia i edytor](#)
- [21 Rozdział 19. Jak czytać repozytorium](#)
- [22 Rozdział 20. Jeden program od źródła do uruchomienia](#)
- [23 Rozdział 21. Testy, szukanie przyczyny błędu i nowa konstrukcja języka](#)
- [24 Rozdział 22. Ćwiczenia](#)
- [25 Dodatek A. Zestawienie składni](#)
- [26 Dodatek B. Słownik pojęć](#)
- [27 Dodatek C. Czego kompilator jeszcze nie potrafi](#)

1 Przedmowa

Bork powstał z praktycznego zderzenia trzech przyzwyczajęń. W Kotlinie pisze się krótko. Blok kodu, funkcja przekazana na końcu wywołania i warunek zapisany jako wyrażenie nie wymagają od programisty osobnego rytuału. W Rustcie program natywny jest szybki i nie ma garbage collector, ale czas życia każdej

referencji trzeba opisać i obronić przed sprawdzaniem pożyczek. W C pamięć zwalnia się ręcznie, więc każdy free jest osobną decyzją, którą łatwo pomylić.

Bork wybiera inną zasadę, bo nawiasy klamrowe nie są tylko sposobem grupowania instrukcji, lecz oznaczają region pamięci. Napis albo tablica utworzona w takim bloku żyje tak długo, jak długo wykonanie pozostaje w tym bloku, a wyjście z bloku zwalnia cały bufor regionu naraz. Nie ma odśmiecania w tle ani ogólnych referencji, które można by schować w polu struktury, bo struktur definiowanych przez programistę w obecnej wersji języka nie ma. Język ma jednak ograniczone, jawne pożyczanie przez & — opisane dalej — i nie ma pełnego, rustowego borrow checkera.

Ta książka opisuje język taki, jaki przyjmuje kompilator w tym repozytorium, oraz sam kompilator, plik po pliku. To rozróżnienie jest ważne. Część kompilatora, która sprawdza program, rozumie wartości puste, operator `?:`, wykrzykniki `!!` oraz funkcję dopisaną na końcu wywołania. Generator kodu maszynowego tłumaczy węższy zestaw konstrukcji. Niektóre programy, które przechodzą sprawdzenie, dostają przy budowaniu zwykły komunikat błędu. Inne, i to trzeba powiedzieć wprost, kończą się awarią samego kompilatora. Dodatek C wymienia te miejsca.

Nazwa diagnostyki jest częścią projektu. Gdy program jest niepoprawny, komunikat mówi, w której fazie kompilator przerwał pracę. Język nie ma wyjątków, typu `Result` ani instrukcji `try`. Błąd programisty jest albo błędem kompilacji, albo przerwaniem programu w czasie działania, na przykład przy dzieleniu przez zero albo przy indeksie spoza tablicy.

Książka jest po polsku. Terminy, które w kodzie i w komunikatach kompilatora pozostają angielskie, podaję przy pierwszym użyciu i potem trzymam się jednego brzmienia. Słowo `move` w pliku źródłowym zostaje słowem `move`. W zdaniu objaśniającym mówię o przeniesieniu własności.

Jeśli chcesz najpierw napisać program, czytaj część pierwszą i trzymaj się przykładów, które dają się zbudować. Jeśli chcesz czytać kod kompilatora albo dopisać nową konstrukcję, część trzecia prowadzi przez repozytorium w kolejności, w której kolejne fazy naprawdę na sobie polegają.

2 O tej książce

Ta książka uczy języka Bork i jednocześnie objaśnia kod kompilatora napisanego w Rustie. Nie jest listą życzeń. Jeśli jakiejś funkcji nie ma w parserze, w sprawdzaniu typów, w analizie własności albo w generatorze kodu, nie opisuję jej jako części języka. Jeśli występuje tylko w notatkach projektowych z katalogu `docs/superpowers` albo w pliku `TODO.md`, zaznaczam, że jest to plan albo

zaległość, a nie gotowa możliwość.

2.1 Dla kogo jest ta książka

Zakładam, że piszesz już w jakimś języku programowania. Najwygodniej będzie, jeśli znasz choć jeden z trzech punktów odniesienia, do których Bork sam się porównuje.

Kotlin albo inny język z blokami, z warunkiem użytym jako wyrażenie i z funkcją dopisywaną na końcu wywołania da ci znajomą składnię. Rust albo C++ da ci słownictwo własności i przeniesienia, nawet jeśli w Borku działa ono inaczej niż sprawdzanie pożyczek. Odrobina wiedzy o kompilatorach wystarczy w części drugiej. Wystarczy wiedzieć, że program najpierw jest czytany jako tekst, potem dostaje strukturę, potem jest sprawdzany, a na końcu zamieniany na kod maszynowy. Nie musisz znać biblioteki Inkwel ani protokołu serwera językowego.

Część pierwszą da się czytać bez zaglądania do plików Rusta. Część druga podaje nazwy funkcji i plików. Część trzecia zakłada, że masz repozytorium otwarte obok książki.

2.2 Jak czytać

Są trzy sensowne przejścia.

Ścieżka języka prowadzi przez przedmowę, rozdziały od 1 do 11 oraz dodatki A i C. Przy każdym listingu, o którym piszę, że został uruchomiony, warto uruchomić go samodzielnie. Listing, o którym piszę, że przechodzi tylko sprawdzenie, kompilator akceptuje poleceniem `bork plik.bork`, ale polecenie `bork build` albo go odrzuca, albo kończy się awarią kompilatora.

Ścieżka kompilatora zaczyna się od rozdziału 2, bo bez powodu istnienia regionów dalsze fazy wyglądają jak zbiór niepowiązanych przejść. Potem idą rozdziały od 12 do 18 i rozdział 20. Rozdział 20 przeprowadza jeden krótki program przez kolejne funkcje kompilatora.

Ścieżka osoby, która chce zmieniać kompilator, to rozdziały od 19 do 22 i dodatek C. Rozdział 21 pokazuje, które testy łapią którą fazę, i jak nowe słowo kluczowe, na przykład `while`, musiało przejść przez wszystkie warstwy.

Komunikaty w książce pochodzą z uruchomienia programu `bork` złożonego w tym środowisku z LLVM 23.1.2. Linia błędu ma postać `plik:linia:kolumna: error: faza: treść`. Kolumna liczy znaki od początku wiersza. Gdy błąd nie ma pozycji w pliku, numer linii znika. Tak jest na przykład przy próbie zwrócenia wyniku funkcji `concat`.

2.3 Umowy typograficzne

Listingi są numerowane w obrębie rozdziału. Pod listingiem zdania omawiają, co program robi i jaki był wynik uruchomienia, jeśli uruchomienie było możliwe. Numer w nawiasie, na przykład (1), odsyła do komentarza w kodzie tylko wtedy, gdy kod naprawdę ma taki znacznik.

Ramki mają trzy role. Nota dopowiada fakt, który łatwo przeoczyć, wskazówka mówi, co zrobić w praktyce, a ostrzeżenie dotyczy albo programu, który sprawdzenie akceptuje, choć budowanie go odrzuca lub na którym kompilator się wywraca, albo zachowania, które przerywa już uruchomiony program.

Kod Borka jest w blokach oznaczonych `bork`, a kod Rusta, który jest częścią kompilatora, jest w blokach `rust` i ma w podpisie ścieżkę pliku. Diagramy są zapisane w Mermaid, natomiast w pliku PDF są obrazkami.

2.4 Czego książka nie obiecuje

W tej wersji Bork nie ma modułów, polecenia `import`, pakietów ani przestrzeni nazw, nie ma też struktur, wyliczeń definiowanych przez programistę, cech ani typów ogólnych. Nie ma wyjątków ani jednoargumentowego minusa, więc nie zapiszesz literału ujemnego, a instrukcji nie rozdziela się średnikiem. Nie ma garbage collector'a ani ogólnego systemu referencji w stylu Rusta: `&T` jest ograniczone do parametrów i stałych widoków, nie można zwracać takich referencji ani przechowywać ich w polach. Nie ma też interpretera, trybu `bork run` ani debuggera. Generator kodu nie tłumaczy funkcji dopisanej na końcu wywołania, wartości `Some` i `None`, operatora `?:` ani asercji `!!`.

Starsze wyniki uruchomień w dodatku C pochodzą sprzed obsługi deskryptorów napisów w argumentach funkcji. Obecny kod ma ścieżkę przekazującą deskryptor, ale dopóki przykład nie zostanie ponownie zbudowany z opcją `codegen`, książka nie podaje nowego wyniku wykonania.

2.5 O źródłach tej książki

Rozdziały leżą w katalogu `docs/book`, a przykłady uruchomione przy pisaniu książki są w `docs/book/przyklady` razem z plikiem `WYNIKI.md`. Historia commitów projektu jest tylko tłem rozdziału 19, bo książka nie streszcza każdego pull requestu.

Język i kompilator są projektem Andrzeja Witkowskiego, a ta książka jest opisem tego kodu dla innych programistów i nie zmienia semantyki. Gdy dokument `docs/language.md` rozmija się z kompilatorem, opis dotyczy tego, co robi kompilator, i mówi, na czym polega różnica. Najważniejszy przykład dotyczy konkatencji dwóch zmiennych napisowych wewnątrz bloku: dokument pokazuje

wywołanie bez `move`, ale kompilator takiego programu nie przyjmuje.

3 Rozdział 1. Pierwszy program i sposób jego uruchomienia

3.1 Ten rozdział obejmuje

- czym jest program napisany w Borku
- jak sprawdzić plik, nie budując programu wykonywalnego
- jak zbudować program i jak odczytać jego kod wyjścia
- jak czytać pierwszy komunikat o błędzie
- które konstrukcje język już rozumie, a których generator kodu jeszcze nie tłumaczy

3.2 Program, który wypisuje jeden wiersz

Bork jest językiem kompilowanym do kodu maszynowego. Kompilator czyta plik tekstowy, sprawdza, czy program jest poprawny, i na życzenie tłumaczy go na plik, który da się uruchomić. Składnia jest zbliżona do Kotliny. Funkcję rozpoczyna słowo `fun`, nazwę niezmienną słowo `val`, nazwę zmienną słowo `var`, a grupę instrukcji obejmuje się nawiasami klamrowymi. Pamięcią nie zarządza `garbage collector`. Sens tej decyzji wyjaśnia następny rozdział. Tutaj celem jest zobaczyć działający program i pierwszy błąd.

Listing 1.1. Funkcja `main` wypisuje słowo i przechodzi do nowego wiersza.

```
fun main() {  
    println("hi")  
}
```

Program składa się z funkcji, więc nie ma instrukcji zapisanych luzem na poziomie pliku. `main` jest zwykłą funkcją, która przy budowaniu programu wykonywalnego dostaje szczególną rolę, bo punkt wejścia procesu nazywa się właśnie tak. Jeśli nie podasz typu wyniku, funkcja nie zwraca wartości i taki `main` kończy się kodem wyjścia zero. `println` jest funkcją wbudowaną, która wypisuje argument i dodaje znak nowego wiersza. Ten program został zbudowany i uruchomiony: na standardowym wyjściu pojawił się tekst `hi` oraz nowy wiersz, a proces zakończył się kodem zero.

Listing 1.2. Wynik funkcji `main` staje się kodem wyjścia procesu.

```
fun add(a: i32, b: i32): i32 {  
    return a + b  
}
```

```

}

fun main(): i32 {
    return add(40, 2)
}

```

Funkcja `add` przyjmuje dwie liczby typu `i32` i zwraca ich sumę. Parametr zapisany jako nazwa: `Typ`, bez słowa `val` albo `var`, jest nazwą niezmienną. `main` z zadeklarowanym wynikiem `i32` przekazuje tę liczbę systemowi jako kod wyjścia. Po zbudowaniu i uruchomieniu proces kończy się kodem 42. Innych typów wyniku funkcji `main` generator kodu nie obsługuje. Próba zwrócenia `i64`, `f64` albo `bool` przechodzi sprawdzenie typów, a przy budowaniu dostaje komunikat, że taki wynik nie jest jeszcze obsługiwany.

NOTA. Sprawdzenie pliku bez funkcji `main` może się udać, jeśli reszta programu jest poprawna. Budowanie programu wykonywalnego wymaga `main`. Brak tej funkcji jest błędem fazy generowania kodu, a nie błędem typu.

3.3 Sprawdzenie i zbudowanie

Samą poprawność programu sprawdzisz bez biblioteki LLVM. LLVM to biblioteka, która później tłumaczy sprawdzony program na kod maszynowy. Na tym etapie nie jest potrzebna. Z katalogu repozytorium:

```

cargo run --bin bork -- sciezka/do/pliku.bork
cargo run --bin bork -- --dump-arenas sciezka/do/pliku.bork

```

Pierwsze polecenie uruchamia sprawdzenie, które obejmuje składnię, typy i własność nazw, a drugie dopisuje na standardowe wyjście drzewo regionów. Komunikaty o błędach idą na standardowe wyjście błędów. Kod zero oznacza brak błędów, kod jeden oznacza błąd w programie, a kod dwa oznacza złe wywołanie albo problem z odczytem pliku.

Zbudowanie programu wymaga opcji kompilacji `codegen`, biblioteki LLVM 23, kompilatora `clang` i bibliotek, z którymi ta wersja LLVM jest powiązana. Szczegóły instalacji są w pliku `README` repozytorium. Ubuntu 24.04 nie ma LLVM 23 w domyślnym archiwum pakietów. Po złożeniu kompilatora budujesz program tak:

```

cargo run --features codegen -- build plik.bork
cargo run --features codegen -- build -o wyjscie plik.bork

```

Jeśli nie podasz `-o`, nazwa pliku wynikowego powstaje przez odcięcie rozszerzenia `.bork`. Kompilator odmawia pracy, gdy plik wynikowy nadpisałby plik źródłowy. Komunikat brzmi wtedy `error: output path would overwrite input file`, a kod wyjścia wynosi dwa. Bez opcji `codegen` pod polecenie `build` kończy się informacją, że ta opcja jest wymagana, również kodem dwa.

Listing 1.3. Próba zbudowania pliku, w którym nie ma funkcji `main`.

Źródło to jedna funkcja pomocnicza zwracająca liczbę 7. Budowanie kończy się kodem jeden i nie zostawia pliku wykonywalnego. Komunikat, przepisany z uruchomienia, brzmi:

```
nomain.bork: error: codegen: `fun main` is required to build an executable
```

Słowo `codegen` w komunikacie jest nazwą fazy. Sprawdzenie tego samego pliku, bez budowania, kończy się sukcesem. Język nie wymaga, żeby każdy plik miał `main`. Wymaga tego dopiero tłumaczenie na program wykonywalny.

3.4 Dwa błędy, które pojawiają się na początku

Instrukcje rozdziela się nową linią, a nie średnikiem, więc dwie instrukcje zapisane w jednym wierszu parser odrzuca.

Listing 1.4. Parser odrzuca dwie deklaracje zapisane bez przejścia do nowego wiersza.

```
fun main() { val a = 1 val b = 2 }
```

Komunikat zaczyna się od fazy `parse` i mówi, że token `val` jest nieoczekiwany, a wśród rzeczy oczekiwanych jest `NL`, czyli znak nowej linii. Lista oczekiwanych tokenów jest surowa. Pochodzi wprost z generatora parserów i nie została przepisana na zdanie dla człowieka. Średnik jest traktowany tak samo. W wierszu `val n = 1;` kompilator zgłasza nieoczekiwany token średnika.

Drugi częsty błąd dotyczy zmiany nazwy, która miała pozostać stała.

Listing 1.5. Przypisanie do nazwy wprowadzonej przez `val`.

```
fun main() {  
    val n = 1  
    n = 2  
}
```

Komunikat, z kolumną wskazującą przypisanie, brzmi:

```
err_val.bork:3:5: error: type: cannot assign to immutable `val` binding `n`
```

Faza `type` pochodzi ze sprawdzania typów, a faza `ownership` z analizy własności, i obie mogą pojawić się w jednym uruchomieniu. W wydruku błędy własności stoją przed błędami typów, chociaż sprawdzanie typów wykonuje się wcześniej. Powód tej kolejności jest techniczny i wraca w rozdziale 12. Tutaj wystarczy czytać fazę w komunikacie i nie zakładać, że pierwsza linia jest zawsze pierwszym błędem, który kompilator znalazł w czasie.

3.5 Co język przyjmuje, a czego nie przetłumaczy na kod maszynowy

Poniższy program przechodzi sprawdzenie. Jest to próbka zapisana w kodzie kompilatora jako MVP_SAMPLE. Budowanie kończy się kodem jeden. Test `build_rejects_mvp_sample_at_codegen_gate` pilnuje, żeby w komunikacie były słowa `codegen` i `not supported`.

Listing 1.6. Funkcja przekazana na końcu wywołania. Sprawdzenie kończy się sukcesem. Budowanie jest odrzucane.

```
fun action(a: Int, b: Int, block: (Int, Int) -> Int): Int {
    return block(a, b)
}

fun main(): i32 {
    val threshold = 5
    var accumulator = 0
    for (i in 0..10) {
        accumulator = action(accumulator, i) { acc, current ->
            if (current > threshold) {
                acc + current
            } else {
                acc
            }
        }
    }
    return accumulator
}
```

To jest prawdziwy fragment języka, a nie martwa reguła gramatyki. Sprawdzanie typów kontroluje liczbę parametrów funkcji dopisanej na końcu i typ jej wyniku. Analiza własności buduje dla niej osobny region. Generator kodu zatrzymuje się wcześniej i wypisuje `trailing closures are not supported by codegen`.

OSTRZEŻENIE. Nie każdy program odrzucony przy budowaniu dostaje taki komunikat. Przekazanie napisu do funkcji napisanej przez programistę buduje się i działa; osobnym błędem codegenu jest `float` jako argument funkcji, który jest jawnie odrzucany. Porównanie floatów przechodzi sprawdzenie, ale może wywołać panikę kompilatora. Rozdziały 5 i 17 opisują te przypadki.

3.6 Drzewo regionów, zanim wyjaśnimy regiony

Dla listingu 1.2 polecenie `z - -dump-arenas` wypisuje drzewo. Węzeł `fun add` ma dwie nazwy lokalne, a oraz b. Węzeł `fun main` nie ma nazw lokalnych, bo jedyną instrukcją jest `return`. Znacznik `Local` przy nazwie oznacza, że nazwa powstała w tym regionie. W rozdziale 8 pojawią się pozostałe znaczniki: kopiowanie, współdzielenie i przeniesienie. Ten sam tekst drzewa pokazuje w edytorze polecenie `Bork: Dump Arenas`.

3.7 Podsumowanie

- Program w Borku jest listą funkcji, a instrukcje rozdziela nowa linia.
- Polecenie `bork plik.bork` sprawdza program. Polecenie `bork build` tłumaczy obsługiwany podzbiór na plik wykonywalny.
- Funkcja `main` z wynikiem `i32` przekazuje ten wynik jako kod wyjścia. Funkcja `main` bez typu wyniku kończy się zerem.
- Komunikat błędu ma fazę. Na tym etapie spotkasz `parse`, `type` i `codegen`. Faza `ownership` dojdzie w rozdziałach o własności.
- Funkcja dopisana na końcu wywołania jest częścią języka sprawdzanego przez kompilator i nie jest jeszcze tłumaczona na kod maszynowy.

4 Rozdział 2. Po co Borkowi regiony pamięci

4.1 Ten rozdział obejmuje

- jaki problem z pamięcią Bork chce rozwiązać bez garbage collector'a i bez ogólnego sprawdzania pożyczek
- czym jest region i czym jest arena
- kopiowanie, współdzielenie, przeniesienie i ograniczone pożyczanie wartości spoza bieżącego bloku
- co już działa w kompilatorze, a co jest jeszcze świadomie odłożone

4.2 Dwa rodzaje danych

W programie systemowym są dane, które mieszczą się w rejestrze albo na stosie, i dane, które tam się nie mieszczą. Liczba, wartość logiczna i brak wartości użytkowej należą do pierwszej grupy. Napis i tablica należą do drugiej. Wartość napisu w gotowym programie jest adresem bajtów oraz długością. Same znaki leżą gdzie indziej. Trzeba wiedzieć, kto te bajty zwalnia i skąd kompilator wie, że adres jest jeszcze ważny.

Garbage collector odpowiada, że środowisko wykonawcze zwolni pamięć kiedyś, a programista nie wskazuje chwili. Sprawdzanie pożyczek w Rustcie odpowiada, że każda referencja ma czas życia dowiązany do konkretnego wypożyczenia i że to wypożyczenie nie może przeżyć właściciela. Oba rozwiązania są dobre w swoich miejscach. Bork nie chce płacić ich kosztu. Odśmiecanie odbiera programiście pewność, kiedy pamięć wróci do puli. Ogólne sprawdzanie pożyczek wymaga składni i komunikatów, których ten projekt świadomie nie buduje.

Odpowiedź Borka jest zapisana w `docs/memory-model.md` i zrealizowana w

analizie własności oraz w bibliotece `bork_runtime`. Blok ograniczony nawiasami klamrowymi otwiera region, a region ma arenę, czyli jeden bufor, do którego dopisuje się kolejne bajty przez przesuwanie wskaźnika. Wyjście z bloku ustawia ten wskaźnik z powrotem na początek, cały bufor wraca do puli buforów wolnych i nie zwalnia się pojedynczego napisu w środku regionu.

Dalej słowo `region` znaczy właśnie to: jest jednocześnie fragmentem programu i czasem, przez który żyje pamięć z nim związana. Słowo `arena` oznacza bufor tego regionu. W kodzie kompilatora ta sama nazwa pojawia się jeszcze raz, w innym sensie, i koniec rozdziału do tego wraca, bo pomieszczenie tych dwóch rzeczy utrudnia czytanie źródeł.

4.3 Bufor regionu, nie pojedynczy obiekt

Listing 2.1. Napis zapisany do zmiennej z zewnątrz bloku pozostaje dostępny po wyjściu z bloku.

```
fun main() {  
    var outer = "a"  
    {  
        outer = "b"  
    }  
    println(outer)  
}
```

Program został zbudowany i uruchomiony: na standardowym wyjściu jest `b` oraz nowy wiersz, a kod wyjścia wynosi zero. Zmienna `outer` powstała w regionie funkcji, więc przypisanie wewnątrz bloku zmienia tę zmienną, a nie wynosi nazwy, która żyje tylko w bloku. Treść nowego napisu trafia do bufora zmiennej `outer`, a nie do bufora wewnętrznego bloku, który za chwilę zniknie. Ten mechanizm nazywa się alokacją w miejscu przeznaczenia. Rozdział 9 rozpisuje go na reguły, a tutaj widać skutek, bo po zamknięciu nawiasu napis `"b"` nadal wolno wypisać.

Gdybyś wprowadził wewnątrz bloku nazwę `inner` i próbował użyć jej za zamykającym nawiasem, nazwa w ogóle nie jest widoczna. To jest zwykły zasięg leksykalny i nie trzeba do niego areny. Region nie przedłuża zasięgu nazwy. Nazwa umiera według bloków, a pamięć napisu jest zwalniana według areny, w której została położona. Te dwie rzeczy są złączone, ale nie są tym samym.

4.4 Sposoby przejścia granicy regionu

Wartość z regionu otaczającego można w regionie wewnętrznym skopiować, współdzielić, przenieść albo jawnie pożyczyć. Kompilator wybiera sposób według typu, rodzaju nazwy i tego, czy w tekście zapisano `&`.

Kopiowanie dotyczy typów, które mieszczą się w wartości samej i nie są puste. Należą do nich `i32`, `bool` i pozostałe niepuste typy proste. Odczyt kopiuje bity.

Nazwa w regionie zewnętrznym zostaje żywa.

Współdzielenie dotyczy stałej, której typ nie jest kopiowany, na przykład napisu wprowadzonego przez `val`. Region wewnętrzny patrzy na te same bajty. Nazwa zewnętrzna zostaje żywa. Oprócz tego Bork ma jawne, regionowe pożyczki `&T` dla niekopiowanych wartości. Nie są to ogólne referencje: nie można ich zwracać ani przechowywać w polach, a kompilator nie implementuje pełnego sprawdzania pożyczek z Rusta.

Przeniesienie dotyczy zmiennej, której typ nie jest kopiowany, albo jawnego słowa `move`. Własność przechodzi na nowe użycie, a stara nazwa staje się martwa.

Pożyczka jest czwartą, węższą możliwością. Zapis `&nazwa` tworzy widok bez kopiowania i bez przenoszenia właściciela. Typ `&T` jest używany w parametrach i stałych widokach; nie można go zwrócić ani przechowywać w polu. To nie jest ogólny borrow checker: regiony i ograniczenia typów dopuszczają tylko wybrane kształty użycia.

Listing 2.2. Stały napis jest w bloku wewnętrznym tylko oglądany.

```
fun main() {  
    val s = "x"  
    {  
        println(s)  
    }  
}
```

Program został zbudowany. Wypisuje `x` i nowy wiersz. Drzewo regionów pokazuje w bloku wewnętrznym nazwę `s` ze znacznikiem współdzielenia z regionu `fun main`. Region wewnętrzny nie kopiuje znaków i nie odbiera własności.

Listing 2.3. Po przeniesieniu stara nazwa nie może być użyta.

```
fun main() {  
    var s: String = "hi"  
    val t = move s  
    val u = s  
}
```

Sprawdzenie kończy się błędem fazy ownership:

```
err_use_after.bork:4:13: error: ownership: use of `s` after move from fun main
```

Przeniesienie nie wycina bajtów ze starej areny, więc zostają one miejscem, do którego nie ma już dojścia, aż arena zostanie wyczyszczona. Kompilator unieważnia nazwę i to wystarcza, dopóki nie ma struktur, w których dałoby się przechować adres, a struktur definiowanych przez programistę nie ma.

Słowo `move` przy wartości kopiowanej nie odbiera nazwy. Program, który robi `val`

`n = 1`, potem `val m = move n`, a potem zwraca `n + m`, przechodzi sprawdzenie i budowanie. Kod wyjścia wynosi 2. Drzewo regionów nie oznacza `n` jako przeniesionej.

4.5 Pętla nie może przenieść nazwy z zewnątrz

Ciało pętli wykonuje się wiele razy. Własność zmiennej utworzonej przed pętlą da się oddać tylko raz. Analiza własności zapamiętuje nazwy widoczne przy wejściu do `for` albo `while` i zabrania przenosić je w ciele.

Listing 2.4. Przeniesienie napisu utworzonego przed pętlą jest błędem.

```
fun main() {  
    var s = "Hello, World!"  
    for (i in 1..10) {  
        move {  
            val t = s  
        }  
    }  
}
```

Kompilator wypisuje dwa komunikaty na tym samym miejscu. Pierwszy mówi, że `s` nie może być przeniesione wewnątrz pętli, bo przy kolejnym obiegu byłoby już przeniesione. Drugi mówi, że `s` nie jest kopiowane i trzeba je przenieść do bloku `move`. Dwa zdania wynikają z dwóch sprawdzeń, które widzą ten sam zapis. Nazwę utworzoną wewnątrz ciała pętli wolno przenieść, bo każdy obieg dostaje świeżą zmienną. Przykład jest w rozdziale 8.

Ciało pętli czyści swoją arenę przy każdym obiegu. Pamięć zajmowana przez wartości tymczasowe pętli nie rośnie razem z liczbą obiegów, o ile wynik nie jest zapisywany do zmiennej z zewnątrz. To jest osobna sprawa od zakazu przenoszenia. Poprawna pętla, która buduje tymczasowy napis i nie wynosi go na zewnątrz, zużywa stałą ilość pamięci regionu.

4.6 Czego ten model świadomie nie robi

Dokument modelu pamięci wymienia rzeczy, których nie planuje: ogólnych referencji, referencji przechowywanych w polach ani pełnego rustowego borrow checkera. Istnieją za to ograniczone pożyczki `&T` do parametrów i stałych widoków; ich reguły są opisane przy składni, typach i tablicach. Nie ma niejawnego głębokiego kopiowania przy każdym odczycie spoza regionu ani możliwości wyniesienia widoku z regionu wewnętrznego do zewnętrznego.

4.7 Co już jest zrobione, a co jest odłożone

Bufor areny ma pojemność 4096 bajtów. Stała nazywa się `ARENA_CAPACITY` i leży w

`crates/bork_runtime/src/lib.rs`, a taką samą pojemność ma model opisany w `src/arena.rs`. Wejście do regionu, który naprawdę alokuje, pobiera bufor z puli, wyjście zwraca go do puli, a czyszczenie wskaźnika bez zwrotu bufora jest używane w pętli, żeby nie oddawać pamięci tylko po to, by za chwilę wziąć ją z powrotem. Przepelnienie bufora przerywa program komunikatem Rusta `arena overflow`, z podaną liczbą bajtów i pojemnością 4096. Nie jest to błąd, który program w Borku mógłby obsłużyć.

Odłożone są rzeczy opisane w tym samym dokumencie i widoczne w kodzie analizy ucieczki. Analiza ucieczki, po angielsku `escape analysis`, sprawdza, czy pamięć wartości nie będzie użyta po zwolnieniu areny, w której powstała. Dziś nie ma osobnego bufora na wynik zwracany do funkcji wywołującej, więc nie zwrócisz świeżo sklejonego napisu. Przeniesienie alokacji do zewnętrznej zmiennej rozpoznaje tylko bardzo prosty układ dwóch sąsiednich instrukcji. Słowa `promote` nie używa się przy `return`. Generator kodu nie pomija kopiowania pamięci nawet wtedy, gdy napis już leży we właściwej arenie.

NOTA. Plik `src/arena.rs` opisuje bufor 4096 bajtów i pulę takich buforów. Analiza własności go nie wywołuje. Ta analiza buduje drzewo regionów. Prawdziwe bufory powstają w `bork_runtime`, gdy generator kodu wstawi wywołanie `bork_arena_push`. Czytając źródła, nie traktuj tych dwóch plików jako jednej struktury.

4.8 Kompromis tej decyzji

Region jest gruboziarnisty, bo napis i duża tablica utworzone w tym samym bloku dzielą jeden bufor 4096 bajtów i giną razem. Nie da się zwolnić napisu wcześniej i to jest cena braku `free` oraz braku `garbage collector`a. Stały rozmiar upraszcza generator kodu, bo region ma jeden wskaźnik końca danych i jedno czyszczenie. Gdy dane mają przeżyć blok, trzeba zapisać je do zmiennej z regionu zewnętrznego, gdyż nie ma sterty, która żyłaby dłużej niż funkcja, poza literałami umieszczonymi w stałych programu.

Literał `"hi"` może w ogóle nie leżeć w arenie, bo generator kodu kładzie znaki w stałej, a w wartości zostawia adres i długość. Koniec bloku nie unieważnia wtedy znaków, ale nazwa po `move` i tak jest martwa, więc stały napis nie jest powodem, żeby używać nazwy po przeniesieniu.

4.9 Podsumowanie

- Blok w nawiasach klamrowych jest regionem. Alokacja dopisuje bajty do bufora, a wyjście z bloku zwalnia bufor w całości.
- Arena w tej książce oznacza bufor regionu. W źródłach kompilatora podobna nazwa oznacza także osobny model tego bufora, którego analiza własności nie używa.

- Kopiowanie, współdzielenie, przeniesienie i ograniczone pożyczanie są czterema sposobami użycia wartości spoza bieżącego regionu.
- Przeniesienie unieważnia nazwę. Nie porządkuje bajtów w starym buforze.
- Pętla nie przenosi nazwy utworzonej poza jej ciałem i czyści arenę ciała przy każdym obiegu.
- Pojemność jednego bufora wynosi 4096 bajtów. Przepełnienie przerywa program.
- Ogólnych referencji ani pełnego borrow checkera nie ma. Ograniczone &T działa dla parametrów i stałych widoków, ale referencji nie można zwracać ani przechowywać w polach. Zwrot świeżo zbudowanego napisu z funkcji też jeszcze nie działa, bo brakuje bufora należącego do wywołującego.

5 Rozdział 3. Z czego składa się plik źródłowy

5.1 Ten rozdział obejmuje

- jak wygląda plik z programem i jak zapisuje się komentarz
- dlaczego nowa linia rozdziela instrukcje
- czym różni się nazwa stała od nazwy zmiennej
- kiedy kilka par nawiasów klamrowych staje się jednym regionem
- dlaczego wywołanie funkcji wolno zapisać w wielu wierszach

5.2 Plik jest listą funkcji

Plik o rozszerzeniu `.bork` zawiera zero lub więcej funkcji. Pusty plik oraz plik złożony z samej pustej linii dają się odczytać jako program bez funkcji. Komentarz zaczyna się od `//` i trwa do końca wiersza. Nie ma komentarza blokowego, atrybutów ani polecenia `import`.

Listing 3.1. Komentarz nie zmienia wyniku funkcji.

```
fun main(): i32 {
    // komentarz
    val n = 1 // też komentarz
    return n
}
```

Program został zbudowany. Proces kończy się kodem wyjścia 1, bo funkcja zwraca liczbę 1.

Słowa rozpoznawane przez lekser jako osobne tokeny to `if`, `fun`, `val`, `var`, `for`, `in`, `return`, `Some`, `None`, `move` i `promote`. Słowa `while`, `break`, `continue`, `true` i `false`

są w gramatyce zapisane wprost. Nie użyjesz ich jako nazw. Słowo `else` ma osobną regułę. Przed nim wolno złamać wiersz i wstawić komentarz, a parser i tak widzi `else`, nie osobny koniec instrukcji. Dzięki temu warunek może mieć drugą gałąź w następnym wierszu.

Nazwa składa się z liter alfabetu łacińskiego, cyfr i znaku podkreślenia, a nie może zaczynać się od cyfry. Polskie znaki w nazwach nie są przyjmowane.

5.3 Nowa linia kończy instrukcję

Listing 3.2. Średnik nie jest częścią języka.

Źródło `val n = 1;` kończy się błędem fazy `parse`. Komunikat mówi, że token średnika jest nieoczekiwany i że w tym miejscu parser spodziewa się między innymi nowej linii albo końca bloku.

Pusta linia między instrukcjami jest w porządku, bo kilka przejść do nowego wiersza skleja się w jeden separator. Dwie instrukcje bez tego separatora nie są poprawne. Gdy komunikat wymienia token `NL`, prawie zawsze dwie rzeczy stoją w jednym wierszu albo w kodzie został średnik przyniesiony z C albo z Rusta.

5.4 Nazwy stałe i nazwy zmienne

Listing 3.3. `val` wprowadza nazwę, której nie wolno przypisać ponownie. `var` wprowadza nazwę, którą wolno zmienić.

```
val n = 1
var count: i32 = 0
var s: String = "hi"
count = count + 1
```

Adnotacja typu po dwukropku jest opcjonalna, gdy wyrażenie po prawej stronie ma znany typ. Przypisanie do nazwy stałej jest błędem sprawdzania typów. Tekst tego błędu był w rozdziale 1. Nazwa wprowadzona w bloku przesłania nazwę z bloku zewnętrznego aż do końca bloku wewnętrznego. Inicjalizator nowej nazwy może jeszcze odczytać przesłanianą nazwę. Program, który na zewnątrz ma `val n = 1`, a w bloku `val n = n + 1` i zwraca to wewnętrzne `n`, został zbudowany i kończy się kodem 2. Drzewo regionów pokazuje w bloku lokalne `n` oraz osobną obserwację, że zewnętrzne `n` zostało skopiowane.

Przypisanie po lewej stronie może dotyczyć tylko nazwy albo elementu tablicy zapisanego jako `nazwa[indeks]`. Pole, wywołanie i dowolne inne wyrażenie po lewej stronie znaku równości nie przechodzą składni. Komunikat gramatyki brzmi `assignment target must be a name or name[index]`. Ten rodzaj błędu, zgłaszany przez samą gramatykę, dostaje pozycję na końcu pliku, a nie przy lewym składniku. To ograniczenie tłumaczenia błędu parsera na diagnostykę, opisane w

`diag::from_parse`. Nie jest decyzją, że błąd „jest na końcu programu”.

Słowo `var` albo `val` przy lokalnej nazwie stoi na początku instrukcji i jest obowiązkowe. Nie ma zapisu `n := 1` ani samego `n = 1` w roli deklaracji. Przy parametrze funkcji te słowa są opcjonalne. Rozdział 5 wyjaśnia, co wtedy znaczą.

5.5 Blok jest instrukcją i otwiera region

Samotna para nawiasów klamrowych w ciele funkcji jest instrukcją. Otwiera region o etykiecie `Block` w drzewie regionów. Kilka warstw nawiasów, które nie zawierają nic poza jednym wewnętrznym blokiem, analiza własności skleja w jeden region.

Listing 3.4. Trzy pary nawiasów, a w drzewie regionów jeden blok.

```
fun main() {  
  val n = 1  
  {  
    {  
      {  
        val m = n  
      }  
    }  
  }  
}
```

Sprawdzenie przechodzi. Wypis drzewa zawiera węzeł `Block` (compacted 2 braces). Zewnętrzna para nawiasów zostaje regionem. Dwie kolejne, które tylko owijają wewnętrzną treść, znikają z drzewa i zostają licznikiem. To nie jest ozdoba wypisu. Generator kodu i analiza własności muszą zdejmować takie opakowania w ten sam sposób. W przeciwnym razie drzewo regionów rozminie się z drzewem programu używanym przy tłumaczeniu na kod. Funkcja `peel_blocks` występuje w dwóch miejscach, w analizie własności i w reprezentacji pośredniej, a komentarz w kodzie każe trzymać je w zgodzie.

Warunek, pętla, funkcja i funkcja dopisana na końcu wywołania nie sklejają się z otoczeniem. Każde z nich ma w drzewie własną etykietę.

5.6 Wyrażenie zapisane jako instrukcja

Każde wyrażenie może stać samodzielnie jako instrukcja. Jego wartość jest wtedy porzucana. Warunek bez drugiej gałęzi jest typowym przykładem. Służy do sterowania, a gdy nikt nie oczekuje od niego wartości, jego typem jest typ pusty. Typ pusty nazywa się `unit`. Funkcja, która nie deklaruje typu wyniku, też ma wynik `unit`.

Listing 3.5. Warunek, który tylko zmienia zmienną.

```
fun main(): i32 {
    var n = 0
    if (true) {
        n = 1
    }
    return n
}
```

Program został zbudowany i kończy się kodem 1. Składnia warunku wymaga nawiasów okrągłych wokół wyrażenia logicznego i wymaga bloków w gałęziach. Nie napiszesz `if n > 0 n else 0`. Gdy warunek ma dać wartość, obie gałęzie są blokami i potrzebna jest gałąź `else`. Szczegóły typów są w rozdziale 6.

5.7 Łamanie wiersza wewnątrz nawiasów

Wywołanie i każda para nawiasów okrągłych może zająć wiele wierszy. Zanim parser zobaczy tekst, funkcja `normalize_parenthesized_newlines` w pliku `src/layout.rs` zamienia znak nowej linii na znak powrotu karetki wewnątrz nawiasów, na tej samej głębokości nawiasów klamrowych, na której nawias został otwarty. Pomija przy tym napisy i komentarze. Lekser traktuje powrót karetki jak zwykły odstęp, więc nowa linia w środku listy argumentów nie kończy instrukcji.

Listing 3.6. Argumenty funkcji w osobnych wierszach.

```
fun add(a: i32, b: i32): i32 {
    return a + b
}

fun main(): i32 {
    return add(
        1,
        2
    )
}
```

Program został zbudowany. Wynik wynosi 3, więc kod wyjścia też wynosi 3. Zamiana znaku nowej linii na powrót karetki nie zmienia długości pliku w bajtach, bo oba znaki zajmują jeden bajt. Pozycje błędów pozostają tam, gdzie były. Testy w `tests/parser.rs` sprawdzają wielowierszowe wywołanie i stabilność miejsca błędu.

Poza nawiasami okrągłymi nowa linia jest znacząca. Nie ma reguły, która ciągnęłaby wyrażenie do następnego wiersza tylko dlatego, że wiersz skończył się operatorem.

5.8 Znak & w ograniczonym pożyczaniu

Znak `&` występuje w dwóch miejscach. W typie zapisuje się nim pożyczony parametr, na przykład `buf: &i32; 2`. W wyrażeniu `&nazwa` tworzy widok na

istniejącą, niekopiowaną nazwę; nie przenosi jej i nie kopiuje zawartości.

```
fun bump(buf: &[i32; 2]) {  
    buf[0] = buf[0] + 1  
}  
  
fun main(): i32 {  
    var a: [i32; 2] = [1, 2]  
    bump(&a)  
    return a[0]  
}
```

Parametr typu `&T` wymaga `&nazwa` przy wywołaniu. Pożyczka jest dozwolona w parametrach i stałych `val`, ale nie w `var`; nie można jej zwrócić z funkcji. `&` przyjmuje nazwę, nie dowolne wyrażenie. Dalsze reguły, w tym różnica między `&String` a `&[T; N]`, są w rozdziałach 4, 5, 7 i 8.

5.9 Czego w składni nie ma

Trzy zapisy, które wyglądają naturalnie, jeśli przychodzisz z innego języka, kończą się błędem fazy parse i kodem wyjścia jeden.

Słowo `struct` na początku pliku daje komunikat, że oczekiwano nowej linii albo słowa `fun`, i tak samo zachowuje się słowo `module`. Zapis `return -1` daje komunikat o nieoczekiwanym tokenie minusa, bo minus jest tylko operatorem między dwiema wartościami, a liczbę ujemną da się uzyskać odejmowaniem, na przykład `0 - 1`. Nie ma literału ujemnego. Funkcji nie zagnieżdża się wewnątrz innych funkcji, bo funkcje stoją wyłącznie na poziomie programu.

5.10 Podsumowanie

- Plik źródłowy jest listą funkcji. Komentarz to tylko `//` do końca wiersza.
- Instrukcję kończy nowa linia. Średnik jest błędem składni.
- Nazwa wprowadzona przez `val` nie przyjmuje późniejszego przypisania. Nazwa wprowadzona przez `var` przyjmuje. Nazwa w bloku może przesłonić nazwę zewnętrzną i w swoim inicjalizatorze może ją jeszcze odczytać.
- Kilka par nawiasów, które tylko owijają jeden blok, składa się na jeden region.
- Wewnątrz nawiasów okrągłych nowa linia jest odstępem. Poza nimi rozdziela instrukcje.
- Nie ma struktur, modułów ani jednoargumentowego minusa. `&` nie oznacza ogólnych referencji takich jak w Ruście; jest ograniczonym zapisem pożyczki opisanym wyżej.

6 Rozdział 4. Typy danych i ich znaczenie

6.1 Ten rozdział obejmuje

- jakie typy wolno napisać i które nazwy są tylko skrótami
- które wartości kompilator kopiuje, a których nie
- jak literal dostaje typ
- jak zapisuje się brak wartości i typ funkcji
- jakie komunikaty pojawiają się, gdy typy do siebie nie pasują

6.2 Nazwy typów

Typ zapisuje się po dwukropku, przy parametrze, przy wyniku funkcji albo przy deklaracji nazwy. Część nazw jest skrótem. Kompilator zamienia skrót na typ właściwy już przy budowie drzewa składni, w funkcji `Type::from_ident`. W dalszych fazach nie ma osobnego typu o nazwie `Int`. Jest trzydziestodwubitowa liczba całkowita ze znakiem.

Zapis w programie	Typ po zamianie
<code>i8, i16, i32, i64</code>	liczba całkowita ze znakiem o podanej szerokości
<code>u8, u16, u32, u64</code>	liczba całkowita bez znaku o podanej szerokości
<code>Int</code>	<code>i32</code>
<code>Long</code>	<code>i64</code>
<code>Byte</code>	<code>u8</code>
<code>Float</code>	<code>f32</code>
<code>Double</code>	<code>f64</code>
<code>f32, f64</code>	liczba zmiennoprzecinkowa
<code>bool</code>	wartość logiczna, literały <code>true</code> i <code>false</code>
<code>unit</code>	typ pusty

Zapis w programie

Typ po zamianie

String	napis
[T; N]	tablica o długości N
&T	pożyczony widok na niekopiowane T, używany w parametrze albo stałej val
T?	wartość, która może być pusta
(A, B) -> R	typ funkcji

Listing 4.1. Skróty typów oraz wartości zmiennoprzecinkowe można zapisać.

```
fun main(): i32 {  
    val n: Int = 1  
    val wide: Long = 2  
    val byte: Byte = 3  
    val f: Float = 1.5  
    val d: Double = 2.25  
    val flag: bool = true  
    return n  
}
```

Sam zapis f32 i f64 oraz arytmetyka zmiennoprzecinkowa są obsługiwane. Przekazanie floatu jako argumentu funkcji jest jawnie odrzucane. Porównanie floatów pozostaje błędem codegenu: frontend je akceptuje, lecz dispatcher widzi wynik bool, wybiera ścieżkę całkowitoliczbową i value_as_int panikuje na FloatValue. Odtworzyłem to na `1.0 < 2.0`; szczegóły są w rozdziale 17.

Każda inna nazwa typu, na przykład Point albo Unit zapisane wielką literą, daje błąd `unknown named type`. Słowo unit małymi literami jest typem pustym. Wielka litera Unit nie jest skrótem.

6.3 Które wartości są kopiowane

Wartość jest kopiowana tylko wtedy, gdy jest typem prostym i nie jest pusta. Poza kopiowaniem zostają napis, każda tablica, każdy typ funkcji, pożyczka &T oraz cokolwiek z pytajnikiem, także i32?. Ta sama odpowiedź obowiązuje przy sprawdzaniu typów i przy analizie własności. Typ, którego nie udało się ustalić, też nie jest traktowany jako kopiowany. Dlatego błąd typu potrafi pociągnąć dodatkowy błąd własności. Drugiego komunikatu nie należy ignorować tylko dlatego, że pierwszy już coś zgłosił.

6.4 Literały

Literał całkowity jest w drzewie składni liczbą ze znakiem o szerokości 64 bitów. Typ, który zostanie w programie, zależy od kontekstu. Jeśli kontekst jest typem całkowitym, literał przyjmuje ten typ. Bez kontekstu staje się `i32`.

Listing 4.2. Literał dopasowany do `i64`. Sprawdzenie przechodzi. Budowanie odrzuca wynik funkcji `main`.

```
fun main(): i64 {  
    val n: i64 = 1  
    return n  
}
```

Komunikat budowania brzmi `main returning i64 is not supported by codegen yet`. Sprawdzanie typów jest zadowolone. Ograniczenie dotyczy tylko tego, co wolno zwrócić z `main`, a nie samego typu `i64` w funkcji pomocniczej. Funkcja, która odejmuje dwie wartości `i64` i porównuje wynik, została zbudowana. Proces kończy się kodem 7. Przykład jest w zestawie programów do rozdziału 17.

Literał `1.5` bez adnotacji staje się `f64`. Z adnotacją `Float` albo `f32` zostaje liczbą o pojedynczej precyzji. Nie ma przyrostka w rodzaju `1.5f32`.

Pożyczony typ `&T` jest dozwolony tylko dla wartości niekopiowanej. Obecnie używa się go przy parametrze funkcji albo przy stałej `val`, na przykład `buf: &[i32; 2]`. Nie ma `&mut`, nullable-reference ani zwrotu referencji. `&[T; N]` pozwala funkcji czytać i zapisywać elementy bufora właściciela; `&String` pozwala czytać napis, ale nie przypisać nowego napisu przez widok. To ograniczone API, nie ogólny system referencji.

Literał napisu używa cudzysłowu. Sekwencje `\n`, `\r`, `\t`, `\\` i `\"` są zamieniane na znak nowej linii, powrót karetki, tabulację, ukośnik i cudzysłów.

Listing 4.3. Napis ze znakami specjalnymi.

```
fun main() {  
    println("a\nb\t\"c\"")  
}
```

Po uruchomieniu na wyjściu jest litera `a`, nowy wiersz, litera `b`, tabulacja, cudzysłów, litera `c`, cudzysłów i jeszcze jeden nowy wiersz.

Operator `+` nie skleja napisów. Wyrażenie `"a" + "b"` daje błąd fazy `type`: operandy arytmetyczne muszą mieć ten sam typ liczbowy, a kompilator widzi dwa napisy. Do sklejania służy funkcja wbudowana `concat`. Jej zasady pamięci są w rozdziale 9.

6.5 Działania na liczbach i porównania

Operatory +, -, * i / wymagają dwóch operandów tego samego typu liczbowego. Nie ma automatycznego rozszerzenia i32 do i64.

Listing 4.4. Dodawanie i64 i i32 jest błędem, i to błędem podwójnym.

```
fun main(): i32 {  
    val a: i64 = 1  
    val b: i32 = 2  
    return a + b  
}
```

Pierwszy komunikat mówi, że operandy mają typy i64 i i32. Drugi, na tym samym miejscu, mówi, że zwracana wartość ma typ i64, a oczekiwano i32. Drugi komunikat jest skutkiem pierwszego. Przy niezgodnych operandach sprawdzanie typów zostawia typ lewej strony jako typ całego wyrażenia, a ten typ nie pasuje do wyniku funkcji.

Porównania >, <, >= i <= wymagają tego samego typu liczbowego. Dwie wartości bool dają komunikat, że uporządkowane porównanie wymaga typu liczbowego. Operatory == i != wymagają identycznego typu, także dla nullable. Porównanie lowerowalnego nullable z None przechodzi sprawdzenie i budowanie; przykłady są w programs/build/conditionals/.

Operatory &&, || i ! działają na wartościach logicznych. Koniunkcja i alternatywa nie liczą prawej strony, gdy lewa już rozstrzyga wynik. Widać to w kodzie maszynowym i w teście builds_logical_short_circuit. Listing w rozdziale 6 uruchamia taki program.

Dzielenie całkowite przez zero przerywa program. Program z return 1 / 0 został zbudowany: budowanie kończy się sukcesem, a uruchomiony proces dostaje sygnał przerwania i w powłoce widać kod 134, bo na standardowym wyjściu błędów nie ma komunikatu Borka, tylko wywołanie funkcji abort z biblioteki języka C. W kodzie generatora jest także strażnik przed dzieleniem najmniejszej liczby typu ze znakiem przez minus jeden. Literału minus jeden nie da się napisać, więc tej drugiej ścieżki nie uruchamiano z poziomu programu w Borku, choć strażnik w funkcji guard_int_div w kodzie jest.

NOTA. Arytmetyka f32/f64 ma ścieżkę codegenu. Porównanie floatów nadal może zakończyć się paniką kompilatora: dispatcher rozpoznaje typ operandu po typie wyniku, a porównanie zwraca bool. Float jako argument funkcji jest osobno odrzucany diagnostyką codegen.

6.6 Brak wartości

Zapis T? oznacza, że wartość typu T może nie istnieć. Część front-endowa

dopuszcza nullable typy proste, napisy i typy funkcji. Zapis nullable tablicy parser przyjmuje, ale sprawdzanie typów odrzuca go komunikatem `nullable array types [T]? are not supported`. Generator kodu obsługuje `String?` i wartości nullable typów prostych poza `unit`, także `f32?` i `f64?`. Nullable `unit` i typy funkcji nie mają obsługiwanej reprezentacji.

Wartość obecną buduje `Some(x)`, a brak wartości zapisuje `None`. Samo `None`, bez oczekiwanego typu, nie przechodzi. Komunikat brzmi `cannot infer type of None`.

Operator `?:` wymaga, żeby lewa strona mogła być pusta. Prawa strona musi pasować do typu po zdjęciu pyłajnika. Zapis `1 ? 0` przy `n` typu `i32` daje `left operand of ?: must be nullable, got i32`. Operator `!!` wymaga wartości, która może być pusta, i daje typ bez pyłajnika. Na zwykłym `i32` dostaniesz `operand of !! must be nullable, got i32`.

Listing 4.5. Operacje nullable na napisie. To fragment funkcji, nie samodzielny program z `main`.

```
fun processUser(name: String?, score: i32): i32 {
    val fallbackName: String = name ?: "Guest"
    val verifiedUser: String? = Some(fallbackName)
    val emptyMiddle: String? = None
    val verifiedLength: i32 = verifiedUser?.length ?: 0
    if (verifiedLength > 0) {
        return score
    }
    return 0
}
```

Zapis `?.` czyta pole tylko wtedy, gdy wartość po lewej nie jest pusta. Na typie, który pusty być nie może, sprawdzanie typów każe użyć zwykłej kropki. Jedyne pole napisu i tablicy nazywa się `length` i ma typ `i32`. Dla tablicy ta długość jest liczbą wpisaną w typ, a nie osobnym licznikiem trzymanym obok danych. Próba odczytu pola `foo` z napisu daje `unknown field foo on type String`.

NOTA. `Some`, `None`, `?:`, `!!`, `?.length` oraz `==/!=` mają codegen dla obsługiwanych typów. Wszystkie sześć przykładów testowych `programs/build/conditionals/nullable_*.bork` zbudowałem i uruchomiłem. Dodatkowy krótki test objął `Some`, `?:`, `!!` oraz równość i nierówność dla `f32?` i `f64?`. Nie oznacza to obsługi każdego `T?: nullable unit` i typy funkcji nadal nie mają reprezentacji, a nullable tablice odrzuca sprawdzanie typów.

Typ funkcji też może dopuszczać brak wartości w frontendzie. Codegen nie ma jednak reprezentacji wartości funkcji, więc `nullable function value` — również po `!!` — pozostaje nieobsługiwane.

6.7 Typ funkcji i nawiasy

Typ funkcji zapisuje się w nawiasach, na przykład `(i32, i32) -> i32`. Wariant, który może być pusty, wymaga dodatkowych nawiasów: `((i32) -> i32)?`.

Gramatyka odrzuca pytajnik przy nawiasach, które nie są typem funkcji.

Komunikaty brzmią `nullable parentheses require one function type` oraz `only function types may be parenthesized for nullability`.

Nie ma krotek jako wartości. Nawias przy typie albo grupuje typ funkcji, albo w wyrażeniu grupuje działanie. Nie napiszesz `(1, 2)` jako pary liczb.

6.8 Typ nieznany

Gdy wyrażenie jest już błędne, sprawdzanie typów podstawia typ nieznany.

Kolejne niezgodności z tym typem są pomijane, żeby jeden błąd nie produkował długiej kaskady. Analiza własności tego pominięcia nie dziedziczy. Typ, którego nie da się przenieść do jej środowiska, zachowuje się jak typ niekopiowany.

Własność nadal może zgłosić błąd.

6.9 Podsumowanie

- Skróty `Int`, `Long`, `Byte`, `Float` i `Double` znikają już przy czytaniu programu. Dalej kompilator widzi typy `i32`, `i64`, `u8`, `f32` i `f64`.
- Kopiowane są niepuste typy proste. Napis, tablica, typ funkcji i każda wartość z pytajnikiem kopiowane nie są.
- Nie ma automatycznego rozszerzania liczb i nie ma operatora `+` dla napisów.
- `None` potrzebuje oczekiwanego typu. Operatory `?:` i `!!` wymagają wartości, która może być pusta.
- Jedyne pole napisu i tablicy nazywa się `length`.
- `&T` jest ograniczonym typem pożyczki dla niekopiowanych wartości; nie można zwracać referencji ani przechowywać jej w `var`.
- Arytmetyka floatów jest lowerowana, porównania floatów pozostają błędem codegeny, a floaty jako argumenty funkcji są odrzucane.
- `Nullable String` i typy proste poza `unit` obsługują `Some`, `None`, `?:`, `!!`, `?.length` oraz `==/!=`; nullable tablice odrzuca sprawdzanie typów, a nullable `unit` i funkcje nie mają reprezentacji.

7 Rozdział 5. Funkcje, parametry i wywołania

7.1 Ten rozdział obejmuje

- jak zadeklarować funkcję, parametr i typ wyniku
- kiedy przy argumencie trzeba napisać `move`
- co robią trzy funkcje wbudowane
- jak dopisać funkcję na końcu wywołania
- jak zadeklarować parametr pożyczany przez `&T`
- co generator kodu robi z argumentem napisowym

7.2 Kształt funkcji

Funkcję zapisuje się jako `fun nazwa(parametry): TypWyniku`, a potem blok ciała. Pominięty typ wyniku oznacza typ pusty `Unit`, a nie nazwę `Unit`. Parametr ma jedną z trzech postaci: `nazwa: Typ`, `val nazwa: Typ` albo `var nazwa: Typ`. Postać bez słowa jest stała, tak jak `val`.

Listing 5.1. Dwa rodzaje parametrów. Sprawdzenie przechodzi. W reprezentacji pośredniej pierwszy parametr jest stały, drugi zmienny, a wynik ma typ `i32`.

```
fun add(val a: Int, var b: Int): Int {  
    return a  
}
```

Słowo `var` przy parametrze znaczy, że wywołujący, który przekazuje wartość niekopiowaną, musi ją oddać słowem `move`. Dla `i32` różnica nie boli, bo liczba jest kopiowana. Dla napisu różnica jest sprawdzana.

Ciało funkcji jest blokiem. Ostatnie wyrażenie w bloku nie staje się samo wynikiem funkcji. Wynik przekazuje się słowem `return`. Jeśli funkcja deklaruje wynik inny niż `Unit`, każda ścieżka musi wykonać `return`.

Listing 5.2. Funkcja z zadeklarowanym wynikiem, która do niego nie wraca.

```
fun f(): i32 {  
    val n = 1  
}  
  
fun main(): i32 {  
    return f()  
}
```

Komunikat nie ma numeru linii, bo błąd dotyczy całej funkcji:

```
missing_ret.bork: error: type: function must return a value of type i32 on all paths
```

Samotne `return` bez wartości ma typ `unit`. W funkcji o wyniku `i32` dostaniesz informację, że pusty powrót ma typ `unit`, a oczekiwano `i32`. `return` z wartością sprawdza zgodność z zadeklarowanym wynikiem.

Nie ma przeciążania. Nazwy funkcji są trzymane w mapie po samym identyfikatorze. Komentarz w `src/sema/env.rs` mówi wprost, że to jest uproszczenie obecnej wersji i że lokalne przesłonięcie wywoływanej funkcji nie jest obsługiwane. Wywołanie funkcji zdefiniowanej niżej w pliku działa, bo sygnatury są zbierane z całego programu, zanim sprawdzane są ciała.

7.3 Parametr pożyczany przez `&T`

Typ parametru może jawnie oznaczać pożyczkę. Wywołujący musi wtedy przekazać nazwę przez `&`, a nie oddać jej własność. Wewnątrz funkcji parametr jest stałym widokiem: `&String` pozwala czytać napis, a `&[T; N]` pozwala także zmieniać elementy tablicy właściciela.

Listing 5.3. Funkcja zmienia element tablicy przez pożyczony parametr.

```
fun bump(buf: &[i32; 2]) {
    buf[0] = buf[0] + 1
}

fun main(): i32 {
    var a: [i32; 2] = [1, 2]
    bump(&a)
    return a[0]
}
```

Nie ma `&mut`. To, że `buf` ma typ `val`, nie zabrania modyfikacji elementów przez parametr tablicowy; widok `&String` pozostaje tylko do odczytu. Pożyczony parametr nie można zwrócić ani zapisać do zmiennej `var`. W zagnieżdżonym `if` lub pętli wolno wykonać bezpośrednie `helper(&buf)`, ale nie wolno przenieść widoku do nowej lokalnej pożyczki w regionie potomnym.

7.4 Własność na granicy wywołania

Dla wartości, która nie jest kopiowana, analiza własności stosuje prostą tabelę. Stałą nazwę wolno przekazać do parametru stałego. Przekazanie jej do parametru zmiennego wymaga `move`. Zmienną nazwę trzeba przenieść zarówno do parametru stałego, jak i do zmiennego. Literał, wywołanie `concat` i inne wyrażenie, które dopiero tworzy wartość, nie wymaga `move`.

Listing 5.4. Literał napisu przekazany do parametru zmiennego.

```
fun f(var a: String) {
```

```
println(a)
}

fun main() {
    f("hello")
}
```

Brak słowa `move` jest zgodny z regułą sprawdzania: świeży literał nie jest nazwą, którą trzeba przenieść. Historyczny przebieg z wcześniejszej wersji kompilatora kończył się paniką `into_int_value`. Bieżący listing zbudowałem i uruchomiłem ponownie: sprawdzenie i budowanie zakończyły się kodem 0, program wypisał `hello` i zakończył się kodem 0. Tabela w `WYNIKI.md` zachowuje stary wynik jako historyczny.

Zwrócenie napisu, który już żyje na poziomie funkcji, sprawdzenie i budowanie akceptują.

Listing 5.5. Zwrot lokalnego napisu i jego wypisanie. Program został uruchomiony. Na wyjściu jest `hi` oraz nowy wiersz.

```
fun shout(): String {
    var s = "hi"
    return s
}

fun main() {
    println(shout())
}
```

Zapis `return "hi"` też działa i daje ten sam wydruk. Zapis `return name` dla parametru przechodzi sprawdzenie. Warto odróżnić te przypadki od świeżego wyniku `concat`, którego zwrot nadal blokuje analiza ucieczki.

Zapis `return concat("a", "b")` jest odrzucany przy sprawdzaniu. Komunikat nie ma numeru linii:

```
err_ret_concat.bork: error: ownership: returning the result of `concat` is not supported yet:
returned `String` bytes must outlive the callee arena
```

Podobny komunikat dotyczy `return move`. Tekst radzi użyć `return` ze zwykłą nazwą parametru albo nazwy lokalnej, albo zwrócić literał.

7.5 Funkcje wbudowane

Trzy nazwy nie są funkcjami, które wolno zdefiniować. Próba deklaracji jest błędem typu, zanim ciało w ogóle ma znaczenie. Dla `fun println(n: i32)` komunikat brzmi `cannot redefine builtin function println`.

`print` wypisuje jeden argument i opróżnia bufor wyjścia. `println` robi to samo i dodaje nowy wiersz. Argumentem może być `i32`, `i64` albo napis, chociaż w

wewnętrznej tabeli sygnatura nominalna mówi o jednym `i32`. To jest specjalny przypadek, nie ogólna zasada, że każdy parametr `i32` przyjmie napis. `concat` przyjmuje dwa napisy i zwraca jeden nowy. Bufor wyniku powstaje w arenie miejsca, do którego wynik jest zapisywany. Gdy takiego miejsca nie ma, powstaje w arenie bieżącego bloku.

Listing 5.6. Kolejność wypisywania. Program został uruchomiony. Na wyjściu jest dokładnie cyfra 1, nowy wiersz, słowo `tail` i cyfra 7, bez końcowego nowego wiersza.

```
fun main() {  
  println(1)  
  print("tail")  
  print(7)  
}
```

7.6 Funkcja dopisana na końcu wywołania

Ostatni argument może być blokiem stojącym po nawiasie wywołania. Parametry tego bloku zapisuje się przed strzałką `->`.

Listing 5.7. Ostatni parametr ma typ funkcji. Sprawdzenie przechodzi. Budowanie odrzuca konstrukcję.

```
fun action(a: Int, b: Int, block: (Int, Int) -> Int): Int {  
  return block(a, b)  
}  
  
fun main(): i32 {  
  val threshold = 5  
  return action(1, 2) { acc, current ->  
    if (current > threshold) { acc + current } else { acc }  
  }  
}
```

Sprawdzanie typów pilnuje trzech rzeczy. Ostatni parametr wywoływanej funkcji musi być typem funkcji. Liczba parametrów bloku musi się zgadzać. Typ wyniku bloku musi pasować do typu wyniku tego parametru. Komunikaty mówią o tym wprost, po angielsku, słowami `trailing closure`. Wywołanie, którego celem nie jest nazwa funkcji, dostaje komunikat `only named functions can be called`. Generator kodu i tak odrzuca wywołanie pośrednie osobnym tekstem.

Analiza własności nie dostaje typów parametrów takiego bloku. Wpisuje im typ nieznany. Skutek widać w drzewie regionów. Parametry bloku są lokalne, a ich odczyt w gałęzi warunku bywa oznaczony jako współdzielenie, mimo że `Int` jest kopiowany. Program mimo to przechodzi sprawdzenie. Drzewo jest w tym miejscu mylące. Nie jest to błąd twojego programu.

Bloku funkcji nie zapiszesz w stałej poza wywołaniem. Gramatyka ma taką funkcję

tylko jako argument końcowy albo jako blok `move`. Typ funkcji istnieje, ale nie ma osobnego literału funkcji.

7.7 Trzy formy przeniesienia przy bloku

Zwykły blok po wywołaniu niczego nie przenosi. Odczyt zewnętrznej zmiennej napisowej jest wtedy błędem. Zapis `move ( )` przed blokiem też niczego nie przenosi, ale robi to jawnie. Zapis `move (a, b)` przenosi dokładnie wymienione nazwy. Zapis `move` bez nawiasu przenosi każdą żywą, niekopiowaną nazwę z regionu zewnętrznego, której blok używa.

To samo da się napisać jako instrukcję, bez wywołania. Rozdział 8 ma zbudowane programy dla tych instrukcji. Formy stojące po wywołaniu odpadają przy budowaniu komunikatem `trailing closures are not supported by codegen`.

Wnioskowanie, które nazwy przenieść, nie patrzy na wyrażenia `move nazwa i promote nazwa` jak na zwykłe użycie nazwy. W przeciwnym razie przeniesienie zapisane w wyrażeniu byłoby liczone drugi raz jako przeniesienie całego bloku.

7.8 Co z tego wynika dla programów, które mają się zbudować

Ciało wywołanej funkcji ma własne regiony. Argumenty są liczone w regionie wywołującego. Wynik liczbowy wraca w rejestrze. Wynik napisowy nie dostaje dziś bufora należącego do wywołującego, dlatego analiza ucieczki zabrania zwrócić napis utworzony w regionie tej funkcji. Listing 5.5 działa, bo napis `"hi"` jest literałem albo wartością na poziomie funkcji, a `println` tylko czyta adres i długość.

WSKAZÓWKA. Nie wszystkie ograniczenia wywołań są takie same. Aktualny kod ma ścieżkę przekazywania deskryptorów `String`, ale `float` jako argument funkcji jest nadal odrzucany w `coerce_value_to_ty`. Nie traktuj historycznej paniki na napisie jako aktualnej reguły języka.

7.9 Podsumowanie

- Wynik funkcji zapisuje się słowem `return`. Brak powrotu na którejś ścieżce jest błędem typu.
- Parametr bez `val` i bez `var` jest stały. Parametr zmienny wymaga `move`, gdy przekazuje się istniejącą nazwę wartości niekopiowanej.
- Świeży literał nie wymaga `move`. Deskryptor napisu jest przekazywany do funkcji użytkownika; `float` jako argument funkcji jest odrzucany osobną diagnostyką codegenu.
- Parametr `&T` jawnie pożycza niekopiowaną wartość; wywołanie wymaga `&nazwa`.

- `print`, `println` i `concat` są wbudowane. Nie wolno ich zadeklarować ponownie.
- Funkcja dopisana na końcu wywołania jest sprawdzana i nie jest tłumaczona na kod maszynowy.
- Analiza własności nie zna typów parametrów takiej funkcji, więc drzewo regionów potrafi pokazać współdzielenie tam, gdzie typ jest kopiowany.

8 Rozdział 6. Warunki, pętle i powrót z funkcji

8.1 Ten rozdział obejmuje

- kiedy warunek daje wartość, a kiedy tylko steruje
- jak działa pętla `for` po zakresie liczb
- jak działają `while`, `break` i `continue`
- dlaczego `&&` i `||` nie zawsze liczą prawą stronę
- czego nie wolno zwrócić z wewnętrznego bloku

8.2 Warunek

Warunek zapisuje się jako `if (wyrażenie) { blok }`, opcjonalnie z `else { blok }`. Wyrażenie w nawiasie musi mieć typ `bool`. Obie gałęzie, jeśli druga istnieje, są blokami. Typ wyniku zależy od tego, czy ktoś oczekuje wartości, i od tego, czy jest `else`. Reguły są w funkcji `check_if` w pliku `src/typeck/expr/control.rs`.

Gdy nie ma `else` i nikt nie oczekuje typu, wynik ma typ `unit`. Gdy nie ma `else`, a oczekiwany typ jest, na przykład przez adnotację `val x: i32`, kompilator zgłasza `value-producing if expression requires an else branch`. Gdy `else` jest i obie gałęzie mają ten sam typ, wynik ma ten typ. Gdy typy gałęzi się różnią, a wynik jest potrzebny, dostaniesz `else branch has type ..., expected ...`. Gdy `else` jest, ale całe wyrażenie stoi w miejscu, które nie prosi o wartość, wynik i tak jest `unit`. Wartości gałęzi są sprawdzane, ale nie wpływają na zewnątrz.

Listing 6.1. Warunek użyty jako wartość. Program został zbudowany i kończy się kodem 42.

```
fun pick(flag: i32): i32 {
    val v = if (flag == 1) { 40 } else { 7 }
    return v + 2
}

fun main(): i32 {
```

```
    return pick(1)
}
```

Nazwa `v` nie ma adnotacji. Obie gałęzie mają typ `i32`, więc cały warunek też ma typ `i32`. W drzewie regionów gałęzie są osobnymi węzłami `IfThen` oraz `IfElse`.

Listing 6.2. Brak `else` przy oczekiwaniu liczby.

```
fun main(): i32 {
    val x: i32 = if (1 > 0) { 1 }
    return x
}
```

Komunikat nie wskazuje numeru linii:

```
if_ann.bork: error: type: value-producing if expression requires an else branch
```

Bez adnotacji, przy `val x = if (n > 0) { n }`, błąd jest inny. Warunek staje się typem `unit`, bo nikt nie oczekiwał konkretnego typu, a dopiero `return x` mówi, że zwracana wartość ma typ `unit`, podczas gdy oczekiwano `i32`. Komunikat o brakującym `else` pojawia się wtedy, gdy oczekiwany typ jest znany.

Gałąź warunku jest regionem. Napis zbudowany tylko w jednej gałęzi nie może być wartością całego warunku, bo arena gałęzi ginie wraz z jej końcem. Analiza ucieczki mówi wtedy, że napis przeniesiony wewnątrz gałęzi nie może być wartością warunku, bo arena gałęzi jest zwalniana. Tekst komunikatu wymienia napis, choć warunek w kodzie patrzy szerzej, na każdą wartość trzymaną w arenie, a więc także na tablicę.

Analiza własności scala stan przeniesienia po warunku. Nazwa jest przeniesiona po całym `if` wtedy, gdy była przeniesiona już wcześniej albo gdy przeniosły ją obie gałęzie. Samo przeniesienie w gałęzi `then`, bez `else`, nie zostawia nazwy martwej po warunku. Inaczej gałąź, która może się nie wykonać, zabijałaby zmienną na stałe. Test `if_then_only_move_without_else_does_not_stick` zamyka tę zasadę.

8.3 Pętla po zakresie

Pętlę po liczbach zapisuje się jako `for` (nazwa `in` początek..`koniec`). Zakres jest otwarty z prawej strony. Liczba `koniec` nie wchodzi do obiegu. Granice mają typ `i32` i są liczone raz. Nazwa pętli ma w sprawdzaniu typów typ `i32`.

Listing 6.3. Suma liczb od zera do czterech włącznie. Program został zbudowany i kończy się kodem 10.

```
fun main(): i32 {
    var total = 0
    for (i in 0..5) {
        total = total + i
    }
}
```

```
    return total
}
```

W drzewie regionów ciało pętli jest węzłem `ForLoop (i)`. Nazwa `i` jest lokalna. Nazwa `total` wewnątrz pętli jest oznaczona jako kopiowana, bo `i32` jest kopiowane. Przypisanie `total = ...` zmienia zmienną z regionu funkcji. Nie jest przeniesieniem. Generator kodu wchodzi do regionu ciała raz, a przy każdym obiegu czyści arenę tego ciała, zamiast oddawać bufor do puli i brać nowy. Dzięki temu pętla nie zużywa nowego bufora na każdy obieg.

Zakres o granicach `i64` jest odrzucany. Komunikat mówi, że granice zakresu muszą mieć typ `i32`. Pętla, której prawa strona nie jest zakresem, dostaje `for-loop iterator must be a range`. Nie ma pętli po elementach tablicy.

8.4 Pętla dopóki i sterowanie obiegiem

Listing 6.4. Przerwanie pętli, gdy licznik dojdzie do trzech. Program został zbudowany i kończy się kodem 3.

```
fun main(): i32 {
    var i: i32 = 0
    while (i < 10) {
        if (i == 3) {
            break
        }
        i = i + 1
    }
    return i
}
```

Warunek `while` musi mieć typ `bool`. `break` i `continue` poza pętlą są błędem fazy `type`, nie osobnej fazy sterowania. Dla samotnego `break` w `main` komunikat brzmi ``break` outside of a loop`. Sprawdzanie typów pamięta głębokość zagnieżdżenia pętli i podnosi ją zarówno przy `for`, jak i przy `while`.

Listing 6.5. `continue` pomija dodanie dwójki. Suma wynosi $0 + 1 + 3 + 4$, czyli 8. Program został zbudowany i kończy się tym kodem.

```
fun main(): i32 {
    var s = 0
    for (i in 0..5) {
        if (i == 2) {
            continue
        }
        s = s + i
    }
    return s
}
```

`continue` działa także w `while`. Nie ma klauzuli `else` przy pętli. Pętlę bez warunku końca zapisuje się jako `while (true)`.

8.5 Koniunkcja i alternatywa

Listing 6.6. Prawa strona koniunkcji nie powinna zmienić zmiennej, bo lewa strona jest fałszywa. Program został zbudowany i kończy się kodem 0.

```
fun side(): i32 {
    return 1
}

fun main(): i32 {
    var n: i32 = 0
    if (false && side() == 1) {
        n = 1
    }
    if (true || side() == 1) {
        return n
    }
    return 99
}
```

Funkcja `side` nie wypisuje nic, więc sam wydruk nie pokazuje, że prawe wywołanie zostało pominięte. Pokazuje to wynik: `n` zostaje zerem, a druga instrukcja wraca od razu, bo `true || ...` nie musi liczyć prawej strony. W kodzie generatora `&&` i `||` dostają osobne bloki. Spacer, który uzgadnia drzewo regionów z generowaniem kodu, nie wchodzi w prawy operand zwykłą ścieżką, gdy ta ścieżka jest już odłożona do osobnego bloku. Dzięki temu decyzja, czy region potrzebuje własnego bufora, pozostaje zgodna z tym, co naprawdę zostanie wyemitowane.

Operator `!` wymaga wartości `bool`. Program, który liczy `val flag = !false` i zwraca 1 albo 0 w zależności od `flag`, został zbudowany i kończy się kodem 1.

8.6 Wczesny powrót i regiony

Powrót z wnętrza bloku musi zwolnić regiony, które funkcja otworzyła. Generator kodu zdejmuje je przy `return` i nie zdejmuje ich drugi raz na ścieżce, która już wróciła. Seria poprawek w historii repozytorium dotyczyła właśnie podwójnego zwolnienia albo zostawienia uchwytu bufora. Test `builds_nested_loops_with_regions_and_early_return` zwraca 63 i pilnuje, że wczesny powrót z zagnieżdżonych pętli daje oczekiwany wynik.

Dla programisty skutek jest taki. `return` w środku bloku jest legalny dla wartości kopiowanej i dla napisu, który żyje na poziomie funkcji. Nie jest legalny dla napisu, którego pamięć leży w głębszym regionie.

Listing 6.7. Zwrot napisu utworzonego w bloku wewnętrznym. Sprawdzenie i budowanie kończą się kodem jeden. Plik wykonywalny nie powstaje.

```
fun mk(): String {
    var s = "esc"
```

```

    {
        val x = move s
        return x
    }
}

fun main() {
    println(mk())
}

```

Komunikat fazy ownership mówi, że zwracany jest napis, którego pamięć leży w regionie wewnętrznym, i że ta arena jest zwalniana, zanim wartość zostanie zwrócona. Faza nazywa się tak samo jak faza analizy własności, bo analiza ucieczki dokłada błędy do tej samej grupy. Gdy analiza ucieczki zgłosi błąd, reprezentacja pośrednia jest wyrzucana. Drzewo regionów zostaje.

8.7 Podsumowanie

- Warunek bez else, użyty tam, gdzie nikt nie oczekuje wartości, ma typ `unit`.
- Gdy oczekiwany typ jest znany, brak else jest osobnym błędem.
- Przeniesienie w jednej gałęzi nie zabija nazwy po warunku, jeśli druga gałąź jej nie ruszyła.
- Pętla for idzie po zakresie liczb `i32`, otwartym z prawej strony, i czyści arenę ciała przy każdym obiegu.
- `break` i `continue` poza pętlą są błędem typu.
- `&&` i `||` nie liczą prawej strony, gdy lewa już rozstrzyga wynik.
- Nie zwrócisz napisu z bloku wewnętrznego. Kompilator odrzuca taki program, zanim powstanie plik wykonywalny.

9 Rozdział 7. Tablice o znanej z góry długości

9.1 Ten rozdział obejmuje

- jak zapisać typ tablicy i literal tablicy
- czym różni się odczyt elementu od wycinka o stałych granicach
- jak czytać długość
- jak własność dotyczy całej tablicy i pojedynczego napisu w tablicy
- co dzieje się w czasie działania, gdy indeks nie mieści się w tablicy

9.2 Dlaczego długość należy do typu tablicy

Tablicę zapisuje się jako `[T; N]`. `T` jest typem elementu, a `N` jest liczbową

długością. Długość nie jest wyrażeniem liczonym w czasie działania. Jest częścią typu, tak jak szerokość liczby `i32` jest częścią typu, a nie wartością trzymaną obok liczby. Tablica trzech liczb `i32` ma inny typ niż tablica dwóch liczb `i32`. Nie przypiszesz jednej do drugiej.

`N` w zapisie typu musi być literałem całkowitym. Za duża liczba, która nie mieści się w `u32`, daje błąd gramatyki `array length is too large`. Pusta tablica nie ma skąd wziąć ani typu elementu, ani długości.

Listing 7.1. Pusta tablica z jawnym typem. Program został zbudowany. Długość wynosi zero, więc kod wyjścia też wynosi zero.

```
fun main(): i32 {  
    val a: [i32; 0] = []  
    return a.length  
}
```

Bez adnotacji zapis `val a = []` daje komunikat, że pusty literał wymaga jawnego typu, i podaje przykład `val a: [i32; 0] = []`.

Elementy literału muszą mieć jeden typ, a ich liczba musi równać się `N`.

Listing 7.2. Trzy elementy w tablicy, która według typu ma dwa.

Sprawdzenie wypisuje dwa błędy fazy `type`. Pierwszy mówi, że literał ma 3 elementy, a oczekiwano 2. Drugi mówi, że inicjalizator nazwy `a` ma typ `[i32; 3]`, a oczekiwano `[i32; 2]`. Źródło to `val a: [i32; 2] = [1, 2, 3]`.

Dozwolony element to wartość kopiowana albo niepusty napis. Napis, który może być pusty, nie może być elementem. Tablica, która sama może być pusta w sensie `?`, też jest odrzucana.

Tablica nie jest kopiowana, nawet gdy jej elementy są. Przypisanie całych tablic wymaga identycznego typu elementu i identycznej długości.

9.3 Indeks

Indeks jest typu `i32` i jest liczony dopiero przy uruchomieniu, więc nie wchodzi do typu i sprawdzenie nie odrzuca `a[9]` na tablicy trzelementowej. Taki program został zbudowany: plik wykonywalny powstaje, a uruchomienie kończy się sygnałem przerwania, z kodem powłoki 134, bo generator kodu wstawia sprawdzenie zakresu i woła `abort`.

Listing 7.3. Zapis elementu i wycinek, który widzi ten sam bufor. Program został zbudowany i kończy się kodem 12.

```
fun main(): i32 {
```

```

var a: [i32; 3] = [1, 2, 3]
a[1] = 9
val b = a[0..2]
return b[1] + a.length
}

```

Po zapisie pod indeksem 1 leży 9. Wycinek od zera do dwóch, bez dwójki, obejmuje dwa pierwsze elementy i nie kopiuje ich do nowego bufora. Dlatego `b[1]` też jest 9. Długość `a` wynosi 3. Suma wynosi 12 i tyle wynosi kod wyjścia.

Nazwa wprowadzona przez `val` nie przyjmuje zapisu elementu. Komunikat jest ten sam co przy zwykłym przypisaniu do stałej: nie można przypisać do niezmienniej nazwy `a`. Indeks złego typu daje `array index must be i32` albo, przy zapisie, informację, że indeks ma inny typ, a oczekiwano `i32`.

9.4 Pożyczony parametr tablicowy

Funkcja, która ma pracować na buforze wywołującego, może przyjąć `&[T; N]`. Przy wywołaniu trzeba jawnie napisać `&nazwa`. Parametr może czytać elementy i przypisywać do nich; zmiana jest widoczna w tablicy właściciela. To specjalna, ograniczona reguła Borka — nie odpowiada rozróżnieniu Rustowego `&[T]` i `&mut [T]`.

```

fun bump(buf: &[i32; 2]) {
    buf[0] = buf[0] + 1
}

fun main(): i32 {
    var a: [i32; 2] = [1, 2]
    bump(&a)
    return a[0]
}

```

Pożyczkę można ponowić bezpośrednio w argumencie funkcji, również wewnątrz `if`, `while` lub `for`, na przykład `bump(&buf)`. Nie można natomiast utworzyć w takim regionie potomnym nowej lokalnej pożyczki `val view = &buf`. Przykłady `build/check` dla tych ścieżek są w `programs/build/borrow/` i `programs/check/borrow/`.

9.5 Wycinek o stałych granicach

Zapis `a[początek..koniec]` wymaga, żeby obie granice były literałami typu `i32`. Typ wyniku to tablica o długości `koniec - początek`. Wycinek nie kopiuje bufora. Nowy opis tablicy wskazuje w środek starej, a długość bierze z typu.

Wycinek `[1..9]` na tablicy `[i32; 3]` jest błędem typu: `slice [1..9] is out of bounds`. Granice, które nie są literałami, dają komunikat, że granice muszą być literałami całkowitymi, żeby typ wyniku miał znaną długość. Generator kodu nie wstawia drugiego sprawdzenia wycinka w czasie działania. Ufa sprawdzeniu

typów. Indeks jest sprawdzany w czasie działania, bo nie da się go wpisać do typu.

Listing 7.4. Wycinek i jego długość. Program został zbudowany. Na wyjściu są wiersze 20 oraz 2.

```
fun main() {  
    val a = [10, 20, 30]  
    val b = a[0..2]  
    println(b[1])  
    println(b.length)  
}
```

Analiza ucieczki traktuje wycinek jak tablicę, z której powstał. Wycinek nie może przeżyć areny, która trzyma bufor. Test `rejects_slice_escaping_inner_region` oczekuje fazy `ownership` i tekstu `inner region`.

9.6 Napis jako element

Odczyt `a[i]`, gdy element jest napisem, jest oglądaniem napisu leżącego w buforze tablicy. Nie ma przeniesienia jednego elementu. Słowa `move` i `promote` dotyczą całej tablicy albo, przy zapisie do elementu, wartości po prawej stronie.

Zapis elementu, który nie jest kopiowany, wymaga `move` albo `promote` po prawej stronie. Bufor tablicy jest miejscem, w którym ląduje treść nowego elementu.

Listing 7.5. Zapis przeniesionego napisu do tablicy. Program został zbudowany. Na wyjściu jest `z` oraz nowy wiersz.

```
fun main() {  
    var a: [String; 2] = ["a", "b"]  
    var s = "z"  
    a[0] = move s  
    println(a[0])  
}
```

Bez `move` sprawdzenie mówi, że własność trzeba przenieść. Po `move` nazwa `s` jest martwa. W drzewie regionów `s` jest oznaczone jako przeniesione z regionu funkcji `main`.

9.7 Jak tablica wygląda w gotowym programie

W reprezentacji LLVM tablica i napis mają ten sam kształt. Jest to para: adres bufora i długość zapisana na 64 bitach. Dla tablicy długość w tej parze równa się `N` z typu. Bajty elementów leżą w arenie, w której tablica powstała, albo w arenie zmiennej, do której tablica jest od razu zapisywana.

Pole `length` w języku ma typ `i32`. Generator kodu wycina długość z pary i obcina ją do 32 bitów. Dopóki `N` mieści się w `i32`, wynik jest zgodny z typem. Gramatyka

przyjmuje większe N, bo długość w typie jest liczbą u32. Testy nie sprawdzają tablicy dłuższej niż maksymalna wartość i32. Nie zakładaj, że pole length takiej tablicy jest dobrze określone.

9.8 Podsumowanie

- Długość tablicy jest częścią jej typu. Literał musi mieć dokładnie tyle elementów, ile mówi typ.
- Pusty literał `[]` wymaga adnotacji typu.
- Indeks spoza zakresu przerywa program. Zły wycinek jest błędem kompilacji.
- Wycinek o stałych granicach wskazuje w istniejący bufor i ma typ tablicy o długości równej różnicy granic.
- Element napisowy czyta się bez kopiowania. Zapis takiego elementu wymaga `move`.
- Całą tablicę wolno przenieść tylko do tablicy o tym samym typie elementu i tej samej długości.
- Parametr `&[T; N]` daje funkcji widok na bufor wywołującego; indeksy można w nim czytać i zapisywać.

10 Rozdział 8. Kopiowanie, współdzielenie i przeniesienie wartości

10.1 Ten rozdział obejmuje

- jak czytać drzewo regionów
- jak przenieść wartość słowem `move` i jak słowem `promote` skopiować ją do zmiennej z regionu zewnętrznego
- czym różnią się trzy formy bloku `move`
- dlaczego przypisanie do zmiennej z zewnątrz bloku nie jest odczytem tej zmiennej
- które błędy pochodzą z nazw, a które z tego, jak długo żyje pamięć napisu

10.2 Jedna faza dla nazw i dla bufora

Własność nazw sprawdza analiza w katalogu `src/sema`. To, czy pamięć napisu przeżyje użycie, sprawdza osobne przejście, analiza ucieczki, w pliku `src/escape.rs`. Analiza ucieczki uruchamia się tylko wtedy, gdy sprawdzanie typów i analiza nazw nie zgłosiły wcześniej żadnego błędu. Oba źródła drukują fazę `ownership`. Dlatego program, który przeszedł analizę nazw, nie musi jeszcze spełniać wszystkich reguł napisów. Znaczy to tyle, że nazwy nie są użyte po

przeniesieniu i że wartość niekopiowana nie ucieka przez gołą nazwę. Czas życia napisu sprawdza dopiero analiza ucieczki.

Drzewo regionów zostaje nawet przy błędach. Reprezentacja pośrednia zostaje tylko wtedy, gdy lista błędów jest pusta, i to po analizie ucieczki. Przy błędzie składni nie ma nawet drzewa.

10.3 Znaczniki w drzewie

Wypis `--dump-arenas` opisuje każdą nazwę krótkim znacznikiem. `Local` oznacza deklarację w tym regionie albo parametr. `Copy` oznacza odczyt wartości kopiowanej z regionu otaczającego. `Shared` z nazwą regionu oznacza odczyt stałej, która nie jest kopiowana. `Moved` z nazwą regionu oznacza, że własność została zabrana z tamtego regionu.

Węzły mają etykiety `fun nazwa`, `Block`, `ForLoop (i)`, `WhileLoop`, `IfThen`, `IfElse`, `MoveBlock (move)`, `Closure i Closure (move)`. Sklejone nawiasy dopisują do etykiety informację, ile par zostało scalonych.

10.4 Przeniesienie zapisane przy nazwie

Zapis `move nazwa` zużywa nazwę. Późniejsze użycie daje `use of nazwa after move from etykieta`.

Sam zapis zmiennej napisowej po prawej stronie przypisania albo w argumencie nie dopisuje `move` za programistę.

Listing 8.1. Brak `move` przy przypisaniu napisu.

```
fun main() {  
    var s: String = "hi"  
    var x = s  
}
```

Komunikat brzmi `use move s to transfer ownership`. W wywołaniu tekst jest inny: `use move s to pass ownership`. Gdy analiza widzi przeniesienie między różnymi regionami, potrafi powiedzieć, że nazwa nie jest kopiowana i trzeba ją przenieść do nazwanego regionu słowem `move`.

Listing 8.2. Przeniesienie i wypisanie. Program został zbudowany. Na wyjściu są wiersze `ab` oraz `42`.

```
fun main() {  
    var s = "ab"  
    val t = move s  
    println(t)  
    print(4)  
    println(2)  
}
```

```
}
```

W drzewie `s` jest przeniesione z regionu `fun main`, a `t` jest nazwą lokalną.

10.5 Słowo `promote`

`promote` nazwa jest legalne tylko po prawej stronie przypisania do zmiennej z regionu ściśle zewnętrznego. Kopiuje treść napisu do bufora tej zmiennej i unieważnia nazwę źródłową.

Listing 8.3. Podniesienie napisu, który już powstał w bloku wewnętrznym. Program został zbudowany. Na wyjściu jest `temp` oraz nowy wiersz.

```
fun main() {  
    var outer = "a"  
    {  
        var held = "temp"  
        outer = promote held  
    }  
    println(outer)  
}
```

W drzewie nazwa `held` pojawia się w bloku dwa razy. Raz jako deklaracja lokalna, raz jako przeniesienie po `promote`.

Są trzy typowe błędy. Użycie `promote` poza przypisaniem do zmiennej zewnętrznej daje komunikat, że `promote` jest legalne tylko przy takim przypisaniu, a użycie na wartości kopiowanej daje komunikat, że `promote` nie jest potrzebne. Użycie w tym samym regionie, na przykład `val x = promote held` obok deklaracji `held`, daje komunikat, że `promote held` nie może trafić do `fun main`, bo wartość już żyje w tej arenie albo jeszcze wyżej. Ostatni komunikat został potwierdzony uruchomieniem kompilatora.

`promote` nie występuje przy `return`. Nie ma miejsca przeznaczenia, do którego analiza mogłaby skopiować treść wyniku.

10.6 Przypisanie do zmiennej z zewnątrz

Zapis `outer = wartość`, gdy `outer` jest zmienną z regionu otaczającego i nie została wcześniej przeniesiona, nie jest odczytem tej zmiennej w bloku wewnętrznym. Analiza nie traktuje lewej strony jak użycia, które wymagałoby `move`. Prawa strona dostaje jako miejsce alokacji arenę zmiennej `outer`, a nie arenę bloku, w którym stoi przypisanie.

Listing 8.4. Przypisanie literału do zmiennej z zewnątrz. Program został zbudowany. Na wyjściu jest `b` oraz nowy wiersz. W bloku wewnętrznym nie ma żadnej nazwy lokalnej.

```
fun main() {
    var outer = "a"
    {
        outer = "b"
    }
    println(outer)
}
```

To jest zmiana istniejącej zmiennej. Nie potrzebujesz nazwy pośredniej. W tym przykładzie dokument docs/language.md i kompilator mówią to samo.

10.7 Blok, który przenosi kilka nazw naraz

Listing 8.5. Dwa bloki move. Program został zbudowany. Na wyjściu są wiersze A oraz B.

```
fun main() {
    var a: String = "A"
    var b: String = "B"
    move (a) {
        val x = a
        println(x)
    }
    move {
        val y = b
        println(y)
    }
}
```

Pierwszy blok wymienia a w nawiasie. Wewnątrz nazwa a jest już lokalna. Nie pisze się przy niej drugi raz move. Drugi blok nie ma listy. Kompilator przenosi te niekopiowane nazwy z zewnątrz, których blok używa. Używa b, więc przenosi b. W drzewie oba bloki nazywają się MoveBlock (move). Przy każdym widać, skąd nazwa została przeniesiona.

Nazwy z listy są już w regionie bloku. Ponowne move takiej nazwy w tym samym regionie jest błędem. Komunikat mówi, że nazwa została już przeniesiona do tego regionu jako przechwycenie.

Zapis move () { } nie zgaduje listy. Nic nie jest przenoszone. Odczyt zewnętrznej zmiennej napisowej wewnątrz takiego bloku jest zwykłym błędem mówiącym, że wartość nie jest kopiowana.

Jawna lista wygrywa ze zgadywaniem. Pusta jawna lista wyłącza zgadywanie. W drzewie składni różnica jest między brakiem listy a listą pustą. Nie wolno tych dwóch zapisów utożsamić, gdy będziesz czytał parser.

10.8 Pętla jeszcze raz

Zakaz z rozdziału 2 dotyczy także bloku move. Nazwę utworzoną w ciele pętli

wolno przenieść.

```
for (i in 1..10) {  
  var a: String = "A"  
  move (a) {  
    val t = a  
  }  
}
```

Test `move_loop_local_binding_is_ok` pilnuje, że takie przeniesienie przechodzi. Każdy obieg ma świeże `a`. Ta sama nazwa utworzona przed pętlą byłaby błędem.

10.9 Stała i zmienna

Odczyt stałej niekopiowanej w regionie wewnętrznym jest współdzieleniem i jest poprawny. Pokazuje to listing 2.2. Odczyt zmiennej niekopiowanej bez `move` nadal jest błędem, ale obecnie istnieje trzecia, jawna możliwość: ograniczona pożyczka przez `&nazwa`.

Pożyczka nie przenosi własności ani nie kopiuje danych. Można ją przekazać do parametru `&T` albo związać ze stałą `val` w dozwolonym regionie. Wartość właściciela musi przeżyć cały widok, dlatego referencji nie wolno zwracać z funkcji ani wynosić z regionu potomnego. Przykład tablicowy jest w rozdziale 7; `&String` służy do odczytu, a `&[T; N]` pozwala także zapisywać elementy bufora.

Ta sama zasada dotyczy wartości schowanej w `Some` i argumentów zagnieżdżonych w większym wyrażeniu. Zmienna napisowa użyta w środku konstruktora też chce `move`. Literał nie chce.

10.10 Podsumowanie

- Analiza nazw pilnuje nazw, a analiza ucieczki pilnuje pamięci napisu, i obie wypisują fazę `ownership`.
- `move nazwa` zużywa nazwę. Zwykłe użycie zmiennej niekopiowanej w przypisaniu i w wywołaniu jest błędem, a komunikat podpowiada `move`.
- `promote` kopiuje treść napisu do zmiennej z regionu zewnętrznego i też zużywa nazwę źródłową.
- Lewa strona przypisania do zmiennej z zewnątrz nie jest odczytem tej zmiennej.
- `move (a, b)`, `move ()` i `move` bez listy to trzy różne polecenia. Pusta lista nie jest tym samym co brak listy.
- `&nazwa` tworzy ograniczony widok na niekopiowaną wartość; nie zastępuje `move` w każdym kontekście i nie jest ogólną referencją Rustową.
- Zgadywanie listy nie rusza wartości kopiowanych i nie rusza nazw, których blok nie wspomina.

11 Rozdział 9. Jak Bork przechowuje napisy w pamięci

11.1 Ten rozdział obejmuje

- czym w gotowym programie jest wartość napisu
- gdzie powstają znaki, gdy zapisujesz wynik do zmiennej
- kiedy kompilator buduje napis od razu w arenie zmiennej docelowej
- jak czytać przykłady z dokumentacji, które kompilator odrzuca
- czego jeszcze nie da się zwrócić z funkcji

11.2 Adres i długość, a osobno znaki

W kodzie maszynowym wartość typu `String` jest parą. Pierwszy element to adres znaków, drugi to długość zapisana na 64 bitach. Same znaki leżą albo w stałej programu, albo w buforze areny. Koniec regionu unieważnia bufor, a nie parę leżącą na stosie. Dlatego kompilator musi odrzucić parę, której adres wskazywałby w już zwolniony bufor, zanim generator kodu ją wyemituje. Robi to analiza ucieczki, funkcja płace w `src/escape.rs`.

Głębokość zero oznacza region funkcji. Każdy region zagnieżdżony, który analiza ucieczki naprawdę otwiera, zwiększa głębokość. Funkcja `płace` odpowiada na pytanie, w jakiej głębokości leżą znaki danego wyrażenia.

11.3 Znaki powstają tam, gdzie stoisz

Bez miejsca przeznaczenia literał i wynik `concat` biorą bieżącą arenę. Wyjście z bloku czyści tę arenę. Nazwa zadeklarowana w bloku i tak nie jest widoczna na zewnątrz. Problem zaczyna się wtedy, gdy parę adresu i długości wynosisz przypisaniem albo instrukcją `return`.

11.4 Zapis do zmiennej wybiera jej arenę

Gdy prawa strona jest zapisywana do zmiennej, znaki wyniku trafiają do areny tej zmiennej. Nie dotyczy to każdego podwyrażenia. Argumenty wywołania są liczone tak, jakby miejsca przeznaczenia chwilowo nie było, więc wartości tymczasowe zostają w bloku.

Listing 9.1. Literał zapisany wprost do zmiennej z zewnątrz. Program został zbudowany. Na wyjściu jest `b` oraz nowy wiersz.

```
fun main() {
```

```
var outer = "a"
{
    outer = "b"
}
println(outer)
}
```

Nie potrzebujesz zmiennej pośredniej. Zapis `outer = concat(lewy, prawy)` kładzie wynik w arenie zmiennej `outer`. Napisy `lewy` i `prawy` są czytane tam, gdzie już żyją.

11.5 Dwie sąsiednie instrukcje mogą uniknąć drugiej kopii

Gdy w bloku jest najpierw `var piece = literał` albo wywołanie, a zaraz potem `outer = move piece`, i nazwa `outer` jest już widoczna, kompilator ustawia przy deklaracji informację, że treść napisu ma powstać w buforze nazwy `outer`. W reprezentacji pośredniej to pole nazywa się `alloc_in_binding`. Druga instrukcja nie kopiuje wtedy znaków drugi raz.

Rozpoznanie jest celowo wąskie. Leży w `src/hoist.rs` i obejmuje tylko następną instrukcję, bez niczego pomiędzy. Inicjalizator musi być literałem napisu albo wywołaniem, nie tablicą i nie dowolnym wyrażeniem. Prawa strona przypisania musi być przeniesieniem tej samej nazwy. Cel musi być nazwą, nie elementem tablicy. Nazwa celu musi być parametrem albo zmienną zadeklarowaną wcześniej.

Pętla, warunek, funkcja dopisana na końcu wywołania i jakakolwiek instrukcja między tymi dwiema wyłącza to rozpoznanie. Zostaje kopia przy `move` albo jawne `promote`.

NOTA. Kompilator nie dowodzi, że nazwa pośrednia istnieje tylko po to, żeby nakarmić zmienną zewnętrzną. Patrzy na dwie sąsiednie instrukcje. Gdy je rozdzielis, płacisz kopię.

11.6 Promote, gdy znaki już powstały za głęboko

Listing 8.3 pokazuje `promote`. Nazwa `held` powstała w arenie bloku. `promote` kopiuje znaki do areny zmiennej zewnętrznej i oznacza źródło jako przeniesione. Nie pomija kopii. Kopii unikasz wtedy, gdy wynik od razu zapisujesz do zmiennej zewnętrznej, albo gdy dwie sąsiednie instrukcje pozwolą zbudować wartość od razu w jej arenie.

11.7 Czego analiza ucieczki nie przepuści

Komunikaty z `src/escape.rs` sprawdzone na przykładach są takie. Zwrot wartości, której znaki żyją głębiej niż region funkcji, mówi, że zwracany napis żyje w regionie wewnętrznym i że arena zostanie zwolniona przed powrotem. Zwrot

wyniku `concat` mówi, że nie jest to jeszcze obsługiwane, bo znaki zwróconego napisu musiałyby przeżyć arenę wywoływanej funkcji. Zwrot napisu przeniesionego albo użytego ze słowem `promote` radzi użyć zwykłej nazwy albo literału. Przypisanie wartości, której pamięć leży głębiej niż zmienna docelowa, mówi, że taki zapis nie jest obsługiwany. Wartość warunku, która jest napisem przeniesionym tylko w gałęzi, jest odrzucana, bo arena gałęzi ginie razem z gałęzią.

Zapis `return if (c > 0) { concat("a", "b") } else { "x" }` wpada w komunikat o `concat`, nie w komunikat o gałęzi. Sprawdziłem to na osobnym pliku. Pozycja w pliku bywa pusta.

11.8 Przykład z dokumentacji, który się nie kompiluje

Plik `docs/language.md` pokazuje w bloku wywołanie `concat(left, right)`, gdy `left` i `right` są zmiennymi. Taki program nie przechodzi sprawdzenia. Obie nazwy są zmienne, więc w bloku wewnętrznym trzeba je przenieść. Komunikaty, które dostałem, mówią, że `left` nie jest kopiowane i trzeba je przenieść do bloku, i to samo o `right`.

Działają dwie poprawki. W obu wynik na wyjściu to LR oraz nowy wiersz, a nie `prefixLR`. `concat` nie dopisuje znaków do istniejącego bufora. Buduje nowy napis, a przypisanie zastępuje parę adresu i długości utrzymaną w zmiennej. Stary napis "`prefix`" zostaje w arenie aż do końca regionu funkcji, ale nazwa już na niego nie wskazuje.

Listing 9.2. Stałe napisy są w bloku tylko oglądane. Program został zbudowany.

```
fun main() {
    var out = "prefix"
    val left = "L"
    val right = "R"
    {
        var piece = concat(left, right)
        out = move piece
    }
    println(out)
}
```

W drzewie regionów widać, że `piece` zostało przeniesione, a `left` i `right` są współdzielone z regionu funkcji. Dwie sąsiednie instrukcje pasują do rozpoznania z `hoist.rs`, więc znaki wyniku mogą powstać od razu w arenie `out`. Dla programisty skutek jest ten sam niezależnie od tego, czy kopiowanie zostało pominięte. Po bloku `out` da się wypisać.

Listing 9.3. Zmienne napisy trzeba przenieść jawnie. Program został zbudowany. Po bloku `left` i `right` są martwe.

```

fun main() {
    var out = "prefix"
    var left = "L"
    var right = "R"
    {
        var piece = concat(move left, move right)
        out = move piece
    }
    println(out)
}

```

11.9 Czego jeszcze nie ma

Kompilator nie zgaduje więcej układów niż te dwie sąsiednie instrukcje. Nie ma bufora wyniku, który należałby do funkcji wywołującej. Nie ma `promote` przy `return`. Nawet `concat` stojący w jednej gałęzi warunku nie może być zwrócony.

Przekazanie napisu do funkcji użytkownika jest obsługiwane: deskryptor `{ ptr, i64 }` przechodzi do funkcji bez konwersji na liczbę. To nie znaczy, że obsługiwane są wszystkie typy argumentów; `float` jako argument funkcji nadal dostaje odmowę `codegenu`. Ponowne uruchomienie listingu z rozdziału 5 wypisało `hello` i zakończyło się kodem 0.

11.10 Podsumowanie

- Wartość napisu jest adresem i długością. Znaki leżą w stałej albo w buforze areny o pojemności 4096 bajtów.
- Przypisanie do zmiennej alokuje wynik w arenie tej zmiennej.
- Dwie sąsiednie instrukcje, deklaracja i zaraz potem przeniesienie do zmiennej zewnętrznej, mogą zbudować znaki od razu w arenie celu.
- `promote` kopiuje, gdy znaki już powstały w zbyt krótkim regionie.
- Przykład `concat` na dwóch zmiennych napisowych z dokumentacji języka nie kompiluje się. Trzeba użyć stałych albo słowa `move`.
- `concat` zastępuje wartość zmiennej. Nie dopisuje znaków do starego bufora.

12 Rozdział 10. Komunikaty kompilatora i błędy podczas działania

12.1 Ten rozdział obejmuje

- jak czytać linię błędu i co znaczą cztery fazy
- czego język nie oferuje w miejsce wyjątków
- które błędy przerywają kompilację, a które przerywają gotowy program
- jak odróżnić aktualne usterki `codegenu` od wyników historycznych

- w jakiej kolejności warto naprawiać kilka błędów naraz

12.2 Język nie ma instrukcji obsługi błędu

Nie ma `throw`, `try`, `catch` ani typu `Result`. Pytajnik przy typie nie jest modelem błędu, tylko wartością nullable. Codegen obsługuje wybrane typy i operacje nullable; ograniczenia reprezentacji są w rozdziale 4 i dodatku C. W czasie działania sprawdzenia kończą program m.in. przy dzieleniu przez zero, indeksie poza tablicą oraz `!!` na `None`.

Dzielenie przez zero woła `abort` z biblioteki języka C. Proces kończy się sygnałem przerwania. W powłoce widać kod 134, a na wyjściu błędów nie ma tekstu Borka. Ten sam mechanizm przerywa program, gdy indeks tablicy nie mieści się w długości zapisanej w typie. Trzecia awaria dotyczy alokacji większej niż 4096 bajtów w jednym buforze areny. Biblioteka wykonawcza, napisana w Ruście, przerywa się wtedy komunikatem `arena overflow`, z liczbą potrzebnych bajtów i pojemnością 4096. Program w Borku nie może tego złapać.

NOTA. W kodzie generatora jest także przerwanie przy dzieleniu najmniejszej liczby ze znakiem przez minus jeden. Literału ujemnego nie da się zapisać, więc tej ścieżki nie uruchamiano z pliku `.bork`. Strażnik w funkcji `guard_int_div` porównuje dzielnik z zerem, a dla typów ze znakiem także parę złożoną z minimum typu i z minus jeden.

12.3 Jak wygląda linia błędu

Funkcja `print_diagnostics` w `src/main.rs` drukuje ścieżkę, numer linii, numer kolumny, słowo `error`, nazwę fazy i treść. Gdy błąd nie ma zakresu w pliku źródłowym, numer linii i kolumny znikają. Zostaje ścieżka: `error: faza: treść`.

Zakres źródłowy, po angielsku `span`, jest parą pozycji bajtowych w pliku. Kolumna na wydruku liczy znaki od ostatniego przejścia do nowego wiersza, plus jeden. Przy tekście ASCII, a taki jest cały dzisiejszy lekser, znak i bajt wypadają w tym samym miejscu.

Fazy nazywają się `parse`, `ownership`, `type` i `codegen`. W strukturze błędu jest też poziom ostrzeżenia, ale programy uruchomione przy pisaniu książki produkują wyłącznie błędy, a wiersz poleceń i tak drukuje słowo `error`.

Serwer edytora dokłada nazwę fazy jeszcze raz, na początku treści, i przelicza bajty na pozycje wymagane przez protokół. Rozdział 18 opisuje różnicę.

12.4 Błędy składni

Tłumaczenie błędu parsera jest w `diag::from_parse`. Niepoprawny znak daje `invalid token`. Nagły koniec pliku daje `unexpected end of file` i listę rzeczy oczekiwanych. Nieoczekiwany token daje `unexpected token` oraz tę samą listę. Dodatkowy token na końcu daje `unexpected extra token`. Błąd zgłoszony wprost przez gramatykę, na przykład zły cel przypisania, dostaje pozycję na końcu pliku.

Listing 10.1. Niedomknięta lista parametrów.

Źródło fun `oops()` daje:

```
err_parse.bork:1:10: error: parse: unexpected end of file; expected r#" [a-zA-Z_][a-zA-Z0-9_]*"#,
")", "val", "var"
```

Lista oczekiwanych rzeczy jest surowa i zawiera wyrażenie regularne nazwy. Przy błędzie składni nie ma drzewa regionów ani dalszych faz. Polecenie `--dump-arenas` nie wypisuje wtedy drzewa.

12.5 Błędy typów

Komunikaty powstają w katalogu `src/typeck`. Rozdziały 4–7 cytowały te, które pojawiły się przy uruchomieniu kompilatora, a kilka dotyczy całej funkcji, nie jednego wyrażenia. Brak powrotu na wszystkich ścieżkach, ponowna deklaracja funkcji wbudowanej, nieznana nazwa, nieznana funkcja oraz zła liczba argumentów należą do tej grupy.

Sprawdzanie typów nie zatrzymuje się na pierwszym błędzie, tylko zbiera całą listę, a potem i tak uruchamia się analiza własności. Dlatego jeden plik potrafi mieć zarówno fazę `type`, jak i fazę `ownership`. Test `reports_ownership_and_type_together` łączy dodawanie liczby do napisu z użyciem nazwy po przeniesieniu.

12.6 Błędy własności

Komunikaty tej fazy powstają w dwóch miejscach: w analizie nazw oraz w analizie ucieczki, która nie startuje, gdy wcześniejsze błędy nie są puste. Przy błędzie typu możesz więc nie zobaczyć błędu ucieczki, który w poprawionym programie by się pojawił. Najpierw napraw typy i brakujące `move`, a dopiero potem czytaj komunikaty o tym, czy bufor z napisem jeszcze istnieje w chwili użycia.

Drzewo regionów powstaje także przy błędzie własności. Wypis przy użyciu nazwy po `move` pokazuje, że nazwa jest przeniesiona, i pokazuje deklarację, która tej nazwy użyła. Drzewo jest obrazem tego, co przejście zdążyło zbudować. Nie jest świadectwem, że program jest poprawny.

12.7 Błędy generowania kodu

Pojawiają się tylko przy bork build. Samo sprawdzenie ich nie produkuje. Bramka gate w src/codegen/gate.rs nadal odrzuca m.in. funkcje dopisane na końcu wywołania i niedozwolone pola. None, Some, !!, ?:, ?.length oraz nullable ==/!= nie są już ogólnie odrzucane; dla typów bez reprezentacji generator zwraca błąd fazy codegen.

Późniejsza emisja odrzuca m.in. brak main, niedozwolony wynik main i wywołania pośrednie. Float jako argument funkcji dostaje odmowę codegen. Osobny błąd dotyczy porównań floatów: frontend je przyjmuje, ale generator kieruje je do value_as_int i panikuje na FloatValue. To błąd kompilatora, nie ograniczenie semantyki porównań.

Błąd tej fazy ma kod wyjścia jeden i nie zostawia pliku wykonywalnego. Zestaw nullable buildów w programs/build/conditionals/ sprawdza obsługiwane przypadki; main bez poprawnej sygnatury nadal jest odrzucany.

12.8 Awaria kompilatora

Wynik historyczny z napisem jako argumentem kończył się kodem 101, gdy value_as_int bezwarunkowo wywoływał into_int_value na strukturze LLVM. Obecny coerce_value_to_ty przekazuje deskryptory. Nie przenoś starej paniki na bieżące zachowanie.

Float jako argument funkcji jest jawnie nieobsługiwany. Porównanie floatów ma inną usterkę: combine_binary_values sprawdza typ wyniku, a ten jest bool, więc operand trafia do value_as_int; odtworzenie kończy się paniką FloatValue.

OSTRZEŻENIE. Awaria kompilatora na programie, który sprawdzenie uznało za poprawny, jest błędem generatora kodu. Nie czytaj jej jako zakazu języka. Sprawdzanie typów i analiza własności tego zapisu nie zabraniają. Dodatek C trzyma ten przypadek razem z innymi zaległościami.

Błąd narzędzia, a nie programu, kończy się kodem dwa. Należy tu złe polecenie, brak pliku, nakładające się ścieżki, brak clang albo brak opcji codegen. Tekst zaczyna się od error: i nie ma fazy.

12.9 Gdy błędów jest kilka

Kompilator nie wybiera jednego błędu i nie milczy o reszcie. Dostajesz wszystkie, które dana faza zdążyła zebrać. Praktyczna kolejność naprawy jest taka. Najpierw faza parse, bo przy błędzie składni reszty nie ma. Potem faza type, zwłaszcza nieznane nazwy i niezgodne typy. Potem faza ownership dotycząca move. Faza

codegen ma sens dopiero wtedy, gdy sprawdzenie jest czyste. Budowanie najpierw sprawdza program. Przy błędach sprawdzenia w ogóle nie wchodzi w LLVM. Wypisuje te błędy i kończy się kodem jeden.

Budowanie nullable String i typów prostych poza unit przechodzi dla obsługiwanych operacji, także dla f32?. Nullable unit i typy funkcji nie mają reprezentacji; zwrot napisu z regionu wewnętrznego nadal jest odrzucany przez analizę ucieczki przed LLVM.

12.10 Podsumowanie

- Język nie ma wyjątków. Ma komunikaty kompilacji i trzy awarie w czasie działania.
- Linia błędu podaje fazę. Brak zakresu źródłowego usuwa numer linii.
- Faza ownership pochodzi albo z analizy nazw, albo z analizy ucieczki. Druga milczy, gdy wcześniej są inne błędy.
- Kontrola przed generowaniem kodu mówi wprost, że konstrukcja nie jest obsługiwana. Część późniejszej emisji mówi, że nie jest obsługiwana jeszcze.
- Deskryptory String są przekazywane do funkcji użytkownika. Float jako argument jest odrzucany, a porównanie floatów może wywołać panikę kompilatora.
- Kod jeden oznacza błąd programu lub odmowę codegenu. Kod dwa oznacza złe wywołanie albo brak narzędzia. Kod 101 w tabeli starych przykładów jest wynikiem historycznej awarii i nie powinien być przypisywany bieżącemu codegenowi bez ponownego testu.

13 Rozdział 11. Funkcje wbudowane i sposób pisania programów

13.1 Ten rozdział obejmuje

- dlaczego w języku nie ma modułów
- jakie funkcje wbudowane naprawdę istnieją
- jak pisać programy, które dają się zbudować
- jak pisać programy, które na razie tylko przechodzą sprawdzenie
- których zapisów kompilator nie przyjmie, choć wyglądają znajomo

13.2 Modułów nie ma

Parser na poziomie programu oczekuje nowej linii albo słowa fun. Słowa module i struct są błędami składni. Nie ma import, ścieżek plików, widoczności ani

osobnych jednostek kompilacji po stronie języka. Jeden plik zawiera jedną listę funkcji i jedną mapę nazw.

Słowo moduł wraca w tej książce tylko przy opisie dwóch skrzynek Rusta, `bork` i `bork_runtime`. Skrzynka, po angielsku `crate`, jest jednostką kompilacji w Cargo. To podział kodu kompilatora, nie mechanizm języka Bork. Serwer edytora i rozszerzenie w `tools/bork-lsp-extension` są narzędziami obok języka. Nie dodają składni.

13.3 Cała biblioteka, która istnieje

W `src/builtins.rs` są trzy nazwy. Nic więcej nie jest zarejestrowane.

`print` wypisuje jedną wartość `i32`, `i64` albo napis i opróżnia bufor standardowego wyjścia. `println` robi to samo i dodaje nowy wiersz. `concat` przyjmuje dwa napisy i zwraca nowy napis. Bufor powstaje w arenie miejsca, do którego wynik jest zapisywany, a gdy takiego miejsca nie ma, w arenie bieżącego bloku.

Nie ma asercji, formatowania, czytania standardowego wejścia, plików, zegara ani argumentów wiersza poleceń. Funkcja `main` nie dostaje `argv`. Generator kodu odrzuca `main` z parametrami już przy deklaracji funkcji LLVM.

Biblioteka wykonawcza ma funkcje, których nie wołasz z Borka wprost. Należą do nich pobranie bufora areny, jego zwrot, wyczyszczenie wskaźnika, alokacja w buforze oraz wypisywanie liczb i napisów. Rozdział 17 wymienia ich nazwy w C, bo taki jest sposób połączenia wygenerowanego kodu z biblioteką wykonawczą.

13.4 Jak pisać programy, które kompilator potrafi zbudować

Wynik liczbowy jako kod wyjścia jest wygodny, gdy ćwiczysz budowanie bez wypisywania. Funkcja `main`, która zwraca 7, została zbudowana i kończy się kodem 7. Tak samo robi `test builds_return_constant`.

Pętla z liczbowym akumulatorem nie walczy z własnością. Listing 6.3 jest wzorem. Indeks i suma są typu `i32`, więc ciało pętli je kopiuje.

Napis, który ma przeżyć blok, trzymaj w `main` i wypisz na końcu. Listing 9.1 pokazuje przypisanie literału do zmiennej z zewnątrz. Nie zwracaj świeżego napisu z funkcji pomocniczej, dopóki nie ma bufora należącego do wywołującego.

Stała napisowa, którą blok wewnętrzny tylko czyta, jest prostsza niż przenoszenie w tę i z powrotem. Listing 2.2 jest wzorem współdzielenia.

Gdy chcesz, żeby kompilator zbudował napis od razu w arenie zmiennej docelowej, napisz deklarację i w następnym wierszu przeniesienie. Nie wstawiaj między nie

println.

Tablicy używaj z wycinkiem o stałych granicach, gdy długość jest znana w typie. Listing 7.4 to pokazuje. Wycinka o granicach trzymany w zmiennych sprawdzanie typów nie przyjmie, bo typ wyniku nie miałby znanej długości.

Warunek, który ma dać liczbę, zapisuj z dwiema gałęziami-blokami. Listing 6.1 jest wzorem.

13.5 Programy tylko do sprawdzenia

Pisz je, gdy ćwiczysz kompilator albo edytor. Nie wkładaj ich do programu, który ma być plikiem wykonywalnym.

Do konstrukcji przyjmowanych przez frontend, ale nadal nieemitowanych, należą funkcje dopisane na końcu wywołania, bloki move po wywołaniu i wywołania pośrednie. Nullable String? i typy proste poza unit mają codegen dla Some, None, ?:, !!, ?.length oraz ==/!=. Float jako argument funkcji jest odrzucany, a porównania floatów pozostają błędem codegenu.

Listing 11.1. Fragment próbki PROCESS_USER_SAMPLE z src/lib.rs, pokazujący nullable napis.

```
fun processUser(name: String?, score: i32): i32 {  
    val fallbackName: String = name ?: "Guest"  
    val verifiedUser: String? = Some(fallbackName)  
    val emptyMiddle: String? = None  
    val verifiedLength: i32 = verifiedUser?.length ?: 0  
    if (verifiedLength > 0) {  
        return score  
    }  
    return 0  
}
```

Oryginalna próbka w lib.rs jest dłuższa i ma dodatkową nazwę oraz zagnieżdżony blok. Działające przykłady nullable do budowania są w programs/build/conditionals/; sam fragment powyżej nie jest pełnym programem z main.

13.6 Zapisy, które wyglądają znajomo i są błędami

Dodawanie dwóch napisów operatorem + jest błędem typu. Operator nie skleja tekstu. return -3 jest błędem składni, bo minus nie jest operatorem jednoargumentowym. val n = 1; jest błędem składni. Przypisanie var x = s przy zmiennej napisowej s prosi o move. Wywołanie concat na dwóch zmiennych napisowych wewnątrz bloku prosi o przeniesienie każdej z nich. return concat(a, b) odpada w analizie ucieczki. Indeks spoza tablicy kompiluje się i przerywa program. Dzielenie 1 / 0 kompiluje się i przerywa program. Druga

deklaracja `println` jest błędem typu. Słowa `struct` i `module` są błędami składni na poziomie programu.

13.7 Styl, który pasuje do regionów

Krótki blok robi jedną arenę na wartości tymczasowe. Długi blok funkcji trzyma dane, które mają przeżyć pętlę. Pętla nie powinna przenosić stanu utworzonego przed nią. Powinna czytać wartości kopiowane albo stałe współdzielone i zapisywać wynik do zmiennej z zewnątrz przez przypisanie.

Stałych używaj do progów i do napisów, które są tylko czytane. Zmiennych używaj do akumulatorów. To nie jest rada stylistyczna obok kompilatora. To jest różnica, którą analiza własności sprawdza.

Nie ma odpowiednika rustowego `clone`. Świadoma kopia napisu wymaga zbudowania nowego napisu, na przykład przez `concat`, gdy oba argumenty są w danym regionie legalne, albo przez przypisanie literału. Nie ma ogólnego kopiowania tablicy poza zbudowaniem nowego literału. Wycinek nie jest kopią.

13.8 Podsumowanie

- Biblioteka języka to `print`, `println` i `concat`.
- Modułów, struktur i `import` nie ma. Program jest jednym plikiem.
- Program, który ma się zbudować, trzyma funkcje przy liczbach, napisy w `main`, stałe przy samym odczycie i zmienne przy akumulatorach.
- `Nullable String?` i typy proste poza `unit` są obsługiwane przez `codegen`; funkcja dopisana na końcu wywołania nadal nie jest emitowana.
- Operator `+` na napisach, średnik, minus jednoargumentowy i gołe użycie zmiennej napisowej są błędami kompilacji.

14 Rozdział 12. Od pliku źródłowego do gotowego programu

Zanim kompilator wyemituje choć jedną instrukcję, musi odpowiedzieć na kilka pytań, które nie mają ze sobą nic wspólnego poza tym, że dotyczą tego samego pliku. Czy tekst w ogóle układa się w program, czy nazwy mają typy, które do siebie pasują, czy napis nie został użyty po tym, jak jego właściciel już go oddał, i czy pamięć napisu, który funkcja chce zwrócić, pożyje po jej powrocie. Każde z tych pytań ma własną fazę, bo pomyłka w odpowiedzi psuje inną rzecz. Zły kształt tekstu uniemożliwia w ogóle zbudowanie drzewa, zły typ psuje późniejsze wywołanie, a napis zwrócony ze zbyt krótkiego regionu psuje pamięć już w działającym procesie.

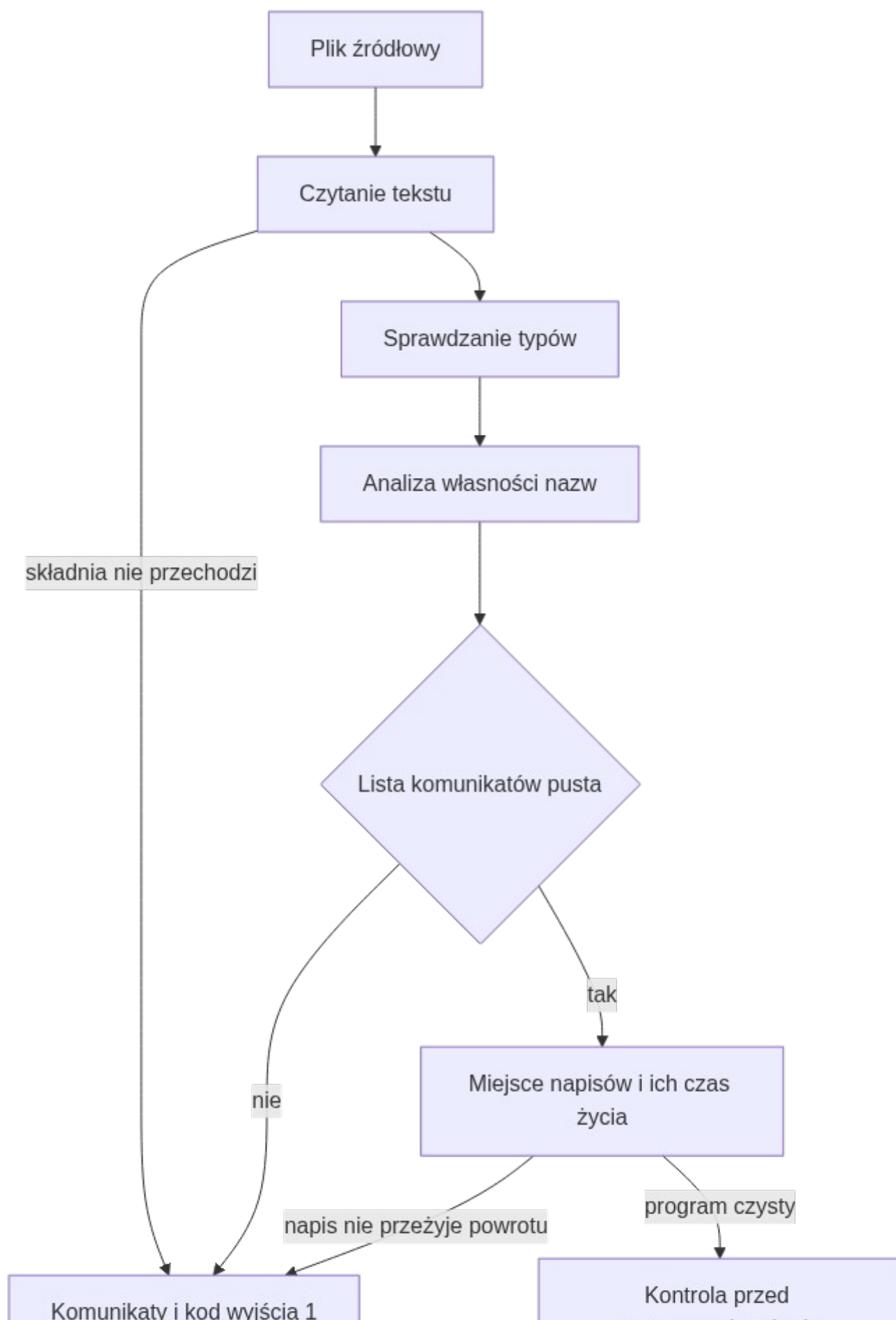
Ten rozdział obejmuje

- pytanie, na które odpowiada cały przebieg kompilacji, zanim powstanie plik wykonywalny,
- dwa polecenia wiersza poleceń i to, który fragment pracy jest im wspólny,
- trzy struktury, które kompilator niesie między fazami, oraz to, kiedy która z nich znika,
- powód, dla którego błąd własności potrafi pojawić się w wydruku przed błędem typu,
- moment, w którym program uznaje się za czysty i wolno ruszyć z tłumaczeniem na kod maszynowy.

14.1 Po co dzielić jedno uruchomienie na fazy

Pytanie tej części dotyczy wiedzy, bez której nie wolno emitować kodu. Kompilator musi wiedzieć, zanim przetłumaczy program na kod maszynowy, co konkretnie zepsułoby się, gdyby którąś odpowiedź pominął. Bez podziału na fazy jeden błąd składni mieszałby się z błędem typu, a generator kodu próbowałby czytać drzewo, którego w ogóle nie ma. Dlatego Bork prowadzi plik przez stałą kolejność, a każda faza albo dopisuje informację, której poprzednia nie miała, albo odmawia iść dalej.

Najpierw powstaje drzewo składni, czyli struktura, która pamięta kształt tekstu, ale jeszcze nie wie, czy `n` jest liczbą, napisem czy nazwą, której wcale nie zadeklarowano. Potem sprawdzanie typów buduje drugie drzewo, reprezentację pośrednią, w której każde wyrażenie ma już typ. Równolegle w sensie danych, choć chwilę później w czasie, analiza własności buduje raport regionów. Raport nie przydziela przy tym ani jednego bajtu pamięci wykonawczej, tylko zapisuje, która nazwa jest lokalna, która została skopiowana, która jest współdzielona z zewnętrznego bloku, a która została przeniesiona. Gdy obie te fazy milczą, dochodzą jeszcze dwie decyzje o napisach. Jedna mówi, czy blok w ogóle potrzebuje własnego bufora, a druga, czy wartość, którą funkcja zwraca, przeżyje powrót. Dopiero czysty wynik wolno oddać generatorowi kodu.



Rysunek 12.1. Droga pliku od tekstu źródłowego do programu wykonywalnego

Rysunek 12.1 pokazuje drogę pliku, a nie każdą strukturę po drodze. Czytanie tekstu jest jedyną fazą, po której porażce nie ma ani drzewa składni do dalszej pracy, ani raportu regionów. Wszystkie późniejsze odmowy zostawiają raport, bo analiza własności zdążyła go zbudować, nawet jeśli program i tak jest błędny. Plik wykonywalny powstaje wyłącznie na gałęzi, na której lista komunikatów została pusta i uruchomiono polecenie budowania.

14.2 Dwa polecenia, jeden wspólny początek

Pytanie jest tu praktyczne i dotyczy dwóch poleceń, które łatwo pomylić. Chodzi o to, czym różni się samo sprawdzenie pliku od zbudowania programu i dlaczego w ogóle są to dwa polecenia, skoro początek pracy jest ten sam. Samo sprawdzenie odpowiada, czy program jest do przyjęcia. Budowanie, gdy odpowiedź jest twierdząca, tłumaczy go na plik, który da się uruchomić. Gdyby budowanie szło własną, krótszą ścieżką, program z błędem typu mógłby dojść do emisji kodu i zepsuć się dopiero w środku generatora, komunikatem Rusta zamiast komunikatem o Twoim pliku.

Polecenie `bork plik.bork` kończy się na sprawdzeniu. Kod wyjścia 0 oznacza pustą listę komunikatów, a kod 1 oznacza, że coś odrzucono. Polecenie `bork build` woła to samo sprawdzenie, a dopiero przy pustej liście przechodzi przez kontrolę konstrukcji, których generator jeszcze nie umie, przez emisję pośredniego zapisu LLVM i przez wywołanie `clang`, który łączy wynik z biblioteką wykonawczą. Flaga `--dump-arenas` dopisuje drzewo regionów na standardowe wyjście i nie zmienia kodu wyjścia. Działa przy sprawdzeniu, a przy budowaniu tylko wtedy, gdy samo sprawdzenie już przeszło, bo przy błędzie budowanie kończy się wcześniej.

Weźmy program, który dodaje liczby w pętli, wypisuje etykietę ze współdzielonego napisu i zwraca sumę. To jest ten sam plik, który w zestawie przykładów nazywa się `18-trace.bork`. Nadaje się do prześledzenia całego początku kompilacji, bo ma funkcję pomocniczą, pętlę, liczbę kopiowaną przy odczycie i napis czytany z wewnętrznego bloku. Żadna z tych konstrukcji nie jest błędem, więc widać sam opis, który kompilator buduje, gdy nie ma czego odrzucić.

Listing 12.1. Program, który przechodzi całe sprawdzenie

```
fun add(a: i32, b: i32): i32 {
    return a + b
}

fun main(): i32 {
    var total = 0
    for (i in 0..3) {
        total = add(total, i)
    }
}
```

```

    val label = "sum"
    {
        println(label)
    }
    return total
}

```

Sprawdzenie z wydrukiem regionów kończy się kodem 0 i pokazuje, co analiza własności zapisała, zanim ktokolwiek pomyślał o instrukcjach. Wydruk nie jest jeszcze kodem maszynowym ani nawet decyzją o tym, który blok dostanie bufor. Jest opisem nazw w funkcjach `add` i `main`, i właśnie dlatego da się go czytać razem z listingiem, nazwa po nazwie. Poniższy wycinek pochodzi z uruchomienia `bork --dump-arenas` na tym pliku i kończy się tak samo kodem 0.

```

Arenas
├─ fun add
│   ├── a [Local]
│   └─ b [Local]
└─ fun main
    ├── total [Local]
    ├── label [Local]
    ├── ForLoop (i)
    │   ├── i [Local]
    │   └─ total [Copy]
    └─ Block
        └─ label [Shared ← fun main]

```

Parametry `a` i `b` są lokalne w `add`, bo funkcja ich nie wynosi na zewnątrz. `total` w pętli jest kopią, bo liczba całkowita kopiuje się przy odczycie, więc pętla może czytać ją wielokrotnie. `label` w zagnieżdżonym bloku jest współdzielony z `main`, bo wewnętrzny blok czyta napis, którego nie zadeklarował. Cały ten wydruk mówi tylko o tym, kto nazwę widzi, i nie jest jeszcze decyzją o buforze pamięci. Budowanie tego samego pliku daje program, który wypisuje `sum` i kończy się kodem 3, bo pętla `0..3` dodaje 0, 1 i 2.

NOTA. Wydruk regionów przy udanym sprawdzeniu nie jest jeszcze opisem wygenerowanego kodu. Mówi wyłącznie o tym, jak kompilator sklasyfikował nazwy w tym przebiegu. O tym, czy blok dostanie własne wywołanie wejścia do regionu, decyduje późniejszy krok, opisany w rozdziale 16, i tylko wtedy, gdy lista komunikatów jest pusta. Dopóki w wydruku stoi samo `Shared`, nie wiesz jeszcze, czy ten blok w ogóle alokuje.

14.3 Co zostaje, gdy program jest błędny

Pytanie brzmi, które wyniki faz wolno zachować po odmowie i dlaczego nie wolno zachować wszystkich. Generator kodu potrzebuje reprezentacji pośredniej, w której każde wyrażenie ma typ i nie ma obok listy błędów. Gdyby przy błędzie ta reprezentacja została w strukturze wyniku, późniejszy kod mógłby ją przypadkiem przetłumaczyć. Raport regionów rządzi się inną zasadą i przydaje się do wydruku

nawet wtedy, gdy program jest zły, bo pokazuje, jak daleko analiza własności doszła.

Wynik jednego przebiegu, w kodzie źródłowym `CheckResult`, niesie trzy pola i nic więcej. Raport regionów jest obecny zawsze poza jedną sytuacją, mianowicie wtedy, gdy tekst w ogóle się nie sparsuje, bo wtedy nie ma drzewa, po którym analiza mogłaby przejść. Reprezentacja pośrednia jest obecna dokładnie wtedy, gdy lista komunikatów jest pusta. Lista komunikatów zbiera odmowy ze wszystkich faz, które zdążyły ruszyć. Po błędzie składni jest w niej jeden komunikat i oba drzewa są nieobecne. Po błędzie typu, własności albo czasu życia napisu raport zostaje, a reprezentacja pośrednia jest odrzucana, nawet jeśli sprawdzanie typów zdążyło ją w całości zbudować.

Widać to na pliku, który ma naraz błąd własności i błąd typu. Nazwa `s` jest przenoszona, a potem używana w dodawaniu z liczbą. Składnia jest w porządku, więc obie późniejsze fazy mają po czym przejść i każda ma powód do odmowy. Dzięki temu na jednym przebiegu widać i kolejność komunikatów, i drzewo, które zostaje mimo błędów.

Listing 12.2. Dwa niezależne błędy w jednym pliku

```
fun main() {  
    var s = "hi"  
    val t = move s  
    val u = s + 1  
}
```

Uruchomienie `bork --dump-arenas` na tym pliku kończy się kodem 1 i wypisuje obie odmowy, a pod nimi drzewo, które analiza własności zdążyła zbudować. Ścieżka w komunikacie zależy od tego, skąd uruchomiono polecenie, więc w Twoim terminalu będzie inna niż w wycinku poniżej. Treść błędów i kształt drzewa są natomiast stałe, bo wynikają z programu, a nie z katalogu. Oba komunikaty wskazują tę samą kolumnę, bo i przeniesienie, i dodawanie dotyczą tego samego odczytu `s`.

```
/tmp/borkch/both.bork:4:13: error: ownership: use of `s` after move from fun main  
/tmp/borkch/both.bork:4:13: error: type: arithmetic operands must have the same numeric type,  
got String and i32  
Arenas  
└─ fun main  
   │ s [Moved ← fun main]  
   │ t [Local]  
   └─ u [Local]
```

Drzewo jest, choć program nie jest czysty, i właśnie dlatego łatwo je pomylić z wynikiem, na którym wolno budować. Nie ma za to reprezentacji pośredniej w wyniku, więc `bork build` nawet nie dochodzi do kontroli przed generowaniem kodu. Gdyby w tym pliku zepsuć nawias albo dokleić drugie `val` w tej samej linii bez nowego wiersza, odpadłoby także drzewo regionów, bo parsowanie zwróciłoby

błąd wcześniej i reszta faz w ogóle by nie wystartowała. Różnica między tymi dwoma odmowami jest więc różnicą tego, co kompilator zdążył zapamiętać, a nie różnicą w kodzie wyjścia, bo w obu razach jest to 1.

OSTRZEŻENIE. Pusta lista komunikatów nie oznacza, że generator kodu umie każdą konstrukcję. Nullable String? i typy proste poza unit są obsługiwane w rozdziale 17; nadal odpadają m.in. funkcje dopisane na końcu wywołania, nullable typy bez reprezentacji, nieobsługiwane pola i niedozwolony wynik main. Porównania floatów przechodzą sprawdzanie typów, ale w aktualnym codegenie mogą wywołać panikę.

14.4 Dlaczego błąd własności bywa pierwszy

Pytanie, które tu wraca z rozdziału 2, dotyczy kolejności na ekranie. Dlaczego komunikat z fazą ownership potrafi stać nad komunikatem z fazą type, skoro typy są liczone wcześniej. Kolejność na ekranie jest kolejnością składania listy, a nie kolejnością myślenia kompilatora. Gdyby wypis iść ściśle za czasem, czytelnik musiałby znać wewnątrz check, żeby rozumieć wydruk. Wydruk jest stabilny, nawet jeśli przez to sprawia wrażenie odwróconego.

Sprawdzanie typów idzie pierwsze, bo analiza własności potrzebuje typów deklaracji w kolejności, w jakiej występują w tekście. Bierze je z wektora, który sprawdzanie typów właśnie wypełniło, i zdejmuje po jednym przy każdej deklaracji. Dlatego typ musi być już policzony, zanim analiza zdecyduje, czy nazwę wolno skopiować. Po obu przejściach lista komunikatów powstaje tak, że najpierw trafiają do niej błędy własności, a potem, na koniec, błędy typów. W listingu 12.2 widać dokładnie ten układ. Użycie s po przeniesieniu jest wypisane wcześniej niż skarga, że do napisu dodano liczbę, choć skarga o typie powstała w czasie wcześniej.

Jest jeszcze jeden skutek wspólnego wektora typów. Analiza własności nie zagląda do reprezentacji pośredniej. Widzi drzewo składni i kolejkę typów deklaracji. Dla zwykłych funkcji w main oba opisy się zgadzają i testy to pilnują. Dla parametrów funkcji dopisanej na końcu wywołania analiza nie dostaje typu, który sprawdzanie typów właśnie wyliczyło w osobnym zejściu, i zostawia typ nieznanym. Rozdział 14 pokazuje, co z tego wynika dla współdzielenia, a rozdział 15 pokazuje to samo od strony typów.

14.5 Kiedy program jest czysty

Pytanie brzmi, co jeszcze musi się udać po typach i własności, zanim wynik wolno nazwać czystym. Sam brak błędów typu i własności nie wystarcza, bo program może poprawnie przenosić nazwy i mimo to zwracać napis, którego pamięć umrze razem z buforem wołanej funkcji. Taki błąd nie jest widoczny w drzewie składni, bo drzewo pamięta tylko, że w return stoi wywołanie. Wychodzi dopiero wtedy,

gdy kompilator wie, w którym regionie wartość powstała i czy wolno ją wynieść do wywołującego.

Gdy lista po typach i własności jest pusta, dochodzą trzy rzeczy, nadal wewnątrz tego samego przebiegu. Najpierw zaznacza się, które bloki w ogóle wołają wejście do regionu przy generowaniu kodu. Potem dopisuje się przy deklaracjach informację, czy napis ma powstać od razu w buforze miejsca, do którego za chwilę zostanie przeniesiony. Na końcu każda funkcja jest sprawdzana pod kątem ucieczki, i dopiero ta trójka domyka sprawdzenie.

Wynik concat zwrócony wprost jest odrzucany, podobnie jak próba wyniesienia wartości z regionu zagnieżdżonego dalej, niż pozwala wynik funkcji. Jeśli któraś z tych odmów dojdzie, reprezentacja pośrednia znowu znika z wyniku, a raport regionów zostaje. Szczegóły tych trzech kroków są treścią rozdziału 16. Tutaj ważne jest tylko ich miejsce w kolejce, bo stoją za typami i własnością, a przed jakąkolwiek emisją kodu.

WSKAZÓWKA. Gdy `--dump-arenas` pokazuje drzewo, a mimo to kod wyjścia wynosi 1, czytaj komunikat nad drzewem, zanim zaczniesz poprawiać regiony. Drzewo przy błędzie bywa kompletne i przez to wygląda wiarygodnie, ale budowanie i tak się nie zacznie. Najpierw zdejmij odmowę z fazy wypisanej w linii error, a dopiero potem wracaj do etykiet przy nazwach.

14.6 Od czystego wyniku do pliku na dysku

Ostatnie pytanie tego rozdziału dotyczy już tylko polecenia `build`. Co dzieje się z czystą reprezentacją pośrednią i dlaczego sam fakt, że sprawdzenie przeszło, nie gwarantuje pliku wykonywalnego. Generator kodu zakłada, że drzewo jest typowane i że błędy języka zostały już zgłoszone. Nie umie jednak każdej konstrukcji, którą język opisuje, więc między czystym wynikiem a emisją stoi jeszcze kontrola. Bez niej nieobsłużony wariant trafiłby do dopasowania, które kończy się awarią procesu kompilatora albo, co gorsza, do ścieżki liczby całkowitej i do złego rzutowania wartości.

Kontrola ogląda kształt programu, zanim powstanie choć jedna instrukcja. Pyta między innymi, czy jest funkcja `main`, czy `main` zwraca `i32` albo `nic`, czy nie ma funkcji dopisanej na końcu wywołania ani pola, którego emisja nie obsługuje. `None` i `Some` nie są już ogólnymi odmowami bramki. `Nullable` typu bez reprezentacji może jednak zostać odrzucone później przez `lowerer`. Przy odmowie dostajesz komunikat fazy `codegen` i kod 1, bez pliku wynikowego. Przy zgodzie powstaje zapis LLVM, z niego plik obiektowy, a `clang` linkuje go z `libbork_runtime.a`. Biblioteka wykonawcza wstawia między innymi bufor regionu, wypisywanie i sprawdzenia, które przerywają proces przy dzieleniu przez zero albo przy indeksie poza tablicą.

Na końcu zostaje mapa do kodu, bo cały ten rozdział da się streścić jednym przebiegiem. Funkcja `check` w pliku `src/frontend.rs` układa fazy w opisanej kolejności i zwraca `CheckResult`. Polecenia w `src/main.rs` albo wypisują komunikaty i ewentualnie drzewo regionów, albo, gdy wynik jest czysty, wołają budowanie. Kontrola przed emisją siedzi w `src/codegen/gate.rs`, a złożenie zapisu pośredniego z linkowaniem jest w `src/codegen/llvm/`. Czytanie tekstu, własność, typy i decyzje o napisach mają własne rozdziały dalej w tej części.

14.7 Podsumowanie

- Kompilator odpowiada po kolei na pytania o kształt tekstu, o typy, o własność nazw i o czas życia napisu, bo każda pominięta odpowiedź psuje inny, późniejszy krok.
- Polecenia `bork plik.bork` i `bork build` zaczynają się od tego samego sprawdzenia, a plik wykonywalny powstaje tylko wtedy, gdy lista komunikatów jest pusta i kontrola przed generowaniem kodu niczego nie odrzuci.
- Raport regionów zostaje także przy błędnym programie, o ile tekst w ogóle się sparsuje, natomiast reprezentacja pośrednia jest w wyniku tylko przy pustej liście komunikatów.
- Błędy własności są wypisywane przed błędami typów, ponieważ tak składa się lista, choć typy liczy się wcześniej, żeby analiza własności mogła zdjąć typ każdej deklaracji.
- Wydruk regionów opisuje klasyfikację nazw, a nie instrukcje, i nie zastępuje kontroli, która tuż przed emisją odrzuca konstrukcje jeszcze nieprzetłumaczone na kod maszynowy.

15 Rozdział 13. Jak kompilator czyta tekst programu

Generator kodu nie umie czytać znaków. Potrzebuje drzewa, w którym wiadomo, co jest deklaracją, co wywołaniem, a co ciałem pętli, i w którym każde takie miejsce pamięta, w którym bajcie pliku stało. Dopóki tego drzewa nie ma, nie da się zapytać ani o typ nazwy, ani o to, czy napis został przeniesiony. Dlatego pierwsza faza nie tłumaczy programu, tylko rozstrzyga, czy tekst w ogóle jest programem.

Ten rozdział obejmuje

- pytanie, na które odpowiada czytanie tekstu, i co odpada, gdy tekst nie składa się w drzewo,
- powód, dla którego nowa linia kończy instrukcję, ale wewnątrz nawiasów już

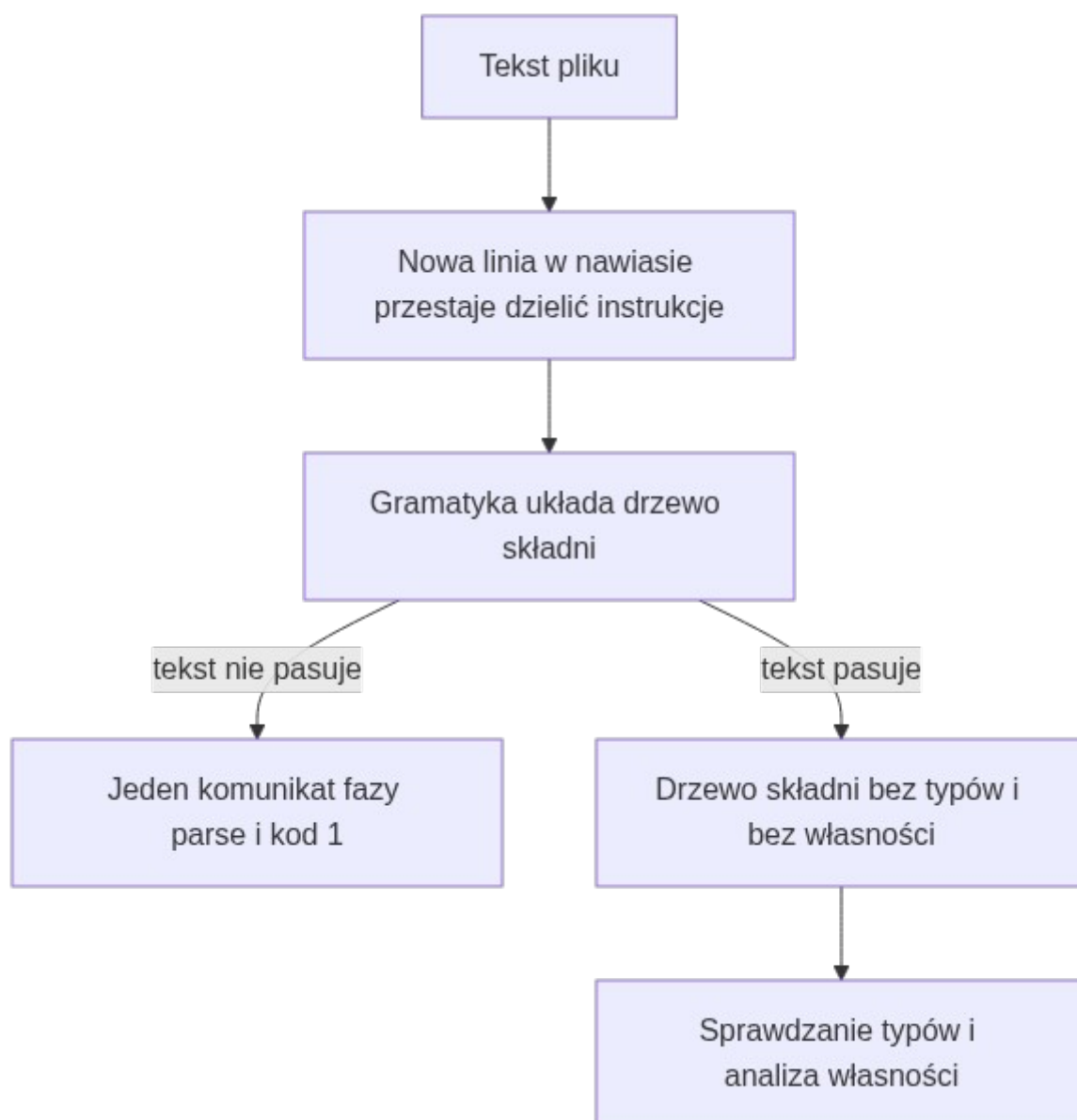
nie,

- to, co drzewo składni pamięta, oraz to, czego jeszcze świadomie nie wie,
- dwa rodzaje odmowy, z których jedna wskazuje winny symbol, a druga koniec pliku,
- miejsce w źródłach, w którym ta faza się zaczyna i kończy.

15.1 Po co w ogóle budować drzewo

Pytanie tej fazy brzmi, czy ciąg znaków da się odczytać jako program Borka, i które fragmenty są instrukcjami, wyrażeniami oraz deklaracjami. Bez odpowiedzi generator nie ma po czym chodzić, a sprawdzanie typów nie ma czego typować. Gorzej, bo błąd w nawiasie wyglądałby wtedy jak błąd typu albo jak awaria w środku emisji kodu, choć program w ogóle nie powstał. Osobna faza czytania odcina te pomyłki, zanim ktokolwiek zacznie mówić o własności albo o buforze.

Drzewo, które z tego powstaje, nazywa się drzewem składni. W źródłach kompilatora jego typ to Program z pliku `src/ast.rs`, ale w tym rozdziale ważniejsze jest to, co ono reprezentuje. Każdy węzeł odpowiada kawałkowi tekstu, a nie kawałkowi pamięci wykonawczej. Węzeł deklaracji pamięta, czy nazwa była `val`, czy `var`, czy podano typ, i jakie wyrażenie stoi po prawej stronie. Węzeł wyrażenia pamięta, czy to literał, nazwa, wywołanie, dodawanie, pożyczanie zapisane jako `&nazwa` albo przeniesienie zapisane słowem `move`. W typach gramatyka przyjmuje też zapis `&T`. Nie pamięta, czy ta nazwa jest liczbą, ani czy wolno ją skopiować albo pożyczyć. To są pytania późniejszych faz, i drzewo zostawia je puste celowo, żeby błąd składni nie udawał błędu typu.



Rysunek 13.1. Jak z tekstu powstaje drzewo składni albo jeden błąd czytania

Rysunek 13.1 podkreśla jedną rzecz, którą łatwo przegapić przy czytaniu komunikatu. Między plikiem a gramatyką jest jeszcze ciche przygotowanie tekstu, opisane w następnej części, i ono nie produkuje własnego komunikatu. Albo gramatyka dostaje tekst, w którym nowa linia w nawiasie już nie kończy instrukcji, albo, gdy coś nie pasuje, cały przebieg kończy się jednym błędem fazy parse. Rozdział 12 mówił, że przy tej odmowie nie powstaje raport regionów, i tutaj widać powód. Nie ma drzewa, po którym analiza mogłaby przejść, więc nie ma też czego wypisać pod flagą --dump-arenas.

15.2 Dlaczego nowa linia raz dzieli instrukcję, a raz nie

Pytanie brzmi, jak kompilator ma pogodzić dwie rzeczy, których programiści oczekują naraz. Nowa linia w Borku kończy instrukcję, więc nie trzeba średnika,

ale wywołanie albo warunek musi dać się złamać między argumentami, tak jak w Kotlinie. Gdyby nowa linia zawsze kończyła instrukcję, zapis `add(` i w następnym wierszu `1, 2)` byłby urwanym wywołaniem. Gdyby nowa linia nigdy jej nie kończyła, dwie deklaracje w kolejnych wierszach sklełyby się w jedną i błąd wychodziłby dużo później, w typach albo w ogóle w złym drzewie.

Rozwiązanie jest lokalne i dzieje się, zanim gramatyka w ogóle ruszy. Nowa linia, która stoi bezpośrednio w nawiasach okrągłych, jest zamieniana na powrót karetki, czyli na znak, który lekser i tak traktuje jak biały znak bez znaczenia. Nowa linia poza nawiasami zostaje i dalej oddziela instrukcje. Jeśli w nawiasie otworzysz nawias klamrowy, czyli blok, nowe linie wewnątrz tego bloku zostają, bo tam znowu oddzielają instrukcje. Komentarz do końca linii oraz literał napisu są przy tym omijane, żeby nawias w komentarzu albo w tekście `"a(b"` nie zmienił układu reszty pliku. Zamiana jest bajt za bajt, więc pozycja błędu w komunikacie nadal wskazuje oryginalny plik, a nie przerobioną kopię.

Widać to na funkcji, która łamie argumenty między wierszami. Plik przechodzi sprawdzenie z kodem 0, choć między `add(` a `1` stoi nowa linia. Gdyby ta nowa linia kończyła instrukcję, `val n = add(` zostałoby urwane i kompilator nie doszedłby do liczb w kolejnych wierszach. To, że kod wyjścia jest 0, znaczy właśnie, że przygotowanie tekstu zdążyło potraktować te złamanie jak biały znak.

Listing 13.1. Wywołanie złamane wewnątrz nawiasów

```
fun main(): i32 {  
    val n = add(  
        1,  
        2  
    )  
    return n  
}  
fun add(a: i32, b: i32): i32 {  
    return a + b  
}
```

Ta sama nowa linia postawiona poza nawiasem już dzieli instrukcje, i o to chodzi. Dwie deklaracje w jednym wierszu, bez nowej linii między nimi, nie mają czym się rozdzielić, więc gramatyka odmawia. To jest odwrotna strona tej samej reguły, a nie osobny wyjątek dopisany do średników, których w Borku nie ma. Poniższy plik jest krótki celowo, żeby w komunikacie było widać sam symbol, na którym czytanie stanęło.

Listing 13.2. Dwie deklaracje w jednym wierszu

```
fun main() { val a = 1 val b = 2 }
```

```
/tmp/borkch/online.bork:1:24: error: parse: unexpected token `val`; expected "!", "!=", "&&",  
"(", "*", "+", "-", ".", "..", "/", "<", "<=", "==", ">", ">=", "?.", "?:", "[", "||", "}", "NL"
```

Komunikat wskazuje drugie `val`, bo po prawej stronie pierwszej deklaracji gramatyka skończyła wyrażenie i spodziewała się albo operatora, albo końca instrukcji. Lista oczekiwanych symboli w tym przebiegu jest długa i nie warto jej tu przepisywać. Ważne jest, że stoi w niej także `NL`, czyli właśnie nowa linia, która w tym miejscu rozdzieliłaby dwie deklaracje. Reszta listy to operatory i nawiasy, którymi wyrażenie mogłoby być kontynuowane. Pełny zestaw symboli bierze się z gramatyki w `src/parser.lalrpop`, a nie z ręcznie pisanej listy komunikatów.

NOTA. Przygotowanie tekstu nie jest fazą, którą da się włączyć albo wyłączyć flagą. Każde wywołanie `parse` najpierw poprawia nowe linie w nawiasach, a dopiero potem uruchamia gramatykę. Gdy nic w pliku nie wymaga zamiany, do gramatyki idzie oryginalny tekst, i pozycje błędów i tak pozostają zgodne z tym, co widać w edytorze.

15.3 Czego drzewo jeszcze nie wie

Pytanie, które tu zostaje po udanym czytaniu, dotyczy granicy tej fazy. Skoro drzewo już jest, dlaczego kompilator nie próbuje od razu powiedzieć, że `n` jest liczbą albo że napis trzeba przenieść. Dlatego, że kształt i znaczenie to dwa różne pytania, a mieszanie ich w gramatyce psuje komunikaty. Gramatyka miałaby wtedy odrzucać program raz za brakujący nawias, a raz za zły typ, tym samym mechanizmem, i nie byłoby fazy `parse` odróżnionej od fazy `type`.

Drzewo pamięta więc kształt i zakres w pliku. Literał `1` jest węzłem liczby, a nie jeszcze wartością typu `i32`, choć później sprawdzanie typów zwykle właśnie tak go odczyta. Nazwa jest węzłem nazwy, nawet jeśli tej nazwy nigdzie nie zadeklarowano. Słowo `move` przed nazwą jest osobnym kształtem wyrażenia, a nie od razu decyzją, że własność przeszła. Promocja zapisana w tekście też zostaje kształtem, dopóki późniejsza faza nie zdecyduje, co z nią zrobić. Funkcja dopisana na końcu wywołania, w nawiasach klamrowych po argumentach, jest w drzewie zwykłym argumentem tego wywołania. O tym, że generator kodu jej jeszcze nie emituje, drzewo nic nie wie i wiedzieć nie powinno.

Pozostałe kształty, których gramatyka się podejmuje, działają tak samo. Pętla `for`, pętla `while`, instrukcja `if`, blok, indeks tablicy i wywołanie są węzłami z dziećmi, a nie gotowymi instrukcjami maszynowymi. Nie ma sensu wyliczać tu każdej produkcji. Wystarczy wiedzieć, że jeśli tekst przeszedł, każdy z tych węzłów ma zakres, a jeśli nie przeszedł, nie ma żadnego z nich. Lista produkcji jest gramatyką w `src/parser.lalrpop`, a struktury węzłów są w `src/ast.rs`.

WSKAZÓWKA. Gdy komunikat ma fazę `parse`, nie szukaj błędu w typach ani w `move`, nawet jeśli tekst wygląda sensownie. Najpierw sprawdź, czy instrukcje są rozdzielone nową linią i czy nawiasy się domykają. Dopiero plik, który przechodzi czytanie, warto oddać analizie własności.

15.4 Dwa sposoby, w jakie czytanie odmawia

Pytanie brzmi, dlaczego jeden błąd składni wskazuje konkretny symbol, a inny potrafi wskazać sam koniec pliku. Oba kończą przebieg, bo przy błędzie czytania nie startują typy ani własność, ale powstają inaczej i inaczej pokazują miejsce. Gdyby wszystkie odmowy wyglądały tak samo, dałoby się ufać kolumnie w każdej linii parse. Tak nie jest, i warto to zobaczyć na przykładzie, zanim zaczniesz poprawiać plik według numeru linii.

Pierwszy sposób to odmowa gramatyki na nieznanym symbolu albo na nieoczekiwanym końcu pliku, i listing 13.2 jest właśnie tego rodzaju. Pozycja 1:24 obejmuje token `val`, którego w tym miejscu nie dało się wstawić, a treść mówi, czego gramatyka się spodziewała. Podobnie zachowuje się urwane wyrażenie, na przykład niedomknięty nawias. Komunikat zostaje jeden, fazy ownership i type nie dochodzą, i `--dump-arenas` nie ma czego wypisać poza tą jedną linią błędu, bo raportu regionów nie ma.

Drugi sposób dotyczy kształtu, który da się sparsować jako wyrażenie, ale nie jako cel przypisania. Po lewej stronie `=` musi stać nazwa albo nazwa z indeksem. Liczba, wywołanie albo inne wyrażenie jest składniowo wyrażeniem, więc gramatyka je czyta, a potem świadomie podnosi błąd. Ten błąd jest dopięty do końca pliku, a nie do złej lewej strony, bo tak skonstruowane jest zgłoszenie w gramatyce. W przebiegu poniżej plik ma trzy wiersze, a komunikat wskazuje wiersz 4 i kolumnę 1, czyli miejsce tuż za ostatnim znakiem.

Listing 13.3. Przypisanie, którego lewa strona nie jest nazwą

```
fun main() {  
    1 = 2  
}
```

```
/tmp/borkch/badassign.bork:4:1: error: parse: assignment target must be a name or name[index]
```

Treść mówi, co jest nie tak, i tej treści warto zaufać. Numerowi wiersza w tym jednym rodzaju błędu ufać nie należy, bo nie wskazuje 1. To ograniczenie sposobu, w jaki błąd użytkownika gramatyki jest dziś zamieniany na diagnostykę, a nie reguła języka, która zabraniałaby liczb po lewej stronie gdzie indziej niż na końcu pliku. Inne odmowy czytania, te od nieznanego symbolu, wskazują sam symbol, tak jak w listingu 13.2.

OSTRZEŻENIE. Błąd czytania jest w jednym przebiegu zawsze tylko jeden. Kompilator nie próbuje dojść do następnej instrukcji i zebrać kolejnych pomyłek składni, więc plik z dwoma urwanymi nawiasami pokaże tylko pierwszą. Poprawka pierwszej odmowy potrafi odsłonić następną, i to nie znaczy, że kompilator zmienił zdanie o reszcie programu.

Na końcu mapy, zgodnie z tym, że nazwy funkcji nie zastępują opisu. Wejście fazy to parse w `src/lib.rs`. Zamiana nowej linii w nawiasie na powrót karetki jest w `normalize_parenthesized_newlines` w `src/layout.rs`. Gramatyka, łącznie z odmową dla złej lewej strony przypisania, jest w `src/parser.lalrpop`. Drzewo, które przy sukcesie dostają sprawdzanie typów i analiza własności, jest zdefiniowane w `src/ast.rs`.

15.5 Podsumowanie

- Czytanie tekstu odpowiada na pytanie, czy plik da się ułożyć w drzewo składni, bo bez tego drzewa późniejsze fazy nie mają czego sprawdzać i nie powinny udawać, że program istnieje.
- Nowa linia kończy instrukcję, natomiast nowa linia bezpośrednio w nawiasach okrągłych jest przed gramatyką zamieniana na biały znak, żeby wywołanie dało się złamać między argumentami bez zmiany pozycji błędów.
- Drzewo pamięta kształt i zakres w pliku, a nie typ nazwy ani to, czy nazwę wolno skopiować, i tę niewiedzę zostawia świadomie kolejnym fazom.
- Nieznany symbol jest wskazywany w miejscu, w którym stoi, a błąd lewej strony przypisania jest dziś dopięty do końca pliku, więc przy tej jednej odmowie liczy się treść komunikatu.
- Jedna odmowa czytania kończy przebieg, bez raportu regionów i bez reprezentacji pośredniej, zgodnie z kolejnością opisaną w rozdziale 12.

16 Rozdział 14. Jak kompilator sprawdza własność nazw

Liczbę całkowitą wolno przeczytać wiele razy, bo odczyt nic nie zabiera. Napis albo tablica tak się nie zachowują. Taka wartość ma jednego właściciela, i drugi odczyt albo musi być jawnym przeniesieniem, albo, w przypadku niezmienniej nazwy czytanej z zewnętrznego bloku, świadomym współdzieleniem. Gdyby kompilator pominął to rozróżnienie, generator kodu albo skopiowałby deskryptor napisu w dwa miejsca, które oba uważają się za właściciela, albo pozwoliłyby użyć nazwy, która już nic nie trzyma.

Ten rozdział obejmuje

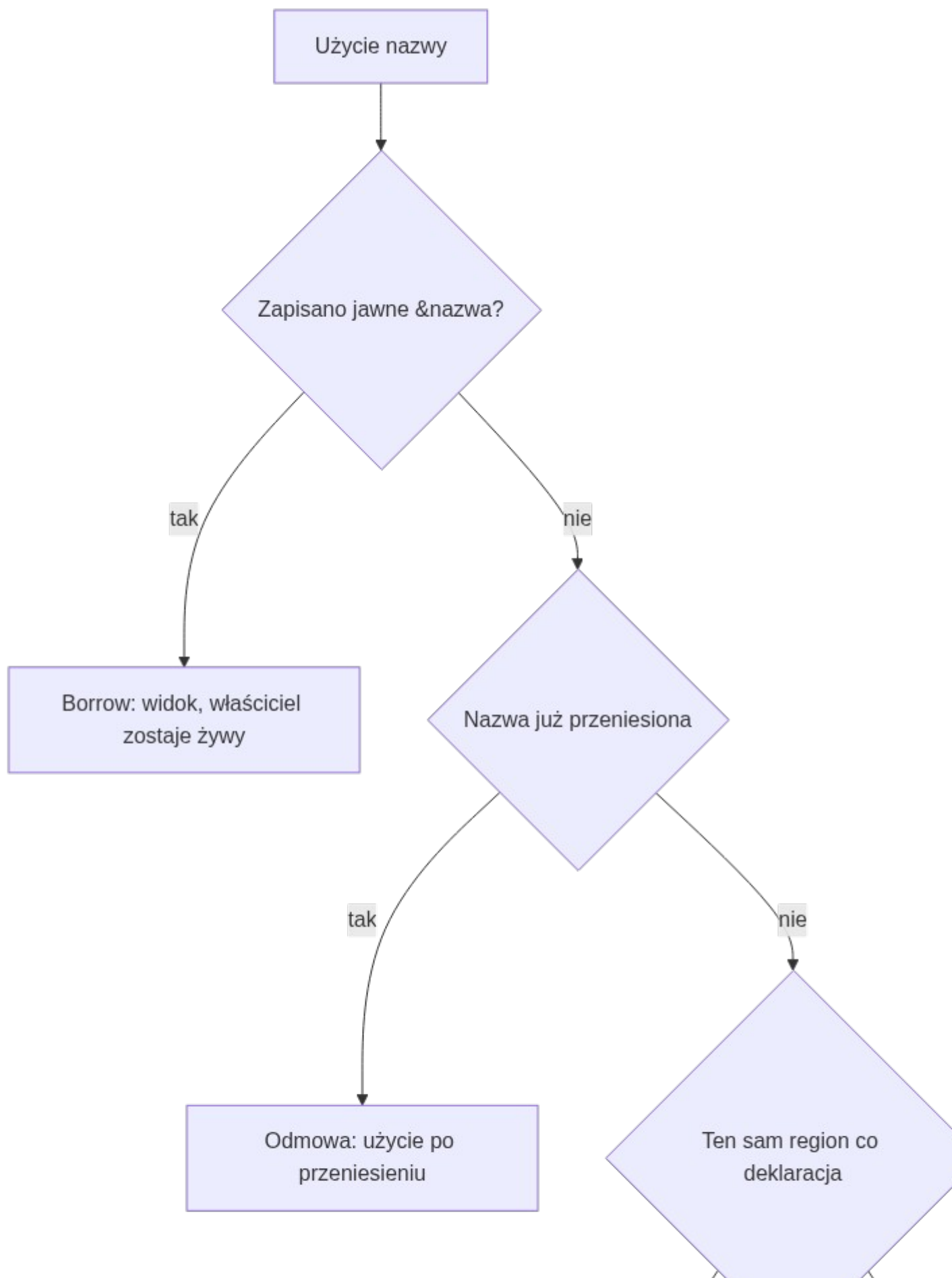
- pytanie, na które odpowiada analiza własności, i co psuje się w programie, gdy odpowiedzi brakuje,
- klasyfikacje odczytu nazwy oraz jawne pożyczanie przez `&nazwa`,
- zakaz przenoszenia nazwy z zewnątrz pętli do jej wnętrza,
- to, jak instrukcja `if` scala ślady przeniesienia z obu gałęzi,
- miejsce w źródłach, do którego warto zajrzeć po przeczytaniu reguł, a nie

zamiast nich.

16.1 Po co w ogóle pytać, kto nazwę posiada

Pytanie tej fazy brzmi, co wolno zrobić z nazwą w miejscu, w którym tekst jej używa. Wolno ją przeczytać na miejscu, wolno skopiować, wolno współdzielić z bloku, który ją zadeklarował, albo trzeba ją przenieść, a po przeniesieniu już nie wolno jej tknąć. Bez tej odpowiedzi późniejszy kod nie wie, czy odczyt `s` jest nieszkodliwy, czy właśnie oddaje jedyne prawo do napisu. Sprawdzanie typów tego nie rozstrzyga, bo napis i liczba potrafią mieć poprawny typ w wyrażeniu, które i tak łamie własność.

Analiza nie przydziela pamięci i nie woła funkcji wykonawczych. Buduje raport regionów, czyli drzewo, którego korzeniami są funkcje, a dziećmi bloki, pętle i gałęzie warunku. Przy nazwie zapisuje, jak została użyta. Region w tym raporcie nie jest buforem o stałej pojemności. Bufor, jeśli w ogóle powstanie, jest decyzją późniejszą, opisaną w rozdziale 16. Tutaj region jest tylko zakresem, w którym nazwa została zadeklarowana albo odczytana, i etykietą, którą zobaczysz w wydruku, na przykład `fun main` albo `Block`.



Rysunek 14.1. Goły odczyt nazwy i jawna pożyczka

Rysunek 14.1 rozdziela zwykły odczyt od jawnej pożyczki. Dla gołej nazwy obowiązuje dotychczasowa reguła: kopiowanie, współdzielenie albo przeniesienie. Zapis `&nazwa` jest osobną operacją, oznaczaną w raporcie jako `Borrow`: widok nie zabiera właścicielowi nazwy ani danych. Analiza typów dopuszcza ją tylko dla wartości niekopiowanych i w kontekstach opisanych w rozdziale 15. Kopiują się nienullowalne typy pierwotne, takie jak `i32`, `i64`, `f64` i `bool`; napis, tablica i typ z pytajnikiem się nie kopiują. Typ, którego ta analiza nie dostała z kolejki typów deklaracji, też nie jest traktowany jak kopia, nawet jeśli sprawdzanie typów w swoim drzewie widzi liczbę. Ten rozjazd dotyczy parametrów funkcji dopisanej na końcu wywołania i wraca w rozdziale 15.

16.2 Goły odczyt, pożyczka i ślad w wydruku

Pytanie, które tu warto rozebrać na przykładzie, brzmi, jak klasyfikacja zwykłego odczytu wygląda w programie, a nie w diagramie. Weźmy najpierw odczyt, który jest legalny, bo niezmienny napis z `main` jest tylko czytany w bloku wewnętrznym. To jest plik `10-shared.bork` z zestawu przykładów. Sprawdzenie kończy się kodem 0, a wydruk pokazuje współdzielenie, nie przeniesienie.

Listing 14.1. Niezmienny napis czytany w bloku wewnętrznym

```
fun main() {  
  val s = "x"  
  {  
    println(s)  
  }  
}
```

```
Arenas  
└─ fun main  
   └─ s [Local]  
      └─ Block  
         └─ s [Shared ← fun main]
```

`Local` przy deklaracji znaczy, że w swoim regionie nazwa jest zwykłym właścicielem. `Shared` przy odczycie w bloku znaczy, że blok nie przejął napisu, tylko czyta nazwę zadeklarowaną w `fun main`. `println` jest funkcją wbudowaną i ten odczyt przez współdzielenie przyjmuje. Gdyby `s` było `var`, a nie `val`, ten sam tekst zostałby odrzucony, bo zmienna nazwa, która nie jest kopią, nie wchodzi do obcego regionu bez `move`.

Listing 14.2. Zmienna nazwa użyta w bloku bez przeniesienia

```
fun main() {  
  var s = "hi"  
  {  
    val t = s  
  }  
}
```

```
}
```

```
/tmp/borkch/needmove.bork:4:17: error: ownership: `s` is not Copy; move it into `Block` with `move`
```

Arenas

```
└─ fun main
   └─ s [Local]
      └─ Block
         └─ t [Local]
```

Komunikat mówi wprost, co zrobić. Trzeba napisać `val t = move s`, i wtedy własność przechodzi do `t`, a `s` jest od tej pory zużyte. Drugi odczyt `s` po takim przeniesieniu dostaje osobną odmowę, tę o użyciu po przeniesieniu, którą listing 12.2 już pokazał na dodawaniu. Wydruk przy błędzie i tak powstaje, bo tekst się sparsuje. Nie wolno go czytać jako zgody na budowanie. Kod wyjścia jest 1, a reprezentacji pośredniej w wyniku nie ma.

Pozostałe odmowy odczytu działają według tego samego rysunku. Użycie nazwy, której nie zadeklarowano, jest błędem własności o nieznanym nazwie. Przeniesienie nazwy, która już została przeniesiona, mówi, skąd poszło pierwsze przeniesienie. Pełna lista komunikatów jest w `classify_use` i w funkcjach obok, w pliku `src/sema/policy.rs`.

Jawna pożyczka jest używana przede wszystkim przy parametrze `&T`. Wtedy deklaracja funkcji mówi, że wywołujący zachowuje własność, a argument musi mieć postać `&nazwa`. Raport zapisuje krawędź Borrow; zwykły odczyt nazwy `var` bez `&` nadal wymaga `move` albo daje błąd.

```
fun bump(buf: &[i32; 2]) {
    buf[0] = buf[0] + 1
}

fun main(): i32 {
    var a: [i32; 2] = [1, 2]
    bump(&a)
    return a[0]
}
```

Pożyczkę można ponowić bezpośrednio przy wywołaniu w `if`, `while` albo `for`, na przykład `inner(&buf)`. Nie można zapisać jej do nowej pożyczki w regionie potomnym (`val view = &buf`), bo widok nie może przeżyć regionu, w którym powstał. To jest wąska reguła regionowa, a nie pełny borrow checker Rusta.

NOTA. Słowo region w tym rozdziale znaczy węzeł raportu, a nie bufor wykonawczy o pojemności 4096 bajtów. Ten drugi byt też bywa w kodzie nazywany areną i jest opisany przy bibliotece wykonawczej. Wydruk `--dump-arenas` pokazuje węzły raportu, czyli zakresy nazw, a nie zawartość bufora.

16.3 Dlaczego pętla nie może przejąć nazwy z zewnątrz

Pytanie brzmi, co złego jest w przeniesieniu, które w pojedynczym bloku byłoby legalne, gdy to przeniesienie stoi w pętli. Pętla wykonuje ciało wiele razy, więc przeniesienie, które wygląda niewinnie w jednym obrocie, powtórzy się w następnym. Pierwszy obrót oddałby własność, a drugi próbowałby oddać ją jeszcze raz, z nazwy, która już nic nie trzyma. Kompilator nie czeka na drugi obrót w działającym programie. Widzi przeniesienie nazwy zadeklarowanej poza pętlą i odmawia od razu, bo żaden poprawny przebieg wielokrotny nie istnieje.

Listing 14.3. Przeniesienie nazwy zewnętrznej w pętli

```
fun main() {  
    var s = "hi"  
    for (i in 0..2) {  
        val t = move s  
    }  
}
```

```
/tmp/borkch/loopmove.bork:4:22: error: ownership: cannot move `s` inside a loop: it would  
already be moved on later iterations
```

Arenas

```
└─ fun main  
   └─ s [Local]  
      └─ ForLoop (i)  
         └─ i [Local]  
            └─ t [Local]
```

`i` jest lokalne w pętli i jest kopią, gdyby je odczytać w zagnieżdżonym bloku, bo zakres liczbowy daje wartości kopiowalne. Zakaz dotyczy `s`, które pochodzi z `main`. To samo dotyczy się pętli `while`. Nazwę zadeklarowaną w samym ciele pętli wolno w tym ciele przenosić, bo każdy obrót tworzy ją od nowa. Wydruk powyżej pokazuje `t` jako lokalne właśnie dlatego, że deklaracja stoi w pętli. Odmowa dotyczy źródła przeniesienia, nie celu.

OSTRZEŻENIE. Zakaz nie zależy od tego, ile razy pętla naprawdę się wykona. Nie obchodzi go warunek, który w praktyce wykonałby się raz, ani `break` po pierwszym przeniesieniu. Analiza nie próbuje udowodnić, że pętla kręci się najwyżej jeden raz, i każdy `move` nazwy z zewnątrz w `for` albo w `while` jest odrzucany.

16.4 Jak warunek scala dwa ślady przeniesienia

Pytanie brzmi, co kompilator ma powiedzieć o nazwie po instrukcji `if`, skoro jedna gałąź mogła ją przenieść, a druga nie. Gdyby uznać nazwę za zużytą już po jednej gałęzi, legalny program, w którym druga gałąź nazwę zostawia, też zostałby odrzucony. Gdyby nigdy nie uznawać jej za zużytą, program, który przenosi ją w obu gałęziach, a potem używa za warunkiem, przeszedłby sprawdzenie i zepsuł się dopiero w ruchu. Analiza uznaje nazwę za zużytą dopiero wtedy, gdy nie ma

gałęzi, która by ją zachowała. Po `if` z `else` nazwa jest zużyta wtedy, gdy była zużyta już przed warunkiem albo gdy przeniosły ją obie gałęzie.

Dlatego przeniesienie w obu gałęziach, a potem kolejne przeniesienie za warunkiem, jest odrzucane. W przebiegu poniżej komunikat mówi, że `s` zostało już przeniesione z `fun main`. Plik jest krótki, żeby widać było samą regułę scalania, bez drugiego, niezależnego błędu w gałęzi. Druga gałąź też używa `move`, więc po warunku nie zostaje żadna ścieżka, na której `s` jeszcze żyje.

Listing 14.4. Przeniesienie w obu gałęziach

```
fun main() {
    var s = "hi"
    if (1 > 0) {
        val t = move s
    } else {
        val u = move s
    }
    val v = move s
}
```

```
/tmp/borkch/ifboth2.bork:8:18: error: ownership: cannot move `s`: already moved from fun main
```

Przeniesienie tylko w jednej gałęzi, przy nieszkodliwej drugiej, nie zostawia nazwy zużytej za warunkiem, i kolejne `move` po takim `if` przechodzi sprawdzenie. Instrukcja `if` bez `else` jest jeszcze ostrożniejsza w drugą stronę. Ślad przeniesienia z samej gałęzi nie jest przenoszony do kodu za warunkiem, więc nazwa po jednostronnym `if` zostaje w stanie sprzed tej instrukcji. To jest reguła scalania, a nie pozwolenie, żeby w gałęzi użyć nazwy już wcześniej przeniesionej. Jeśli nazwa była zużyta, zanim warunek się zaczął, po warunku też jest zużyta.

Na końcu zostaje mapa do kodu, już po regułach, a nie zamiast nich. Wejście analizy to `analyze_with_decl_tys` w `src/sema/analyze.rs`, wołane z `check` po sprawdzeniu typów, bo potrzebuje kolejki typów deklaracji. Klasyfikację gołego odczytu liczy `classify_use` w `src/sema/policy.rs`; pożyczki odnotowuje `note_borrow` w `src/sema/walk.rs`. Scalanie śladu po `if` jest w `apply_moved_merge` w `src/sema/env.rs`. Zagnieżdżony blok, w którym jedyną instrukcją jest kolejny blok, jest spłaszczany przez `peel_blocks` w `src/sema/mod.rs`, żeby puste owinięcie klamrami nie tworzyło osobnego regionu. Wydruk, który widziałeś w listingach, składa `src/dump.rs`.

16.5 Podsumowanie

- Analiza własności rozróżnia użycie lokalne, kopię, współdzielenie, przeniesienie i jawne `Borrow`; pożyczka nie odbiera właścicielowi nazwy.
- Kopiują się nienullowalne typy pierwotne, a napis, tablica i typ z pytajnikiem wymagają albo współdzielenia niezmienniej nazwy, albo jawnego `move`.
- Po przeniesieniu nazwy drugi odczyt jest odrzucany, a przeniesienie nazwy

zadeklarowanej poza pętlą jest odrzucane od razu, bo kolejny obrót nie miałby już czego przenieść.

- Po `if` z `else` nazwa jest zużyta za warunkiem tylko wtedy, gdy przeniosły ją obie gałęzie albo była zużyta już wcześniej, a samo `if` bez `else` nie wynosi śladu przeniesienia na zewnątrz.
- Raport regionów opisuje zakresy nazw i zostaje także przy błędzie, natomiast nie jest buforem pamięci i nie zastępuje decyzji z rozdziału 16 o tym, gdzie napis fizycznie powstanie.

17 Rozdział 15. Jak kompilator sprawdza typy

Drzewo składni wie, że w pliku jest dodawanie, ale nie wie, czy wolno dodać te dwie rzeczy do siebie. Wie, że jest deklaracja, ale nie wie, czy prawa strona pasuje do typu napisanego po dwukropku. Gdyby generator kodu dostał takie drzewo wprost, musiałby sam zgadywać, czy literał `1` jest wartością `i32`, czy `i64`, i każda pomyłka kończyłaby się złą instrukcją albo awarią przy rzutowaniu wartości. Sprawdzanie typów zdejmuje to zgadywanie, zanim ktokolwiek emituje kod, i zostawia drugie drzewo, w którym typ jest już wpisany przy każdym wyrażeniu.

Ten rozdział obejmuje

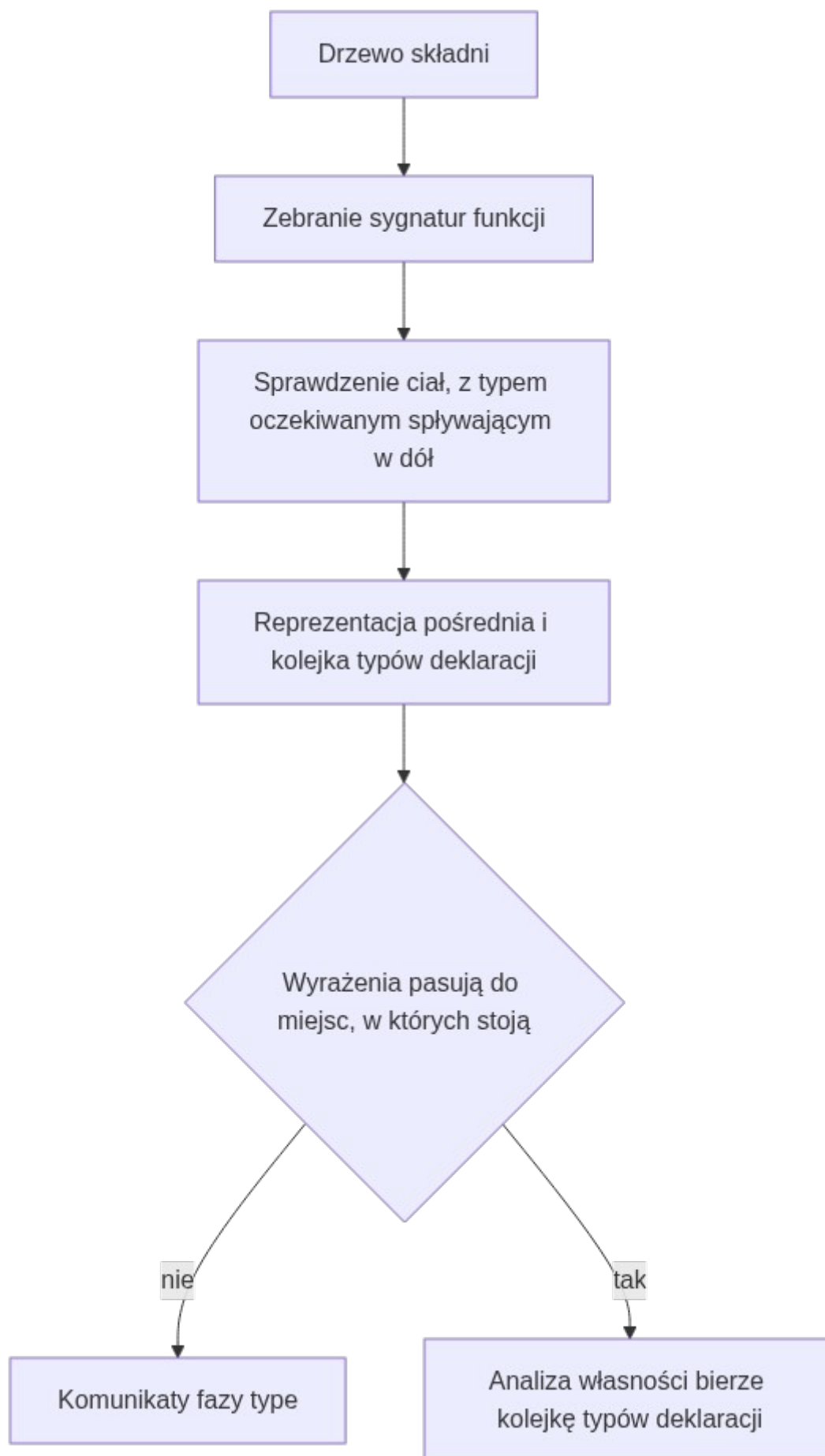
- pytanie, na które odpowiada sprawdzanie typów, i dlaczego nie wystarczy do tego samo drzewo składni,
- sposób, w jaki oczekiwany typ spływa w dół wyrażenia i zmienia odczyt literału,
- odmowy, które zostają nawet wtedy, gdy literał da się dopasować, na przykład pustą tablicę i zapis do `val`,
- granicę między tą fazą a analizą własności, która idzie tuż po niej,
- miejsce w źródłach, w którym powstaje reprezentacja pośrednia.

17.1 Po co wpisywać typ przy każdym wyrażeniu

Pytanie tej fazy brzmi, jaki typ ma każde wyrażenie i czy w miejscu, w którym stoi, taki typ jest dopuszczalny. Bez tej odpowiedzi nie da się odróżnić dodawania dwóch liczb od dodawania liczby do napisu, ani zwrotu `i32` z funkcji, która obiecała `i64`. Błąd wyszedłby dopiero przy emisji, często jako wewnętrzna panika kompilatora, a nie jako komunikat o Twoim pliku. Osobna faza zamienia te pomyłki w diagnostykę fazy `type` i, gdy nic nie odrzuci, w drzewo, któremu generator może ufać.

To drugie drzewo nazywa się reprezentacją pośrednią. W kodzie nosi skrót `HIR`, od

angielskiego high-level intermediate representation, i ten skrót zostaje w nazwach struktur HirProgram oraz HirExpr. W dalszych zdaniach pada sama nazwa reprezentacja pośrednia, bez powtarzania skrótu. Każde wyrażenie niesie w niej typ, zakres w pliku i rodzaj wyrażenia. Sprawdzanie typów buduje ją zawsze, nawet gdy zbierze błędy. Do wyniku przebiegu, opisanego w rozdziale 12, trafia ona tylko wtedy, gdy lista komunikatów na końcu jest pusta. Dzięki temu generator nie dostaje drzewa, obok którego leżą odmowy.



Rysunek

15.1. Od drzewa składni do reprezentacji pośredniej

Rysunek 15.1 zaczyna się od sygnatur, bo ciało funkcji wolno sprawdzać dopiero wtedy, gdy wiadomo, jakie funkcje w ogóle istnieją i co przyjmują. Przy okazji kompilator odmawia ponownego zdefiniowania funkcji wbudowanej, takiej jak `println` albo `concat`, bo te nazwy są już zajęte przez język. Dopiero po zebraniu sygnatur sprawdzanie schodzi do ciał. Kolejka typów deklaracji, którą rysunek stawia obok reprezentacji pośredniej, jest osobnym produktem tej samej fazy. Analiza własności zdejmuje z niej typ przy każdej deklaracji, w kolejności tekstu, i dlatego sprawdzanie typów musi skończyć się wcześniej, choć jego komunikaty są dopisywane do wydruku na końcu.

17.2 Dlaczego literał 1 nie ma jednego typu

Pytanie, które najłatwiej pomylić z prostą tabelą, brzmi, skąd bierze się typ liczby zapisanej bez przyrostka. W wielu językach 1 jest zawsze `i32` albo zawsze `int`, a resztę załatwia rzutowanie. Bork robi inaczej tam, gdzie otoczenie już wie, jakiego typu liczbowego się spodziewa. Oczekiwany typ spływa w dół, do literału, i literał przyjmuje właśnie ten typ, o ile jest liczbowy. Gdy otoczenie nic nie oczekuje, literał zostaje `i32`. Dzięki temu da się podać 1 funkcji, która chce `i64`, bez dopisku przy literale, a jednocześnie gołe `val n = 1` nie staje się niespodziewanie typem szerszym.

Widać to na parze programów, które różnią się tylko tym, czy do funkcji trafia literał, czy nazwa już zadeklarowana. Funkcja `f` chce `i64` i zwraca `i32`, żeby wynik dało się zwrócić z `main`. Pierwszy plik powinien przejść, bo literał stoi dokładnie w miejscu, które oczekuje szerszego typu. Drugi powinien odpaść, bo nazwa zdążyła dostać typ `i32`, zanim ktokolwiek poprosił o `i64`.

Listing 15.1. Literał w miejscu, które oczekuje `i64`

```
fun f(x: i64): i32 {
    return 0
}
fun main(): i32 {
    return f(1)
}
```

Ten plik przechodzi sprawdzenie z kodem 0. Literał 1 stoi w argumencie, argument oczekuje `i64`, więc 1 jest wartością `i64`, a nie `i32`, które dostałby w gołej deklaracji. Ta sama liczba zapisana najpierw do nazwy bez adnotacji jest już `i32`, i tej nazwy nie wolno potem wstawić w miejsce `i64`, bo nazwa ma typ ustalony przy deklaracji. Oczekiwany typ nie wraca wstecz do wcześniejszej linii.

Listing 15.2. Nazwa typu `i32` w miejscu, które oczekuje `i64`

```
fun f(x: i64): i32 {
```

```

    return 0
}
fun main(): i32 {
    val n = 1
    return f(n)
}

```

```

/tmp/borkch/namearg.bork:6:14: error: type: argument 1 to `f` has type i32, expected i64

```

Komunikat nazywa argument, typ zastany i typ oczekiwany. Kompilator nie próbuje przy tym sam rzutować `n` na szerszy typ. Gdyby to zrobił, błąd w szerokości liczby chowałby się w milczącym przekształceniu i wychodził dopiero w wyniku, który nie mieści się w tym, co programista napisał. Ten sam mechanizm dotyczy także instrukcji zwrotu. Wyrażenie `return 1` w funkcji o wyniku `i64` przechodzi, bo oczekiwany typ dociera do literału, natomiast `val n = 1 + 2` i potem `return n` z takiej funkcji już nie, bo suma bez oczekiwanego typu jest `i32`.

WSKAZÓWKA. Gdy komunikat mówi, że zastany typ to `i32`, a oczekiwany to `i64`, sprawdź, czy wartość powstała w miejscu bez adnotacji i bez kontekstu liczbowego. Literał wpisany od razu w argument albo w `return` często przyjmie szerszy typ, a ta sama cyfra zapisana wcześniej do `val` już nie.

17.3 Co pozostaje błędem, choć otoczenie coś oczekuje

Pytanie brzmi, których braków kompilator nie uzupełni, nawet gdy oczekiwany typ spływa w dół. Nie każdy węzeł da się uratować kontekstem. Pusta tablica nie ma ani typu elementu, ani długości, a wpisanie ich z powietrza, bez adnotacji przy nazwie, zgadłoby strukturę, której w tekście nie ma. Zapis do nazwy zadeklarowanej jako `val` jest z kolei błędem niezależnie od typu, bo `val` nie jest miejscem, do którego wolno przypisać po raz drugi. Oba przypadki warto zobaczyć, bo wyglądają jak coś, co „dałoby się wywnioskować”, a jednak są odrzucane.

Listing 15.3. Pusta tablica bez adnotacji

```

fun main() {
    val a = []
}

```

```

/tmp/borkch/emptyarr.bork:2:13: error: type: empty array literal requires an explicit type, e.g.
`val a: [i32; 0] = []`

```

Komunikat podaje kształt adnotacji, który tę samą pustą tablicę przeprowadza przez sprawdzenie. Chodzi o to, że długość i typ elementu są częścią typu tablicy w Borku, a nie szczegółem, który wolno dopisać później w generatorze. Plik z adnotacją `[i32; 0]` przy tej samej pustej tablicy kończy się kodem `0`, co potwierdza, że brakuje informacji w tekście, a nie że pusta tablica jest w ogóle zabroniona. Podobnie odrzucany jest zapis do `val`, i tu oczekiwany typ w ogóle nie

wchodzi w grę.

Listing 15.4. Drugi zapis do niezmienniej nazwy

```
fun main(): i32 {  
    val n = 1  
    n = 2  
    return n  
}
```

```
/tmp/borkch/valas.bork:3:5: error: type: cannot assign to immutable `val` binding `n`
```

Pozostałe odmowy tej fazy są tego samego rodzaju. Warunek pętli albo `if` musi być `bool`, zwrot musi pasować do typu funkcji, a dwa operandy dodawania muszą być tym samym typem liczbowym. Lista operatorów nie wnosi tu nowego mechanizmu. Gdy typy operandów się rozmiągają, komunikat podaje oba, tak jak przy dodawaniu liczby do napisu, i na tym kończy się ta gałąź. Pełne zestawienie miejsc, w których typ oczekiwany jest porównywany z typem zastanym, jest w `src/typeck/stmt.rs` i w katalogu `src/typeck/expr/`.

17.4 Typ pożyczki &T

Typ `&T` oznacza widok na istniejącą wartość niekopiowaną. Typy proste, takie jak `i32`, są kopiowalne i nie można ich pożyczyć. Pożyczony parametr musi być `val`, a lokalny widok można związać tylko przez `val`; `var p: &T` jest odrzucany. Typy nullable-reference także nie są obsługiwane, a zwrot pożyczki blokuje analiza ucieczki.

```
fun bump(buf: &[i32; 2]) {  
    buf[0] = 1  
}  
  
fun main() {  
    var a: [i32; 2] = [1, 2]  
    bump(&a)  
}
```

Typ parametru wymusza jawne `&` przy wywołaniu. Samo `bump(a)` nie jest skrótem dla pożyczki ani dla przeniesienia; checker wymaga argumentu pożyczającego. W przypadku `&[T; N]` funkcja może przypisywać do `buf[i]`, co zmienia bufor właściciela. To szczególna reguła Borka: nie ma `&mut`, ale modyfikacja elementów tablicy przez pożyczony parametr jest dozwolona.

NOTA. Sprawdzanie typów nie pyta, czy nazwę wolno skopiować albo przenieść. Wyrażenie `move` i promocja zapisana w tekście stają się w reprezentacji pośredniej zwykłą nazwą z dopiskiem, jakiego użycia sprawdzanie się dopatrzyło. Odmowę, że nazwa została użyta po przeniesieniu, zgłasza analiza z rozdziału 14, nawet jeśli typ tego wyrażenia jest poza tym w porządku.

17.5 Gdzie ta faza się kończy, a gdzie tylko zapisuje

Pytanie brzmi, czego świadomie nie ma w komunikatach fazy type, choć reprezentacja pośrednia już coś o tym pamięta. Generator kodu później ufa rodzajowi użycia zapisanemu przy nazwie, gdy kopiuje deskryptor napisu albo liczbę. Ten zapis nie jest jednak werdyktem własności. Werdykt własności przychodzi z osobnego przejścia po drzewie składni i potrafi, przy funkcji dopisanej na końcu wywołania, widzieć typ nieznany tam, gdzie sprawdzanie typów widzi kopię. Dwa przejścia mają dwa środowiska, i rozdział 14 pokazał skutek od strony współdzielenia. Tutaj ten sam skutek widać od strony kolejki. Parametry takiej funkcji nie trafiają do kolejki typów deklaracji, którą zdejmuje analiza, więc analiza nie dostaje typu wyliczonego w zagnieżdżonym zakresie.

Dla zwykłych funkcji i dla main oba opisy trzymają się razem, i testy kompilatora tego pilnują. Warto o rozjeździe wiedzieć tylko po to, żeby wydruk regionów przy błędzie w funkcji dopisanej na końcu wywołania nie wyglądał jak sprzeczność z komunikatem o typie. To wciąż ten sam program, czytany dwa razy, z inną ilością informacji o parametrach. Żadna z tych faz nie naprawia drugiej w locie.

Na końcu zostaje mapa do kodu. Wejście fazy to check w `src/typeck/mod.rs`. Oczekiwany typ jest argumentem sprawdzenia wyrażenia w `src/typeck/expr/mod.rs` i schodzi do literałów, wywołań i operandów. Porównanie typu zastanego z oczekiwanym przy deklaracji, zwrocie i przypisaniu jest w `src/typeck/stmt.rs`. Struktury reprezentacji pośredniej, razem z rodzajem użycia nazwy, są w `src/hir/`. Kolejka typów deklaracji opuszcza tę fazę jako drugi wynik check i jest zużywana przez analizę własności, zanim powstanie wydruk, o którym mówił rozdział 12.

17.6 Podsumowanie

- Sprawdzanie typów wpisuje typ przy każdym wyrażeniu i odrzuca miejsce, w którym typ zastany nie pasuje do oczekiwanego, bo generator kodu nie powinien tego zgadywać sam.
- Literał liczby przyjmuje typ liczbowy oczekiwany przez otoczenie, a w braku takiego otoczenia zostaje `i32`, więc `f(1)` przy parametrze `i64` przechodzi, a nazwa zadeklarowana jako `gołe 1` już nie.
- Pusta tablica bez adnotacji i drugi zapis do `val` są odrzucane także wtedy, gdy reszta otoczenia ma jasny typ, bo kompilator nie dopisuje długości tablicy ani nie zdejmuje niezmienności.
- `&T` może opisywać parametr lub stały widok na niekopiowaną wartość; wymaga jawnego `&nazwa` przy wywołaniu.
- Ta faza nie rozstrzyga własności, tylko zapisuje rodzaj użycia nazwy w reprezentacji pośredniej, a odmowy o przeniesieniu zostawia analizie z poprzedniego rozdziału.
- Reprezentacja pośrednia trafia do wyniku przebiegu tylko przy pustej liście

komunikatów, natomiast kolejka typów deklaracji jest budowana zawsze i zasila analizę własności.

18 Rozdział 16. Jak kompilator wybiera miejsce na napis

Analiza własności potrafi powiedzieć, że nazwa została przeniesiona, i na tym poprzestać. Nie mówi jeszcze, w którym buforze powstanie napis ani czy jego pamięć pożyje dość długo, żeby funkcja mogła go zwrócić. Gdyby pominąć te pytania, program z poprawnym move i tak mógłby oddać napis, którego pamięć znika razem z blokiem albo razem z wołaną funkcją. Dlatego po czystym sprawdzeniu typów i własności kompilator zadaje jeszcze trzy pytania o miejsce, i dopiero ich odpowiedzi wolno uznać za koniec sprawdzenia.

Ten rozdział obejmuje

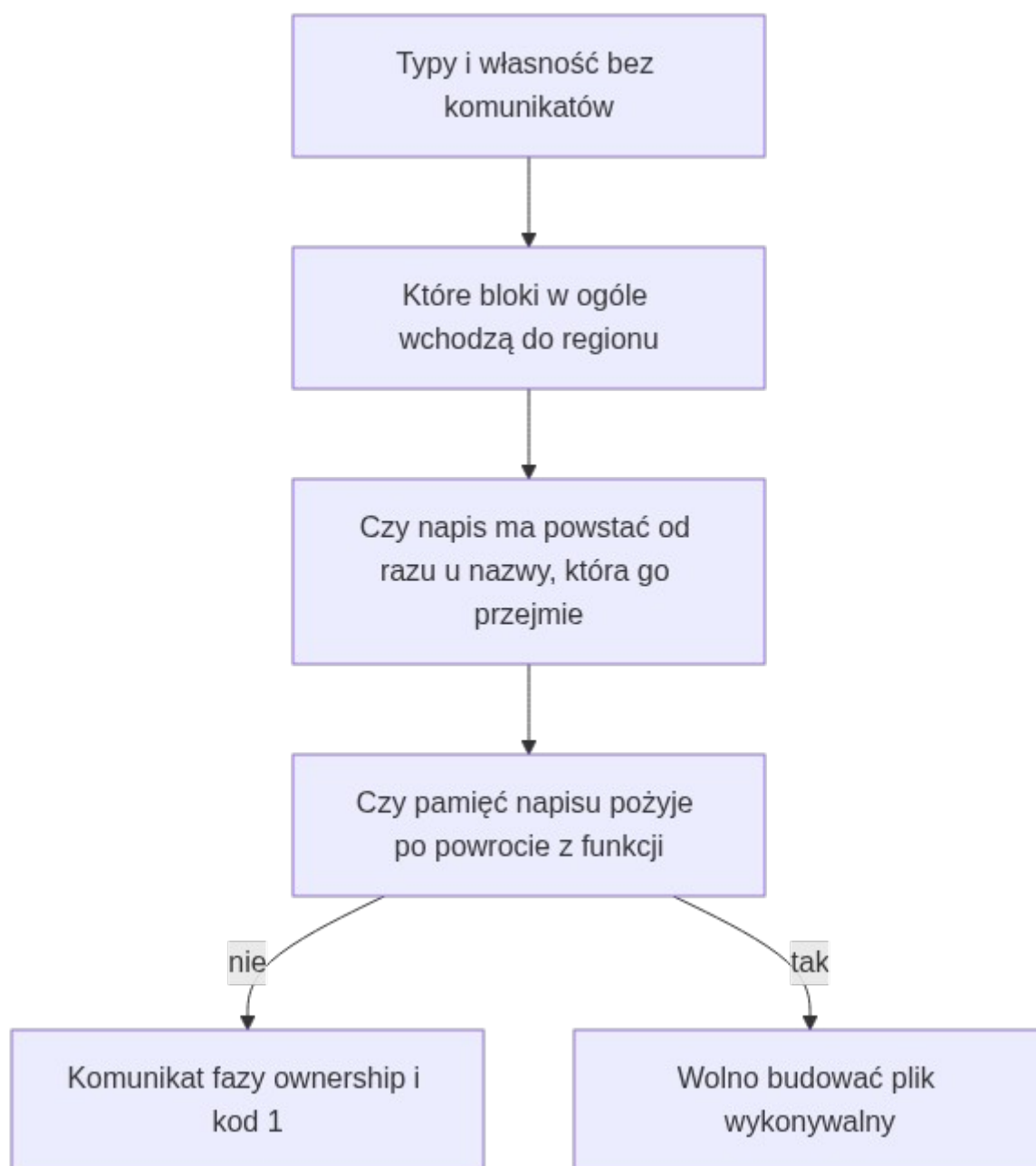
- pytanie, które bloki w ogóle muszą wejść do własnego regionu przy wykonaniu,
- pytanie, czy napis deklarowany tylko po to, by zaraz oddać go nazwie zewnętrznej, ma powstać od razu u niej,
- pytanie, czy pamięć zwracanego napisu pożyje po powrocie z funkcji,
- pełny przykład, w którym napis nadpisuje nazwę z zewnątrz i nadal da się go wypisać,
- odmowę zwrotu wyniku concat, z prawdziwym komunikatem tej fazy.

18.1 Po co pytać o miejsce, skoro własność już jest sprawdzona

Pytanie tej części brzmi, gdzie ma leżeć treść wartości, która nie jest kopią, i jak długo to miejsce wolno jej zajmować. Liczba całkowita nie ma tego problemu, bo mieści się w samej wartości i kopiuje się przy odczycie. Napis i tablica trzymają treść inaczej, bo leży ona w buforze regionu, a nazwa pamięta tylko opis, gdzie ta treść jest i jak jest długa. Gdy region się kończy, bufor jest zwalniany, i każdy opis, który wciąż na niego wskazuje, zostaje wskaźnikiem do zwolnionej pamięci, po angielsku dangling pointer. Odczyt przez taki wskaźnik nie jest już odczytem napisu, tylko odczytem pamięci, która do tego napisu nie należy. Własność z rozdziału 14 pilnuje, żeby dwóch właścicieli nie było naraz. Ten rozdział pilnuje, żeby jedyny właściciel nie przeżył swojego bufora.

Te trzy pytania padają tylko wtedy, gdy lista komunikatów po typach i własności jest pusta. Przy błędzie składni, typu albo przeniesienia kompilator nie wybiera miejsca na napis, bo i tak nie będzie generował kodu. Gdy pytania padną i któreś odpowie odmową, reprezentacja pośrednia znika z wyniku tak samo jak przy

błędzie typu, a raport regionów zostaje. Komunikat takiej odmowy ma fazę ownership, nie osobną nazwę, bo z punktu widzenia wydruku to wciąż skarga o czas życia napisu, a nie o kształt tekstu.



Rysunek 16.1. Kolejność trzech pytań o miejsce napisu w jednym przebiegu

Rysunek 16.1 jest kolejnością w jednym przebiegu, a nie trzema niezależnymi narzędziami. Wejście do regionu przy wykonaniu jest potrzebne tylko blokowi, który naprawdę coś alokuje. Blok, który jedynie czyta liczbę albo współdzieli napis zadeklarowany wyżej, nie dostaje własnego bufora, bo nie miałby czego w nim położyć. To jest pierwsza odpowiedź i ona nie zmienia tekstu programu. Zaznacza tylko, które węzły raportu mają przy generowaniu kodu wołać wejście i wyjście regionu.

18.2 Gdzie powstaje napis, który zaraz zmienia właściciela

Pytanie brzmi, co zrobić z napisem, który w tekście stoi w bloku wewnętrznym, ale jego jedynym dalszym życiem jest nazwa z bloku zewnętrznego. Gdyby napis powstał w buforze wewnętrznym, koniec bloku zwolniłby go, zanim zewnętrzna nazwa skończyłaby z niego korzystać. Dałoby się to odrzucić, i część takich zapisów rzeczywiście jest odrzucana. Da się też zauważyć, że nazwa wewnętrzna jest tylko przystankiem, i położyć napis od razu w regionie nazwy, która go przejmie. Kompilator robi to drugie wtedy, gdy układ instrukcji jest jednoznaczny.

Jednoznaczny układ da się rozpoznać po dwóch sąsiednich instrukcjach. Deklaracja, której prawa strona jest napisem albo wywołaniem, stoi tuż przed przypisaniem, a to przypisanie przenosi właśnie tę nazwę do nazwy widocznej na zewnątrz. Wtedy deklaracja dostaje dopisek, że alokacja ma iść do tamtej nazwy zewnętrznej. Inne układy tego dopisku nie dostają. Zwykle przypisanie literału do nazwy zewnętrznej, bez pośredniej deklaracji, rozwiązuje się inaczej, ale z tego samego powodu. Literał jest kładziony w regionie nazwy, która go przyjmuje, a nie w bloku, w którym akurat stoi znak równości.

Widać to na programie, który nadpisuje `outer` literałem z bloku i potem ten napis wypisuje. Plik `15-assign-up.bork` przechodzi sprawdzenie z kodem 0. Samo sprawdzenie nie wypisuje miejsca alokacji, bo wydruk regionów tego nie pokazuje, więc o tym, że napis przeżył blok, rozstrzyga dopiero uruchomienie. Poniższy tekst jest całym programem, bez skrótów.

Listing 16.1. Nadpisanie nazwy zewnętrznej literałem z bloku

```
fun main() {  
    var outer = "a"  
    {  
        outer = "b"  
    }  
    println(outer)  
}
```

Zbudowany program wypisuje `b` i kończy się kodem 0. Literał `"a"` żyje tylko do przypisania, a `"b"` musi przeżyć koniec bloku, bo `println` stoi już w `main`. Gdyby napis `"b"` został w buforze bloku, wypisanie po klamrze byłoby czytaniem zwolnionej pamięci. Dlatego miejsce tego literału jest region `outer`, a blok wewnętrzny nie staje się właścicielem treści. Podobny efekt daje `promote` w pliku `14-promote.bork`, który też przechodzi sprawdzenie. Słowo `promote` mówi w tekście to, co przy zwykłym literale kompilator uzupełnia sam.

NOTA. Dopisek o alokacji przy deklaracji nie jest widoczny w `--dump-arenas`. Wydruk regionów pokazuje, kto nazwę posiada, a nie to, w czym buforze powstanie treść napisu. O tym drugim mówi dopiero ten rozdział, i tylko dla programu, który typy oraz własność już przepuściły.

18.3 Czego nie wolno zwrócić

Pytanie brzmi, które napisy wolno oddać wywołującemu, a które umarłyby razem z funkcją, która je stworzyła. Parametr i lokalna nazwa, zwrócone wprost, są dozwolone, i tak samo dozwolony jest literał napisu. Kompilator zakłada wtedy, że treść da się utrzymać tak długo, jak wynik. Inaczej jest z wartością, która powstaje w czasie wywołania i nie ma własnej, trwalszej nazwy. Wynik `concat` powstaje właśnie w ten sposób i nie ma trwalszej nazwy, która utrzymałaby jego treść. Jego treść leży w regionie wołanej funkcji, a ten region kończy się przy powrocie, więc opis zwrócony na zewnątrz wskazywałby bufor, którego już nie ma.

Listing 16.2. Zwrot wyniku `concat`

```
fun shout(): String {  
    return concat("a", "b")  
}  
fun main(): i32 {  
    return 0  
}
```

```
/tmp/borkch/retconcat.bork: error: ownership: returning the result of `concat` is not supported  
yet: returned `String` bytes must outlive the callee arena
```

Komunikat nie podaje w tym przebiegu numeru wiersza, tylko plik i treść, i ta treść wystarczy, żeby wiadomo było, czego nie robić. Bajty wyniku `concat` musiałyby przeżyć region wołanego, a dzisiejsza implementacja tego nie zapewnia, więc zwrot jest odrzucany, zamiast budować program, który zepsuje się przy odczycie. Zwrot przez `move` albo `promote` jest odrzucany osobnym komunikatem, który każe napisać `return s` dla parametru albo lokalu, ewentualnie zwrócić sam literał. To nie jest sprzeczność z listingiem 16.1. Tam napis zostawał w tej samej funkcji, w nazwie, która go przeżywa. Tutaj miałby opuścić funkcję razem z buforem, który właśnie się kończy.

Pozostałe odmowy tej kontroli są o tym samym. Przypisanie do nazwy wartości, której pamięć leży w regionie wewnętrznym, kończącym się wcześniej niż ta nazwa, jest odrzucane, gdy nie da się położyć jej od razu u celu. Gałąź `if`, która przenosi napis i sama jest wartością warunku, też nie może oddać napisu ze swojego regionu, bo ten region znika razem z gałęzią. Pełna lista takich miejsc jest w `check_function` w pliku `src/escape.rs`. Nie wnosi ona nowego pytania ponad to, czy opis napisu nie przeżywa bufora, w którym leży jego treść.

OSTRZEŻENIE. To, że `return s` dla lokalnego napisu przechodzi sprawdzenie, nie znaczy, że każdy sposób oddania tego samego napisu jest równoważny. `return move s` jest dziś odrzucany, choć właściwościowo wygląda jak przekazanie jedyne go właściciela. Komunikat mówi, jakiej formy zwrotu kompilator oczekuje, i tej formy trzeba się trzymać.

Na końcu mapy, już po tym, co fazy rozstrzygają. Zaznaczenie, które bloki wchodzi do regionu przy generowaniu kodu, robi `stamp_codegen_push` w `src/region_walk.rs`. Dopisek, że deklaracja ma alokować u nazwy zewnętrznej, wpisuje `annotate` w `src/hoist.rs`. Kontrolę zwrotów i przypisań, które przeżyłyby swój bufor, robi `check_function` w `src/escape.rs`. Wszystkie trzy woła `check` w `src/frontend.rs`, i tylko na pustej liście komunikatów z typów oraz własności.

18.4 Podsumowanie

- Po czystych typach i własności kompilator pyta jeszcze, które bloki alokują, gdzie ma powstać napis przejmowany przez nazwę zewnętrzną i czy zwrot przeżyje powrót z funkcji.
- Blok, który niczego nie alokuje, nie dostaje własnego bufora przy wykonaniu, bo region w raporcie nie jest jeszcze pamięcią, tylko zakresem nazw.
- Literał albo przeniesienie, którego jedynym dalszym życiem jest nazwa z szerszego zakresu, powstaje w regionie tej nazwy, dlatego program z listingu 16.1 wypisuje `b` już po zamknięciu bloku.
- Zwrot wyniku `concat` oraz zwrot przez `move` są odrzucane, bo pamięć napisu nie przeżyłaby regionu wołanej funkcji, natomiast zwrot lokalnej nazwy albo literału jest dozwolony.
- Odmowy tej fazy mają w wydruku fazę `ownership` i usuwają reprezentację pośrednią z wyniku, więc budowanie się nie zaczyna, dopóki miejsce na napis nie jest rozstrzygnięte.

19 Rozdział 17. Jak powstaje kod maszynowy

Czyste sprawdzenie mówi tylko tyle, że program jest do przyjęcia jako tekst Borka. Nie jest jeszcze plikiem, który da się uruchomić. Między jednym a drugim stoi tłumaczenie na kod maszynowy, i to tłumaczenie ma własne odmowy, bo generator nie umie każdej konstrukcji, którą wcześniejsze fazy już opisują. Gdyby puścić je wszystkie do emisji, część skończyłaby się czytelnym komunikatem, a część awarią procesu kompilatora, ze śladem stosu Rusta zamiast wskazania na Twój plik.

Ten rozdział obejmuje

- pytanie, co jeszcze musi się udać po czystym sprawdzeniu, zanim powstanie plik wykonywalny,
- kontrolę, która odrzuca kształty nieobsłużone przez emisję, zanim ta emisja w ogóle ruszy,

- sposób, w jaki napis jest reprezentowany w kodzie pośrednim, i z czym ten kod jest łączony,
- ograniczenia i znane błędy codegenu, w tym porównania liczb zmiennoprzecinkowych,
- program z rozdziału 12 doprowadzony aż do uruchomienia.

19.1 Po co osobna kontrola tuż przed emisją

Pytanie tej fazy brzmi, czy czystą reprezentację pośrednią da się przetłumaczyć na kod, który ten kompilator umie wyemitować. Sprawdzanie typów odpowiada, czy program ma sens w języku; nie gwarantuje obsługi każdego kształtu przez generator. `Nullable String?` i wartości typów prostych poza `unit` są obsługiwane dla `None`, `Some`, `?:`, `!!` oraz `==/!=`. Tablice nullable odrzuca sprawdzanie typów, a nullable `unit` i typy funkcji nie mają reprezentacji. Funkcja dopisana na końcu wywołania oraz niedozwolony wynik `main` nadal są odrzucane.

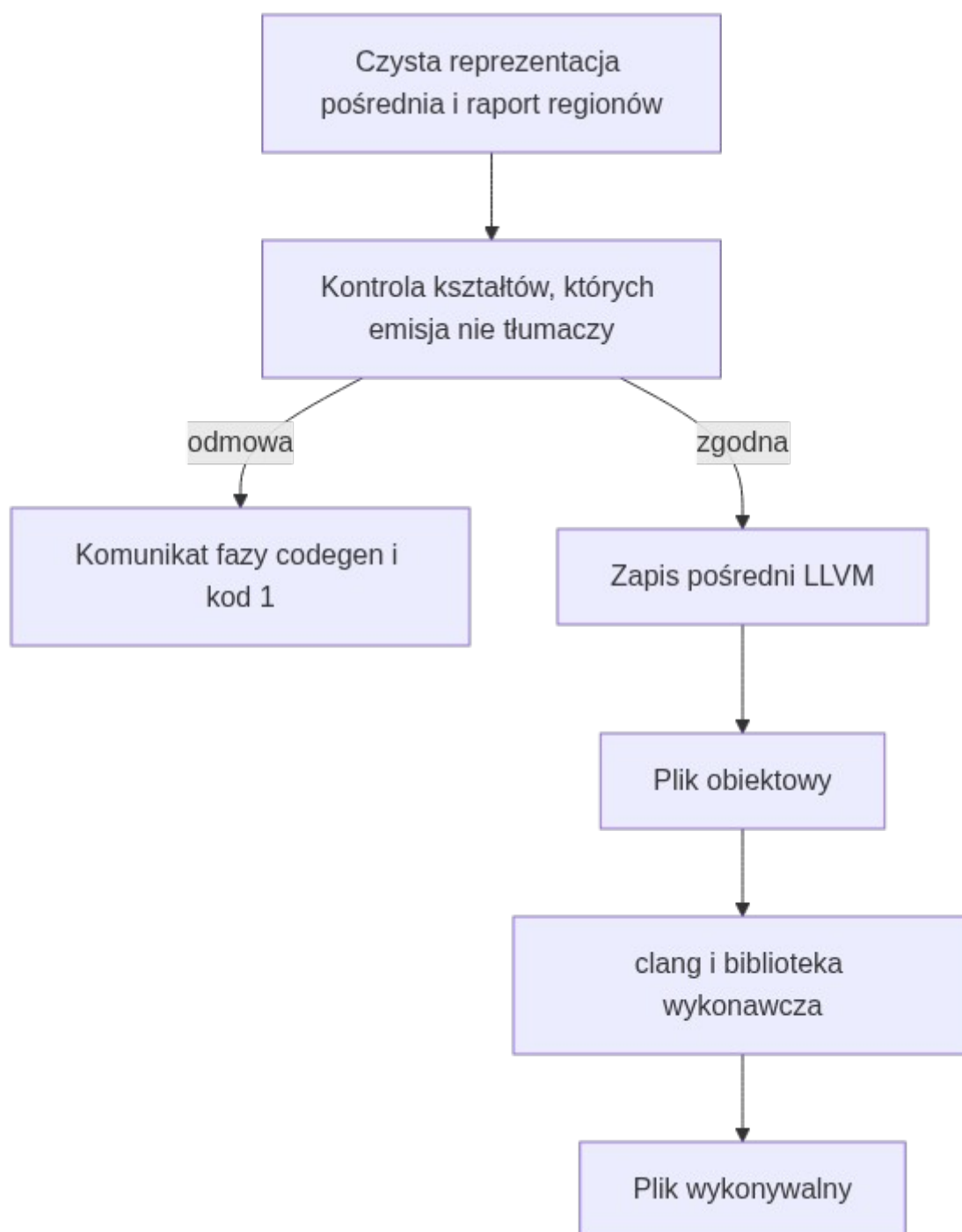
Kontrola ogląda kształt drzewa, a nie treść napisu w pamięci. Szuka między innymi braku funkcji `main`, niedozwolonego wyniku `main`, nieobsługiwanych operatorów, dostępu do pola i funkcji dopisanej na końcu wywołania. `None`, `Some`, `?:` i `!!` przechodzą bramkę, gdy ich typ nullable ma reprezentację. Gdy kontrola odrzuci program, dostajesz komunikat fazy codegen i kod wyjścia 1, bez pliku wynikowego. Pełna lista kształtów jest w funkcji `gate` w pliku `src/codegen/gate.rs`.

Listing 17.1. Nullable napis: zbudowanie, asercja i wypisanie wartości

```
fun main() {
    val s: String? = Some("x")
    println(s!!)
}
```

```
x
```

Listing odpowiada `fixture nullable_assert.bork`: sprawdzenie i budowanie przechodzą, a program wypisuje `x`. Brak funkcji `main` i niedozwolony typ jej wyniku nadal są odmowami codegenu; `main` bez jawnego typu wyniku jest zamieniane na `i32` równe zero.



Rysunek 17.1. Od czystego programu do pliku wykonywalnego

Rysunek 17.1 stawia kontrolę przed zapisem pośrednim celowo. Zapis pośredni powstaje dopiero dla programu, który kontrola przepuściła, a plik obiektowy powstaje z tego zapisu przez maszynę docelową LLVM. Na końcu clang łączy plik obiektowy z biblioteką `libbork_runtime.a`. Bez tej biblioteki wygenerowany kod nie miałby bufora regionu, wypisywania ani sprawdzeń, które przerywają proces przy dzieleniu przez zero i przy indeksie poza tablicą.

NOTA. Arytmetyka f32/f64 ma ścieżkę w `emit_float_binary`.
Porównanie przechodzi sprawdzanie typów, ale dispatcher wybiera

ścieżkę na podstawie typu wyniku, którym jest `bool`, i przekazuje `float` do `value_as_int`. Odtworzenie porównania `f64` kończy się paniką kompilatora (`FloatValue` zamiast `IntValue`). To usterka codegenu, nie reguła typów. `Float` jako argument funkcji jest osobno odrzucany przez `coerce_value_to_ty`.

19.2 Jak napis wygląda w kodzie pośrednim

Pytanie brzmi, co generator właściwie emituje dla napisu, skoro w Borku napis nie jest liczbą. W zapisie pośrednim LLVM napis i wycinek tablicy są strukturą z dwóch pól. Pierwsze jest wskaźnikiem na bajty, drugie jest długością typu `i64`. Nie ma na końcu bajtu zerowego, bo długość jest osobno, i właśnie ten opis widzi biblioteka wykonawcza, gdy wypisuje napis. Liczby całkowite i logiczne zostają liczbami LLVM, bez takiej struktury. Dzięki temu kopiowanie liczby jest kopiowaniem wartości, a kopiowanie napisu, tam gdzie analiza na to pozwala, jest kopiowaniem dwóch pól, nie treści.

Funkcje inne niż `main` dostają w module nazwy z przedrostkiem `bork_`, żeby nie zderzyły się z nazwami z biblioteki C. Funkcja `main` zostaje w module pod zwykłą nazwą `main`, bez przedrostka. Wejście do regionu, wyjście z niego, czyszczenie bufora i alokacja w buforze są wywołaniami biblioteki wykonawczej, odpowiednio `bork_arena_push`, `bork_arena_pop`, `bork_arena_reset` i `bork_arena_alloc`. Wypisanie liczby i napisu idzie przez `bork_println_i64` oraz `bork_println_str`. Pojemność jednego bufora to 4096 bajtów i ta stała żyje w bibliotece, czyli w procesie uruchomionego programu, a nie w analizie własności. Pozostałe symbole biblioteki są wołane analogicznie, gdy emisja potrzebuje dzielenia z kontrolą zera albo odczytu tablicy z kontrolą indeksu. Ich deklaracje są przy emisji w `src/codegen/llvm/context.rs`, a ciała w `crates/bork_runtime/src/lib.rs`.

Weźmy znowu program z listingu 12.1, ten z pętlą i etykietą `sum`. Budowanie tego pliku kończy się kodem 0. Uruchomiony plik wypisuje `sum` i kończy się kodem 3, bo dodaje 0, 1 i 2. Po drodze generator widzi kopię liczby `total` w pętli i współdzielenie napisu `label` w bloku, czyli dokładnie to, co wydruk regionów pokazał w rozdziale 12, i tłumaczy to na odczyt liczby oraz na wypisanie dwóch pól deskryptora. Nie ma tu `None`, nie ma zwrotu innego niż `i32` i nie ma przekazania napisu do funkcji napisanej obok `println`, więc kontrola i emisja mają ścieżkę, którą umieją dojść do końca.

WSKAZÓWKA. Gdy `bork` plik `bork` milczy, a `bork build` wypisuje fazę `codegen`, nie szukaj błędu typu. Szukaj konstrukcji, której emisja jeszcze nie tłumaczy, albo braku `main` w kształcie, którego generator oczekuje. Poprawka typu nic tu nie zmienia, dopóki kształt zostaje ten sam.

19.3 Deskryptory napisów i wyniki historyczne

Starszy wynik z dodatku WYNIKI.md pokazuje awarię przy przekazaniu napisu do funkcji użytkownika. Obecny `coerce_value_to_ty` przekazuje deskryptor `{ ptr, i64 }` bez rzutowania go na liczbę. Ponownie zbudowałem poniższy listing z codegen: sprawdzenie i budowanie zakończyły się kodem 0, program wypisał `hello` i zakończył się kodem 0. Wiersz z kodem 101 w tabeli pozostaje wynikiem historycznym.

Listing 17.2. Napis przekazany do funkcji użytkownika — przebieg historyczny

```
fun f(var a: String) {
    println(a)
}

fun main() {
    f("hello")
}
```

Wynik zapisany dla `26-pass-string.bork` pochodzi ze starszej wersji kompilatora: wtedy proces kończył się kodem 101, bo emisja traktowała deskryptor jak liczbę. Bieżący listing został uruchomiony ponownie i przechodzi; odróżniaj jego rezultat od starego wiersza tabeli.

OSTRZEŻENIE. Nie przenoś wyniku historycznej paniki na obecny kod. Jeśli podobna awaria wróci, szukaj w emisji wywołania i w konwersji argumentu; sama zgodność `frontend::check` nie dowodzi, że codegen działa.

Parametr `&T` ma w emiterze reprezentację tej samej wartości deskryptorowej co jego typ wewnętrzny — nie tworzy dodatkowego wskaźnika do deskryptora. Przykłady `&[T; N]` z odczytem i zapisem elementów są częścią korpusu `programs/build/borrow/`. Nie ma osobnej instrukcji LLVM odpowiadającej operatorowi `&`; to adnotacja semantyczna i jawny kontrakt parametru.

Na końcu zostaje mapa do kodu. Kontrola kształtów jest w `gate` w `src/codegen/gate.rs`. Złożenie modułu LLVM, łącznie z wymaganiem `main`, jest w `emit_module` w `src/codegen/llvm/mod.rs`. `Nullable` loweruje `src/codegen/llvm/nullable.rs`, a deskryptor `{ ptr, i64 }` powstaje w `src/codegen/llvm/context.rs`. Konwersja argumentów jest w `src/codegen/llvm/expr.rs`.

19.4 Podsumowanie

- Plik wykonywalny powstaje tylko z czystej reprezentacji pośredniej, i tylko wtedy, gdy kontrola przed emisją nie odrzuci kształtu, którego generator jeszcze nie tłumaczy.

- Nullable String? i wartości typów prostych poza unit są obsługiwane; nullable tablice odrzuca sprawdzanie typów, a nullable unit i typy funkcji nie mają reprezentacji.
- Napis w kodzie pośrednim jest parą wskaźnika i długości, a clang łączy wynik z biblioteką wykonawczą, która daje bufor regionu, wypisywanie i sprawdzenia indeksu oraz dzielenia.
- Arytmetyka zmiennoprzecinkowa ma ścieżkę emisji; porównanie floatów przechodzi sprawdzanie typów, ale obecnie panikuje w codegenie.
- Float jako argument funkcji pozostaje nieobsługiwany. Deskryptory napisów przechodzą do funkcji użytkownika; ich dawna panika 101 jest historyczna.
- Pożyczone parametry &T są obsługiwane, a &[T; N] może czytać i zapisywać elementy bufora.

20 Rozdział 18. Komunikaty błędów, polecenia i edytor

Komunikat jest, obok wydruku regionów, jedynym śladem fazy, która odmówiła. Programista widzi jedną linię i ma z niej wyczytać, czy zepsuł tekst, typ, własność, czy kształt, którego generator jeszcze nie tłumaczy. Edytor dostaje to samo sprawdzenie, ale w innym opakowaniu, bo protokół edytora liczy wiersze i kolumny, a kompilator w środku liczy bajty. Gdy te dwa opisy się rozjadą, podkreślenie w edytorze wskaże nie ten znak, albo faza zniknie z treści i zostanie sam tekst, z którego nie wiadomo, którą warstwę poprawiać.

Ten rozdział obejmuje

- kształt linii, którą wypisuje polecenie w terminalu, i cztery fazy, które mogą w niej stanąć,
- dwa polecenia oraz flagę wydruku regionów, włącznie z tym, kiedy wydruk w ogóle powstaje,
- różnicę między pozycją bajtową a pozycją, której oczekuje edytor,
- tekst odpowiedzi po najechaniu na nazwę, z prawdziwym wynikiem tej funkcji,
- polecenie edytora, które oddaje ten sam drzewiasty wydruk co flaga w terminalu.

20.1 Jak czytać jedną linię błędu

Pytanie brzmi, co w linii komunikatu jest miejscem w pliku, co jest fazą, a co jest treścią, którą naprawdę trzeba przeczytać. Bez tego podziału łatwo poprawiać typ tam, gdzie zepsuła się składnia, albo szukać move tam, gdzie generator nie umie None. Linia z terminala ma stały układ. Zaczyna się od ścieżki pliku, potem jest

wiersz i kolumna, gdy kompilator ma zakres, następnie słowo `error`, nazwa fazy i treść. Kod wyjścia przy każdej takiej odmowie wynosi 1, także wtedy, gdy komunikatów jest kilka.

Fazy są cztery i każda odpowiada innej części tej książki. Faza `parse` jest odmową czytania tekstu z rozdziału 13. Faza `type` jest odmową sprawdzania typów z rozdziału 15. Faza `ownership` jest odmową własności z rozdziału 14 albo odmową dotyczącą czasu życia napisu z rozdziału 16, bo zwrot wyniku `concat` też jest wypisywany jako `ownership`. Faza `codegen` jest odmową budowania z rozdziału 17 i nie pojawia się przy samym sprawdzeniu, tylko przy `bork build`. Kolejność na ekranie, gdy w jednym pliku są i własność, i typ, jest taka, że `ownership` stoi wyżej, o czym mówił rozdział 12. Nie znaczy to, że własność była liczona pierwsza.

Weźmy plik z dwoma błędami naraz, ten sam, który listing 12.2 rozbierał od strony struktur wyniku. W tym rozdziale liczy się sama linia, którą widać w terminalu, a nie to, które drzewo zostało w pamięci. Oba komunikaty da się rozłożyć na te same pola, i właśnie ten rozkład jest narzędziem, a nie ozdobą wydruku. Poprawka, która zmienia tylko jedno z tych pól, zostawia drugą linię bez zmiany.

Listing 18.1. Dwie linie i dwie fazy w jednym uruchomieniu

```
fun main() {  
    var s = "hi"  
    val t = move s  
    val u = s + 1  
}
```

```
/tmp/borkch/both.bork:4:13: error: ownership: use of `s` after move from fun main  
/tmp/borkch/both.bork:4:13: error: type: arithmetic operands must have the same numeric type,  
got String and i32
```

Obie linie wskazują tę samą kolumnę, bo oba błędy dotyczą odczytu `s` w dodawaniu. Pierwsza każe zająć się tym, że nazwa już została przeniesiona. Druga każe zająć się tym, że do napisu dodano liczbę. Poprawienie tylko dodawania, na przykład zamiana na coś, co przyjmuje napis, zostawi błąd własności, i odwrotnie. Ścieżka na początku linii zależy od tego, skąd uruchomiono polecenie, i w Twoim terminalu będzie inna. Reszta linii jest własnością programu, nie katalogu.

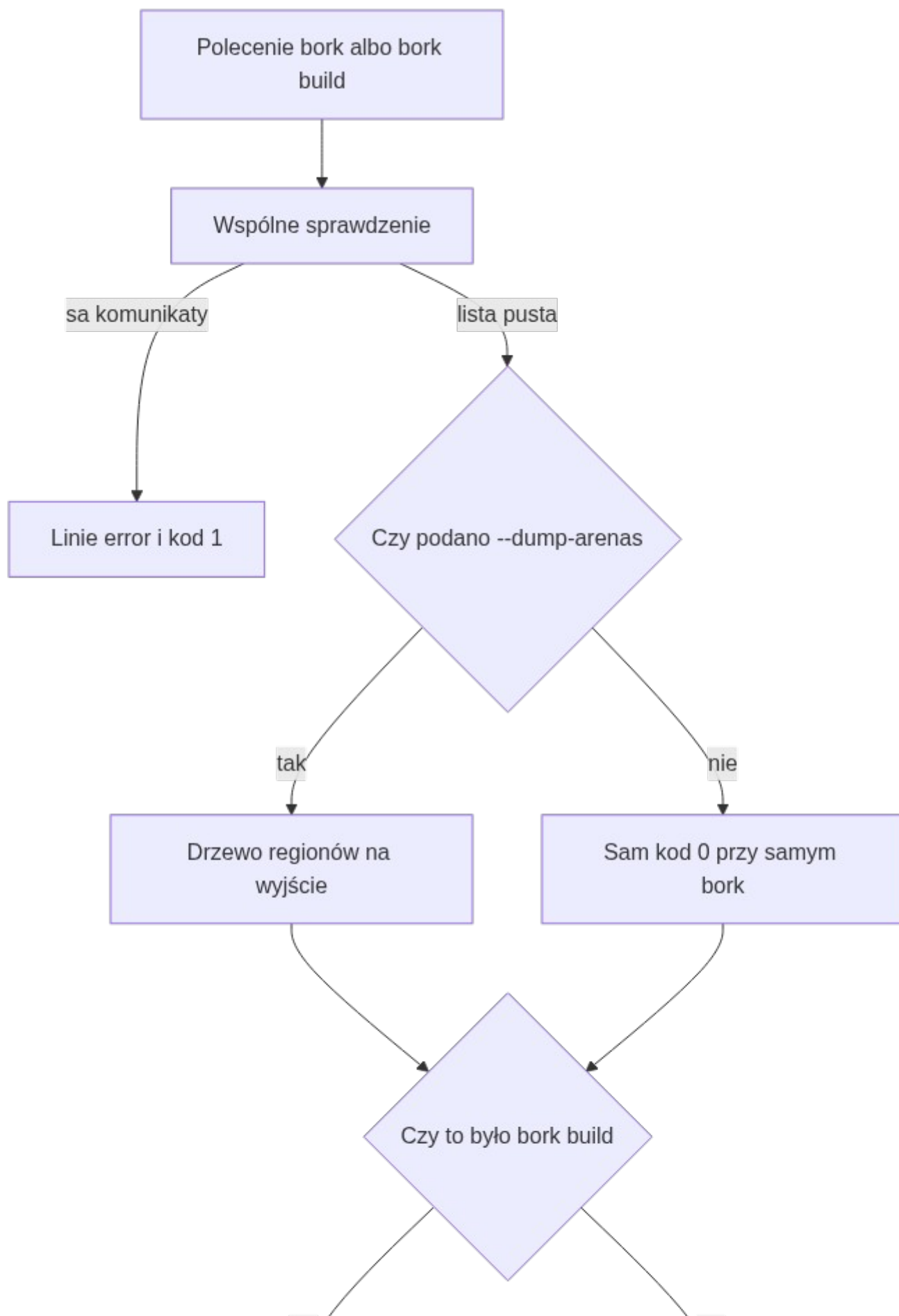
WSKAZÓWKA. Gdy w linii nie ma wiersza i kolumny, tylko ścieżka, dwukropek i od razu `error`, zakres diagnostyki był pusty. Tak bywa przy zwrocie wyniku `concat` i przy części odmów budowania, na przykład przy braku `main`. Treść nadal mówi, co jest nie tak. Nie da się wtedy skoczyć do kolumny, bo kompilator jej nie zapisał.

20.2 Co robią polecenia, a czego nie powtarzają

Pytanie brzmi, które polecenie dochodzi do której fazy, skoro początek pracy jest wspólny. Polecenie `bork plik.bork` kończy się na sprawdzeniu. Wypisuje

komunikaty faz `parse`, `ownership` i `type`, a kod 0 oznacza, że lista była pusta. Polecenie `bork build` woła to samo sprawdzenie i, tylko przy pustej liście, dochodzi do kontroli oraz emisji, więc dopiero ono może wypisać fazę `codegen`. Flaga `--dump-arenas` dopisuje drzewo regionów na standardowe wyjście i nie zmienia kodu wyjścia. Przy błędzie składni drzewa nie ma, co widać po tym, że wydruk urywa się na jednej linii `parse`. Przy błędzie typu albo własności drzewo jest, i właśnie dlatego łatwo uznać je za zgodę. Zgoda jest dopiero przy kodzie 0.

Nie ma osobnego polecenia, które uruchamiałoby same typy albo samą własność. Jedno sprawdzenie zawsze idzie tak daleko, jak da się dojść, i zbiera, co zdąży. Dlatego plik z błędem składni nie pokaże przy okazji błędu typu, nawet jeśli dalej w tekście typy też są złe. Po poprawieniu nawiasu następne uruchomienie może pokazać fazę `type` albo `ownership`, i to nie jest zmiana zdania kompilatora o starym błędzie. To jest pierwsza faza, która w ogóle dostała drzewo.



Rysunek 18.1. Które polecenie dochodzi do sprawdzania, a które do budowania

Rysunek 18.1 rozdziela milczenie od sukcesu budowania. Kod 0 przy samym `bork` znaczy, że program jest czysty, a nie że powstał plik wykonywalny. Kod 0 przy `bork build` znaczy, że plik powstał. Panika przy przekazaniu napisu z rozdziału 17 pochodzi ze starszego przebiegu i nie jest potwierdzona dla aktualnego `main`; tabela `WYNIKI.md` oznacza ją jako historyczną.

NOTA. Flaga `--dump-arenas` przy budowaniu działa tylko wtedy, gdy sprawdzenie już przeszło. Przy komunikacie z fazy `type` albo `ownership` budowanie kończy się przed emisją, ale samo polecenie `bork --dump-arenas plik.bork` drzewo jeszcze wypisze, bo wydruk nie wymaga czystego wyniku, tylko sparsowanego tekstu.

20.3 Co z tego samego sprawdzenia dostaje edytor

Pytanie brzmi, czym diagnostyka w edytorze różni się od linii w terminalu, skoro obie biorą się z tego samego przebiegu. Różnią się opakowaniem, a nie samym werdyktem sprawdzenia. Serwer języka Borka czyta standardowe wejście i pisze na standardowe wyjście, według protokołu, którego używają edytory, i przy każdej zmianie dokumentu woła to samo sprawdzenie co polecenie `bork`. Pozycje w protokole są parą wiersza i kolumny liczonych od zera, a w kompilatorze zakres jest parą bajtów w pliku. Serwer przelicza bajty na tę parę, zanim wyśle diagnostykę. Dzięki temu podkreślenie pada na znak, a nie na surowy offset, którego edytor nie umie pokazać.

Treść, którą edytor dostaje w polu komunikatu, zaczyna się od nazwy fazy, potem jest dwukropek i ta sama reszta, którą widzisz po `error:` w terminalu. Źródło diagnostyki jest ustawione na `bork`. Nie ma w tej treści ścieżki pliku, bo ścieżkę niesie sam dokument otwarty w edytorze. Stąd linia terminala i dymek w edytorze da się złożyć w jedną informację, ale nie wyglądają jak kopia jeden do jednego. Błąd składni, który w gramatyce jest dopięty do końca pliku, tak jak zła lewa strona przypisania z rozdziału 13, w edytorze też wskaże koniec, bo przeliczenie nie wymyśla zakresu, którego kompilator nie zapisał.

Osobno od diagnostyki edytor może zapytać o nazwę pod kursorem. Odpowiedź nie jest typem w sensie reprezentacji pośredniej. Jest zdaniem o regionie i o tym, jak nazwa została sklasyfikowana. Dla parametru `x` w funkcji `fun add(x: Int): Int { return x }` odpowiedź, liczona tą samą funkcją, której używa serwer, wygląda tak.

```
`x` in arena `fun add`  
Ownership: Local
```

`Local` znaczy tutaj to samo co w wydruku regionów. Nazwa jest używana w regionie, w którym została zadeklarowana. Dla napisu czytanego z bloku

zewnętrznego w tym miejscu stałoby Shared, razem z etykietą regionu deklaracji, a dla nazwy po move stałoby Moved. Ta odpowiedź nie zastępuje komunikatu błędu i nie mówi, czy program w ogóle da się zbudować. Przy programie, który się nie sparsuje, nie ma raportu, więc nie ma też na co najechać.

OSTRZEŻENIE. Odpowiedź bierze raport regionów, a nie reprezentację pośrednią, więc przy funkcji dopisanej na końcu wywołania potrafi pokazać współdzielenie tam, gdzie sprawdzanie typów widzi kopię. To jest ten sam rozjazd, który rozdziały 14 i 15 opisują przy kolejce typów deklaracji. Nie traktuj tekstu odpowiedzi jako drugiego, niezależnego sprawdzenia typów.

Na końcu zostaje mapa, włącznie z tym, czego w terminalu nie widać. Składanie linii ścieżka:wiersz:kolumna: error: faza: treść jest przy poleceniach w src/main.rs, a same fazy i treść bez ścieżki powstają w src/diag.rs. Serwer edytora jest w src/bin/bork_lsp.rs i korzysta z src/lsp.rs. Przeliczenie bajtów na pozycję protokołu oraz doklejenie nazwy fazy na początek treści robi frontend_diagnostic_to_lsp. Tekst po najechaniu na nazwę składa hover_for_analysis. Polecenie bork.dumpArenas, zarejestrowane w tym serwerze, oddaje ten sam drzewiasty wydruk co --dump-arenas, już dla dokumentu otwartego w edytorze, a nie dla ścieżki podanej w terminalu.

20.4 Podsumowanie

- Linia w terminalu składa się ze ścieżki, zwykle wiersza i kolumny, słowa error, nazwy fazy i treści, a kod wyjścia przy odmowie wynosi 1.
- Fazy parse, type, ownership i codegen wskazują odpowiednio czytanie tekstu, typy, własność albo czas życia napisu oraz budowanie, i nie wolno ich czytać jako kolejności liczenia w czasie.
- Polecenie bork kończy się na sprawdzeniu, bork build dochodzi do emisji tylko przy pustej liście, a --dump-arenas dokłada drzewo regionów, o ile tekst w ogóle się sparsuje.
- Edytor dostaje ten sam werdykt, ale pozycję przeliczoną z bajtów na wiersz i kolumnę protokołu oraz treść zaczynającą się od nazwy fazy.
- Najechanie na nazwę pokazuje region i klasyfikację własności, na przykład Local dla parametru używanego w jego własnej funkcji, i nie jest osobnym sprawdzeniem typów.

21 Rozdział 19. Jak czytać repozytorium

21.1 Ten rozdział obejmuje

- które katalogi mają znaczenie przy czytaniu kompilatora

- za co odpowiada każdy większy plik
- w jakiej kolejności rosły warstwy, które widać w kodzie
- od czego zacząć, gdy chcesz zrozumieć drogę pliku źródłowego
- które dokumenty wyglądają na opis bieżącego kodu, a opisują wcześniejszy etap

21.2 Katalogi, które mają znaczenie

W korzeniu plik `Cargo.toml` opisuje przestrzeń roboczą: skrzynkę `bork` i skrzynkę `crates/bork_runtime`. Plik `build.rs` uruchamia generowanie parsera. Przy opcji `codegen` buduje też statyczną bibliotekę wykonawczą. Plik `src/lib.rs` wystawia funkcję `parse`, eksporty modułów i próbki używane w testach. Plik `src/main.rs` jest wierszem poleceń. Plik `src/bin/bork_lsp.rs` jest binarką serwera. Gramatyka jest w `src/parser.lalrpop`. Drzewo składni jest w `src/ast.rs`. Zamiana nowych linii w nawiasach jest w `src/layout.rs`. Zakres źródłowy jest w `src/span.rs`. Komunikat jest w `src/diag.rs`. Sklejenie faz jest w `src/frontend.rs`.

Sprawdzanie typów leży w `src/typeck/`, reprezentacja pośrednia w `src/hir/`, a analiza własności w `src/sema/`. Analiza ucieczki jest w `src/escape.rs`, wyniesienie alokacji w `src/hoist.rs`, a wspólne przejście po drzewie regionów w `src/region_walk.rs`. Wydruk drzewa jest w `src/dump.rs`, funkcje wbudowane w `src/builtins.rs`, a model bufora 4096 bajtów, którego analiza własności nie używa, w `src/arena.rs`. Adapter protokołu edytora jest w `src/lsp.rs`, generowanie kodu w `src/codegen/`, a biblioteka wykonawcza w `crates/bork_runtime/`.

Testy czytania składni są w `tests/parser.rs`. Testy tablic przez samo sprawdzenie są w `tests/arrays.rs`. Testy budowania i uruchamiania binarki są w `tests/build.rs`. Klient edytora jest w `tools/bork-lsp-extension/`. Opis języka jest w `docs/language.md`. Model pamięci jest w `docs/memory-model.md`. Starsze specyfikacje i plany są w `docs/superpowers/`. Lista znanych braków jest w `TOD0.md`. Skrypt kopiujący bibliotekę LLVM obok binarki `bork` jest w `scripts/bundle-llvm.sh`.

Moduły `escape`, `region_walk`, `layout` i `builtins` są prywatne. Widać je wewnątrz skrzynki, a nie w publicznym API. Z zewnątrz wchodzisz przez `parse`, `frontend`, `sema`, `hir`, `typeck`, `dump`, `diag`, `lsp`, `codegen`, `arena`, `span` i `ast`.

21.3 Za co odpowiada każdy moduł

Moduł `ast` trzyma nietypowane drzewo po parserze. Moduł `layout` zamienia znak nowej linii na znak powrotu karetki wewnątrz nawiasów i nie zmienia długości pliku. Moduł `span` trzyma parę przesunięć bajtowych. Moduł `diag` trzyma fazę, treść i zakres. Moduł `frontend` skleja fazy w wynik sprawdzenia. Moduł `typeck`

produkuje reprezentację pośrednią i błędy typu. Moduł `hir` trzyma typowane drzewo bez numerów regionów. Moduł `sema` buduje drzewo regionów i błędy nazw. Plik `escape.rs` odrzuca użycie napisu, którego pamięć byłaby czytana po zwolnieniu bufora. Plik `hoist.rs` rozpoznaje dwie sąsiednie instrukcje i ustawia miejsce alokacji. Plik `region_walk.rs` jest jednym przejściem po reprezentacji pośredniej razem z raportem regionów. Plik `dump.rs` zamienia drzewo na tekst. Plik `builtins.rs` deklaruje `print`, `println` i `concat`. Plik `arena.rs` opisuje bufor i pulę jako model w kompilatorze. Katalog `codegen` ma kontrolę przed generowaniem kodu, emisję LLVM i konsolidację. Katalog `codegen/llvm` tłumaczy funkcje, wyrażenia, tablice i uchwyt regionów przez bibliotekę `Inkwell`. Moduł `lsp` liczy pozycje UTF-16, podpowiedź i wydruk. Skrzynka `bork_runtime` trzyma bufory i wypisywanie dla zlinkowanego programu.

21.4 Jak rosły warstwy widoczne w kodzie

Kolejność na gałęzi `main`, skrócona do rzeczy, które widać w źródłach, jest następująca. Najpierw był parser i drzewo składni, z próbką funkcji dopisanej na końcu wywołania. Serwer edytora najpierw wołał samo czytanie składni. Dziś woła pełne sprawdzenie. Potem doszła składnia wartości pustych, napisów oraz słów `Some` i `None`. Potem analiza regionów, wydruk drzewa i podpowiedź własności. Potem wyrażeniowe `move` oraz `val` i `var` na parametrach. Potem typowana reprezentacja pośrednia i funkcja `frontend::check` z fazą w komunikacie. Potem LLVM, polecenie `bork build`, biblioteka wykonawcza i kontrola przed generowaniem kodu. Potem miejsce przeznaczenia alokacji, wyniesienie, słowo `promote`, funkcja `concat` i analiza ucieczki. Potem tablice o długości w typie, wycinki, pętle `while` z `break` i `continue`, koniunkcja i alternatywa oraz wspólne przejście po drzewie regionów. Na końcu przypięcie LLVM 23 i skrypt pakowania biblioteki.

Specyfikacje w `docs/superpowers/specs/` opisują wczesne kroki i projekt serwera. Nie opisują tablic ani pętli `while`. Plik `TODO.md` jest świeższy i mówi wprost, żeby plany w `docs/superpowers/plans/` przenieść albo usunąć, gdy zostaną wchłonięte. Czytaj je jako historię decyzji. Kontraktem jest kod. Dodatek C wymienia miejsca, w których dokument i kod mówią co innego.

21.5 Od czego zacząć czytanie

Gdy celem jest zrozumienie, co dzieje się z plikiem o rozszerzeniu `.bork`, idź w tej kolejności.

Najpierw przeczytaj `docs/language.md` i `docs/memory-model.md`, a potem tę książkę w miejscach, gdzie dokument rozmija się z kompilatorem. Najważniejszy przykład to `concat` na dwóch zmiennych napisowych. Potem otwórz `src/lib.rs` i zobacz funkcję `parse` oraz dwie próbki. Próbką pierwszej wersji sprawdza, że

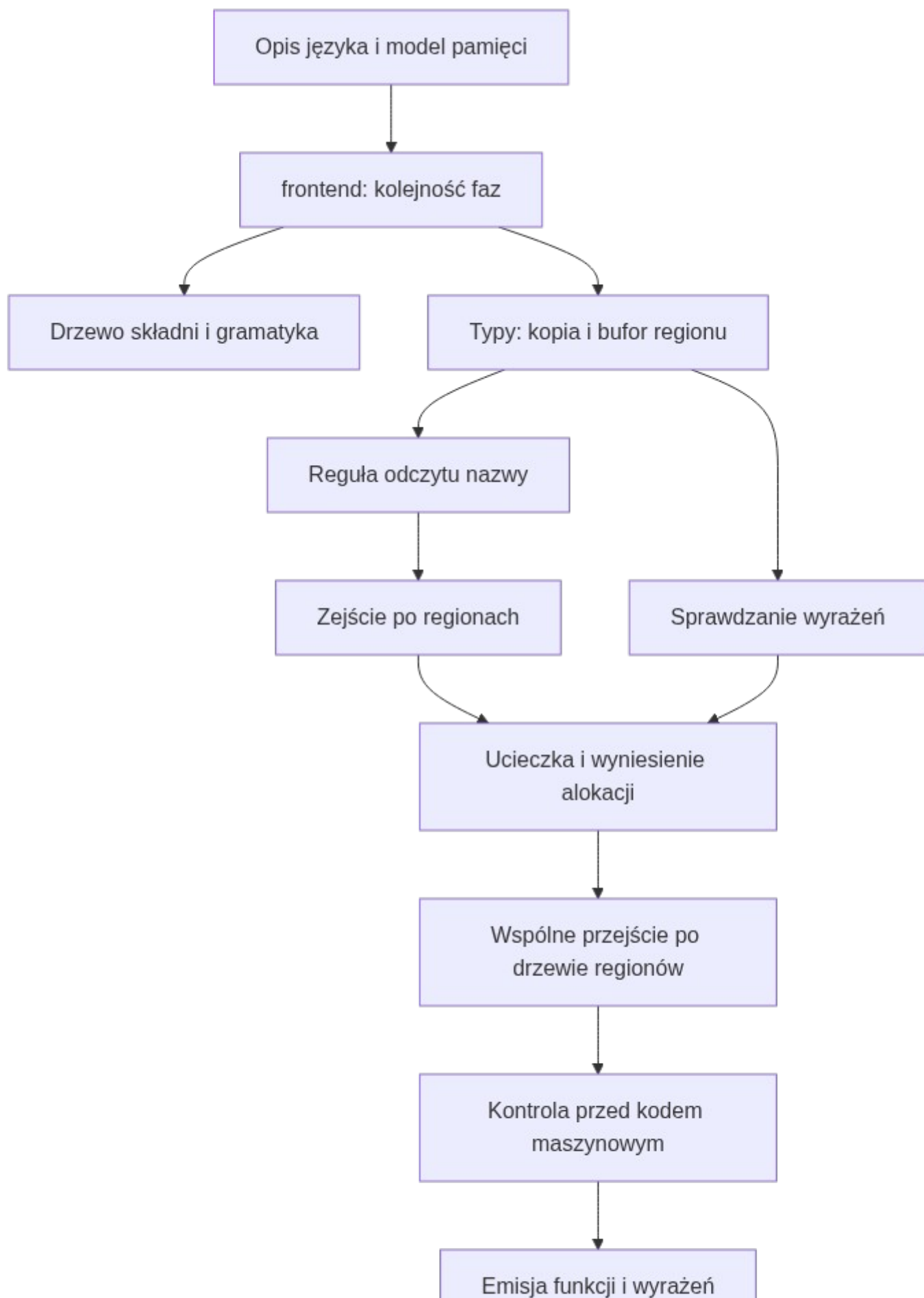
parser i analiza własności dają radę. Nie jest testem generowania kodu. Potem przeczytaj `src/frontend.rs`. Cała kolejność faz mieści się tam w kilkudziesięciu liniach.

Drzewo składni w `src/ast.rs` czytaj równolegle z początkiem gramatyki, czyli z produkcjami programu, instrukcji i wyrażenia. Nie czytaj kilkuset linii gramatyki ciągiem, dopóki nie znasz wariantów drzewa. Potem przeczytaj `src/hir/ty.rs`, zwłaszcza `is_copy` i `uses_arena_storage`. Do tych dwóch predykatów wraca reszta kompilatora. Potem `src/sema/policy.rs` i funkcję `classify_use`. Zejście w `src/sema/walk.rs` czytaj najpierw wokół otwarcia zwykłego regionu, wokół przypisania i wokół warunku.

Wyrażenia w `src/typeck/expr/control.rs` i `src/typeck/expr/call.rs` pokazują miejsca, w których składnia najbardziej przypomina Kotliną. Funkcja `place` w `src/escape.rs` i funkcja `try_hoist` w `src/hoist.rs` są krótkie. Przejście w `src/region_walk.rs` zacznij od typu, który odwiedza węzły, i od oznaczenia pobrania bufora, a nie od makr kursora. Makra `arena_cursor` są mechaniką pożyczania w Rustcie. Plik `TODO.md` sam proponuje je kiedyś zastąpić typem kursora.

Kontrolę przed generowaniem kodu czytaj w `src/codegen/gate.rs`, potem wejście budowania w `src/codegen/mod.rs`, potem `emit_module` w `src/codegen/llvm/mod.rs`, potem emisję funkcji i wyrażeń. Emisję tablic zostaw na koniec, gdy wiesz, jak wygląda deskryptor. Biblioteka wykonawcza w `crates/bork_runtime/src/lib.rs` ma około stu pięćdziesięciu linii i jest całym kodem wykonawczym. Testy w `tests/build.rs` mówią, co naprawdę zwraca `proces`.

Gdy celem jest tylko język, zatrzymaj się przed generowaniem kodu i czytaj testy analizy własności oraz sprawdzania typów.



Rysunek 19.1. Kolejność czytania źródeł kompilatora

Rysunek 19.1 zbiera tę kolejność w jednym miejscu, od dokumentów języka do biblioteki wykonawczej. Nie jest to spis każdego pliku w repozytorium, tylko ścieżka, którą da się przejść bez skakania między katalogami. Strzałka znaczy „przeczytaj to dopiero, gdy poprzedni węzeł jest już zrozumiały”, a nie zależność kompilacji. Pliki wymienione wyżej w tym podrozdziale stoją na tej ścieżce w tej samej kolejności.

21.6 Pułapki przy nawigacji

Plik `src/arena.rs` i węzeł `ArenaNode` nie są tym samym obiektem. Pierwszy ma przesunięcie i pojemność 4096 bajtów. Drugi ma dzieci i wiązania w drzewie regionów.

Funkcja `sema::analyzer` i funkcja `frontend::check` nie są wymienne. Pierwsza liczy sprawdzanie typów wewnętrznie jeszcze raz. Druga podaje gotowy wektor typów deklaracji.

Rodzaj użycia w reprezentacji pośredniej i rodzaj własności w raporcie mają podobne nazwy, między innymi kopia, współdzielenie i przeniesienie. Nie są tym samym typem w Ruście.

Opcja `codegen` wyłącza część plików z kompilacji, gdy jest wyłączona. Ostrzeżenia o martwym kodzie w przejściu po drzewie regionów bez tej opcji są oczekiwane, bo przejście jest wtedy używane przez oznaczenie regionów, a część metod tylko przez emisję LLVM.

Plany w `docs/superpowers/plans/` potrafią opisywać LLVM 18. Plik `Cargo.toml` przypina Inkwell z opcją `llvm23-1-force-dynamic`. Wierz manifestowi.

21.7 Podsumowanie

- Kolejność faz mieści się w `src/frontend.rs`. Reszta katalogu `src` to poszczególne fazy.
- Regułę odczytu nazwy czytaj przed zejściem po regionach, a predykaty typu przed jednym i przed drugim.
- Specyfikacje w `docs/superpowers` są historią. Plik `TOD0.md` i kod są stanem.
- Plik `src/arena.rs` nie stoi na ścieżce sprawdzenia.
- Testy w `tests/build.rs` definiują to, co robi binarka. Komentarze przy emisji LLVM tego nie zastępują.

22 Rozdział 20. Jeden program od źródła do uruchomienia

22.1 Ten rozdział obejmuje

- krótki program, który został zbudowany i uruchomiony
- co widzi czytanie składni
- co dopisuje sprawdzanie typów
- jakie drzewo regionów wypisuje analiza własności
- dlaczego wyniesienie alokacji i analiza ucieczki w tym programie milczą
- które funkcje wygenerowanego kodu i biblioteki wykonawczej naprawdę pracują

Program jest krótki celowo. Ma funkcję, pętlę, wywołanie, stały napis i blok, który ten napis tylko czyta. Polecenie `bork build` tłumaczy go na plik wykonywalny. Nie ma tu nullable ani porównania floatów; osobne przykłady nullable są w `programs/build/conditionals/`.

22.2 Program, który zwraca trzy i wypisuje sumę

Listing 20.1. Uruchomiony. Standardowe wyjście to `sum` i nowa linia. Kod wyjścia to 3, bo suma `0 + 1 + 2` wynosi 3.

```
fun add(a: i32, b: i32): i32 {  
    return a + b  
}  
  
fun main(): i32 {  
    var total = 0  
    for (i in 0..3) {  
        total = add(total, i)  
    }  
    val label = "sum"  
    {  
        println(label)  
    }  
    return total  
}
```

Zakres `0..3` jest otwarty z prawej strony, więc są trzy iteracje, a nie cztery, i indeks przyjmuje wartości 0, 1 i 2.

22.3 Czytanie składni widzi dwie funkcje

W pliku nie ma nawiasu, który łamałby linię, więc funkcja normalizująca nowe linie zwraca brak zmiany i parser dostaje oryginał. Parser buduje dwie funkcje.

Dla `add` drzewo składni ma dwa parametry. Nie napisano przy nich `val`, a goła forma parametru jest stała, typ obu to `i32`, a ciało ma jeden powrót i dodawanie.

Dla `main` jest deklaracja zmiennej `total` bez adnotacji typu, pętla `for` z zakresem, deklaracja stałej `label`, blok z wywołaniem `println` oraz powrót.

Gdyby w pliku był średnik, czytanie składni zwróciłoby nierozpoznany token. Sprawdzenie skończyłoby się jednym komunikatem fazy `parse`, bez drzewa regionów. Średnika nie ma.

Wejście jest w funkcji `bork::parse` w `src/lib.rs`. Reguły gramatyki to produkcje funkcji, instrukcji, zakresu i atomu.

22.4 Sprawdzanie typów widzi liczby i napis

Sprawdzanie typów widzi dwie sygnatury: funkcja `add` bierze dwie liczby `i32` i zwraca `i32`, a funkcja `main` zwraca `i32`. Nazwa `println` jest wbudowana i nie koliduje.

Zmienna `total` nie ma adnotacji. Inicjalizator `0` bez oczekiwanego typu jest literałem całkowitym, więc dostaje `i32`. Ten typ wpada do wektora typów deklaracji. Stała `label` ma typ `String`. Indeks `i` w pętli dostaje `i32`, bo granice zakresu są typu `i32`. Warunek pętli nie jest osobnym wyrażeniem logicznym. Zakres jest osobnym typem, który istnieje dopiero po sprawdzeniu typów.

Wywołanie `add(total, i)` ma nazwę jako rzecz wywoływana i dwa argumenty typu `i32`. Odczyt `total` w ciele pętli dostaje rodzaj użycia oznaczający kopię, bo region jest inny, a typ jest kopiowalny. Wywołanie `println(label)` jest funkcją wbudowaną. Argument jest napisem. Odczyt `label` dostaje rodzaj użycia oznaczający współdzielenie.

Powrót `total` zgadza się z `i32`. Ścieżka funkcji zawsze wraca. Reprezentacja pośrednia funkcji `add` to jeden powrót z dodawaniem.

Pliki tego kroku to `src/typeck/mod.rs`, `src/typeck/stmt.rs`, `src/typeck/expr/call.rs` i `src/typeck/expr/binary.rs`.

22.5 Analiza własności widzi kopię w pętli i współdzielenie w bloku

Analiza dostaje kolejkę typów deklaracji: najpierw `total`, potem `label`. Pamięta, że oba parametry `add` są stałe.

Korzeń `fun add` wiąże `a` i `b` jako nazwy lokalne. Wydruk nie pokazuje osobnych obserwacji kopiowania dla parametrów użytych w tym samym regionie. Reguła odczytu zwraca użycie lokalne, a wydruk pomija lokalne obserwacje przy

deklaracji.

Korzeń fun main wygląda tak. Ten tekst pochodzi z uruchomienia ze znacznikiem `--dump-arenas`, a nie z rekonstrukcji na papierze.

```
Arenas
├─ fun add
│   ├── a [Local]
│   └── b [Local]
└─ fun main
    ├── total [Local]
    ├── label [Local]
    ├── ForLoop (i)
    │   ├── i [Local]
    │   └── total [Copy]
    └─ Block
        └─ label [Shared ← fun main]
```

Węzeł pętli powstał przy zejściu w instrukcję `for`. Indeks `i` jest parametrem tego regionu. Nazwa `total` w ciele przechodzi regułę odczytu: inny region i typ kopiowalny, więc obserwacja to kopia. Przypisanie `total = ...` idzie do zmiennej z regionu zewnętrznego. Cel jest zmienną, a numer jego regionu jest mniejszy, więc lewa strona nie jest błędem użycia zmiennej rodzica. Prawa strona to `i32`, więc `i` tak jest kopią.

Blok po pętli otwiera zwykły region o etykiecie `Block`. Stała `label` nie jest kopiowalna i żyje w innym regionie, więc obserwacja to współdzielenie z etykietą `fun main`.

Na czas pętli zbiór zakazu przeniesienia zawiera `total`. Nikt nie woła `move`, więc zakaz milczy. Zdjęcie opakowań nic nie skleja, bo żaden blok nie jest gołym opakowaniem drugiego bloku.

Pliki tego kroku to `src/sema/analize.rs`, `src/sema/walk.rs`, `src/sema/policy.rs`, `src/sema/region.rs` i `src/dump.rs`.

22.6 Oznaczenie buforów, wyniesienie i ucieczka nie mają tu pracy poza korzeniem funkcji

Komunikatów nie ma, więc oznaczanie schodzi wspólnym przejściem `region_walk`, a korzeń każdej funkcji dostaje flagę pobrania bufora.

Ciało pętli to przypisanie liczby `i32` i wywołanie `add`. Ani jedno, ani drugie nie alokuje napisu w miejscu przeznaczenia, więc predykat `block_may_allocate_sink` dla ciała złożonego z samej arytmetyki jest fałszywy. Węzeł pętli zostaje w raporcie, ale nie prosi o `bork_arena_push`: w wydruku drzewa regionów ten blok nadal jest osobnym węzłem, natomiast w czasie działania pętla nie dostaje własnego bufora.

Blok z `println(label)` nie tworzy nowego napisu. Czyta wartość współdzieloną. Samo wywołanie `println` nie jest ani `concat`, ani literałem napisu. Literał `"sum"` stoi przy deklaracji `label` w ciele funkcji, a nie w bloku wewnętrznym. Blok wewnętrzny też nie musi pobierać własnego bufora.

Wyniesienie alokacji nie znajduje pary: deklaracja zmiennej wewnętrznej i zaraz potem przeniesienie do zmiennej zewnętrznej. Pole miejsca alokacji zostaje puste.

Analiza ucieczki widzi powrót liczby `total`, a nie typu trzymanego w buforze. Stała `label` nie wraca z regionu wewnętrznego. Funkcja licząca głębokość nic nie zgłasza.

Reprezentacja pośrednia zostaje w wyniku sprawdzenia.

22.7 Kontrola, moduł i wygenerowane wywołania

Kontrola przed generowaniem kodu nie znajduje funkcji dopisanej na końcu wywołania ani obcego pola. Nullable konstrukcje `None`, `Some`, `?:` i `!!` przechodzą bramkę i są emitowane dla obsługiwanych typów; nie należą do tego śladu.

Emisja modułu deklaruje `main` jako funkcję zwracającą `i32` bez parametrów oraz `bork.add` jako funkcję wewnętrzną. Potem emituje ciała.

Dla `main` otwiera blok wejścia, rezerwuje slot na `total` i slot na deskryptor `label`. Region funkcji pobiera bufor, bo korzeń zawsze ma flagę pobrania. Literał `"sum"` jest stałą globalną. Deskryptor ze wskaźnikiem i długością 3 ładuje w slot. Bufor funkcji w tym programie może zostać nietknięty przez alokację, bo znaki są w stałej globalnej. Pobranie bufora i tak jest.

Pętla trzyma indeks, porównuje go z 3, woła `bork.add` i przypisuje wynik. Miejsce powrotu na początek obiegu nie musi czyścić bufora pętli, jeśli wejście w pętlę w ogóle nie położyło uchwytu. Przejście `region_walk` i tak woła punkt zaczepienia tego miejsca, ale emiter na braku uchwytu tego regionu nie czyści cudzego bufora.

Wypisanie schodzi do `bork_println_str`. Po bloku, jeśli nie było pobrania, nie ma zwrotu, powrót ładuje `total` i zwraca, a zdjęcie uchwytów zdejmuję uchwyt funkcji.

Funkcja `add` ładuje dwa parametry, dodaje je instrukcją całkowitą i wraca. Napisów w niej nie ma.

22.8 Plik obiektowy, konsolidacja i proces

Zapis obiektu bierze natywny cel. Na maszynie, na której książka była sprawdzana, był to Linux x86-64. Ustawia trójkę docelową i układ danych, potem

pisze plik obiektowy. Konsolidacja woła clang z archiwum biblioteki wykonawczej.

Proces zwraca 3. Wypisanie pisze trzy bajty słowa `sum` i nową linię, potem opróżnia bufor wyjścia. Pula buforów w bibliotece wykonawczej żyje w statycznym muteksie. Po zwrocie bufor funkcji wraca na listę. Proces się kończy, więc pula znika razem z pamięcią procesu.

22.9 Czego ten ślad nie pokrywa

Nie było wyniesienia alokacji, promocji, przeniesienia napisu, wycinka, pętli `while`, zwierania ani błędu ucieczki. Każde z nich ma listing w części pierwszej i test w `tests/build.rs` albo w `src/escape.rs`. Ślad miał pokazać udaną drogę przez funkcje graniczne, a nie każdy hak.

Gdy będziesz powtarzał ślad na programie, który deklaruje kawałek napisu wynikiem `concat` i w następnym wierszu przenosi go do zmiennej zewnętrznej, zatrzymaj się dłużej przy wyniesieniu. Pole miejsca alokacji zostanie ustawione, a emisja deklaracji ustawi miejsce przeznaczenia na dom zmiennej zewnętrznej, zanim wyemituje `concat`. Reszta kroków jest ta sama.

22.10 Podsumowanie

- Śledzony program zwraca 3 i drukuje `sum`. Sprawdzenie i bork `build` są zgodne.
- Parser widzi dwie funkcje i zakres. Sprawdzanie typów widzi `i32` i `String`.
- Analiza własności oznacza `total` jako kopię w pętli i `label` jako współdzielenie w bloku.
- Pętla bez alokacji nie dostaje własnego bufora, choć ma węzeł w wydruku.
- Wyniesienie alokacji i analiza ucieczki nie mają tu nic do zrobienia. To też jest wynik.
- Wygenerowany kod woła `bork.add` i `bork.println_str`. Biblioteka wykonawcza dostaje deskryptor literału ze stałej globalnej.

23 Rozdział 21. Testy, szukanie przyczyny błędu i nowa konstrukcja języka

23.1 Ten rozdział obejmuje

- które polecenie uruchamia który zestaw testów

- gdzie szukać, gdy binarka się wywraca, a sprawdzenie milczy
- w jakiej kolejności warstw doszła do języka pętla while
- które braki są już zapisane w TODO.md
- czego nie robić w pierwszej poprawce

23.2 Dwa polecenia testów, bo generowanie kodu jest osobną opcją

Z pliku README i z `.github/workflows/ci.yml` wynikają dwa polecenia. Pierwsze to `cargo test --workspace`. Drugie to `cargo test --workspace --features codegen`.

Pierwsze polecenie nie składa LLVM. Łapie parser, analizę własności, sprawdzanie typów, analizę ucieczki przez `frontend::check`, serwer edytora oraz testy jednostkowe puli w bibliotece wykonawczej. Kontrolę przed generowaniem kodu łapie tylko o tyle, o ile test tej kontroli jest za warunkiem opcji `codegen`. Zadanie ciągłej integracji o nazwie `codegen` jest osobne. Instaluje LLVM 23 i `clang`, puszcza przestrzeń roboczą z tą opcją i jeszcze sprawdza spakowaną bibliotekę LLVM.

Testy w `tests/parser.rs` utwierdzają tokeny, nowe linie, słowo `move`, kolejność operatorów i zakresy błędów. Testy w `src/sema/tests` utwierdzają własność nazw. Testy w `src/typeck/tests` utwierdzają typy, często przez pełne sprawdzenie. Plik `tests/arrays.rs` sprawdza tablice bez uruchamiania binarki. Plik `tests/build.rs` buduje program i sprawdza kod wyjścia albo standardowe wyjście. Plik `src/codegen/gate.rs` trzyma teksty o braku wsparcia. Plik `src/lsp.rs` sprawdza pozycje, odpowiedź i wydruk. Skrzynka `bork_runtime` sprawdza przesunięcie wskaźnika, wyrównanie i przepełnienie. Test `tools/bork-lsp-extension/test/server.test.js` pilnuje ścieżki binarki wobec `cargo run`.

Plik `tests/build.rs` jest kompilowany tylko z opcją `codegen`. Bez niej znika z zestawu. Nie dziw się, że samo `cargo test` nie widzi testu sumy pętli `for`.

Testy budowania piszą źródło do katalogu tymczasowego `cargo`, wołają binarkę `bork` z podpoleceniem `build` i ścieżką `-o`, a potem uruchamiają powstały program. Gdy dodajesz zachowanie widoczne w czasie działania, dopisuj tu oczekiwany kod albo tekst wyjścia. Test jednostkowy samej emisji tego nie zastępuje. Listingi w tej książce opierają się na tym samym kontrakcie.

23.3 Gdzie szukać, gdy wynik nie zgadza się z oczekiwaniem

Gdy drzewo regionów nie ma węzła, którego oczekujesz, błąd jest w zejściu analizy własności, w `src/sema/walk.rs`, a nie w LLVM. Gdy drzewo jest, a `bork build` kończy się awarią kompilatora, błąd jest w emisji albo w funkcji `coerce_value_to_ty`. Wydruk drzewa dostaniesz poleceniem `cargo run --bin bork -- --dump-arenas plik.bork`.

Prefiks fazy mówi, którego katalogu nie czytać w pierwszej kolejności. Faza `parse` prowadzi do gramatyki. Faza `type` prowadzi do `src/typeck`. Faza `ownership` bez słów o użyciu po przeniesieniu i bez słów o braku kopiowania często pochodzi z `src/escape.rs`, a nie z `src/sema/policy.rs`. Słowa o regionie wewnętrznym, o `concat` i o wartości przeniesionej albo promowanej są z analizy ucieczki. Faza `codegen` prowadzi do kontroli albo do emisji.

Komunikat o wewnętrznej niezgodności harmonogramu regionów znaczy, że wspólne przejście `region_walk` nie dostało dziecka węzła albo dostało je w złym miejscu. Porównaj etykietę w analizie własności z miejscem w typie `RegionSite`. Najczęstsza przyczyna przy nowej konstrukcji jest taka, że analiza własności otwiera region, a kod emisji o nim nie wie, albo odwrotnie.

Awaria w `into_int_value` znaczy, że wartość LLVM nie jest liczbą całkowitą. Historyczna panika na deskrytorze napisu została usunięta przez gałąź `uses_arena_storage()` w `coerce_value_to_ty`; ponowny build i wykonanie listingu z napisem przeszły. Float jako argument funkcji jest jawnie odrzucany. Porównanie floatów ma osobną usterkę: `combine_binary_values` wybiera ścieżkę float według typu wyniku, a porównanie zwraca `bool`, więc operand trafia do `value_as_int` i bieżące odtworzenie panikuje. Nie zakrywaj takich przypadków łapaniem paniki.

Przerwanie procesu użytkownika bez komunikatu kompilatora to zwykle indeks albo dzielenie. Uruchom binarkę pod debuggerem i zobacz, czy stanęła w `abort`. Przepełnienie bufora regionu daje panikę Rusta z tekstem `arena overflow`, bo biblioteka wykonawcza jest pisana w Rustie i ta panika nie jest łapana przez program w Borku.

Weryfikacja modułu LLVM po emisji wywraca budowanie, gdy reprezentacja pośrednia LLVM jest zepsuta, na przykład przy złym typie skalenia gałęzi albo przy braku powrotu. To lepsze niż cichy zły kod. Gdy weryfikacja pada, błędu nie szukaj w `clang`.

WSKAZÓWKA. Binarka `bork` bez opcji `codegen` jest szybka, gdy poprawiasz komunikaty sprawdzenia. Włączaj tę opcję dopiero wtedy, gdy sprawdzenie jest czyste i chcesz zobaczyć kontrolę przed generowaniem kodu albo sam proces.

23.4 Jak do języka doszła pętla `while`

Poniżej jest kolejność, którą widać po fakcie na `while`, `break` i `continue`. Nie jest to przepis skopiowany z planu. Jest to kolejność warstw, które musiały ruszyć, bo każda z nich w kodzie o pętli `while` wie.

Najpierw gramatyka. Dochodzi terminal i produkcja instrukcji. Dla `while` warunek

jest w nawiasach, a ciało jest blokiem. `break` i `continue` są instrukcjami z zakresem źródłowym. Test w `tests/parser.rs` sprawdza, że nowa linia działa i że słowo nie jest identyfikatorem w złym miejscu.

Potem drzewo składni dostaje warianty instrukcji `while`, `break` i `continue`. Na tym etapie nie ma typów.

Potem sprawdzanie typów. Warunek oczekuje `bool`. Głębokość pętli rośnie tak samo jak przy `for`. `break` poza pętlą jest błędem typu. Dochodzi nowy wariant instrukcji w reprezentacji pośredniej. Test trzyma dokładny tekst komunikatu.

Potem analiza własności. Region `WhileLoop` otwiera się zwykłą ścieżką. Obowiązuje ten sam zakaz przeniesienia co przy `for`. Jeśli zapomnisz zakazu, istniejący test przeniesienia nazwy zewnętrznej w pętli nie pokryje `while`, dopóki go nie skopiujesz. Warto go skopiować.

Jeśli ciało jest blokiem, nie dokładaj drugiego regionu w sprawdzaniu typów, bo analiza własności i reprezentacja pośrednia muszą mieć po jednym dziecku. Inaczej przejście po drzewie się nie zepnie.

Nowa instrukcja musi być odwiedzona w analizie ucieczki i w wyniesieniu alokacji. Inaczej deklaracja wewnątrz `while` nie będzie widziana. Dla tej pętli wystarczy zejść w ciało tak, jak przy `for`.

We wspólnym przejściu `region_walk` dochodzi miejsce regionu i punkt pętli `while`, więc oznaczenie pobrania bufora zaczyna widzieć alokacje w ciele. Bez tego wygenerowany kod i raport przestaną pasować do siebie w chwili, gdy ciało alokuje napis.

Pętla `while` nie potrzebuje odmowy w kontroli przed generowaniem kodu, bo emisja ją umie. Nowa konstrukcja, której emisja nie umie, powinna dostać odmowę w `src/codegen/gate.rs`, żeby użytkownik dostał komunikat zamiast awarii kompilatora; trailing closure jest takim przykładem. Przekazanie napisu do funkcji użytkownika nie jest już tym przypadkiem: deskryptor jest przekazywany bez rzutowania na liczbę.

Emisja dokłada bloki podstawowe: nagłówek, ciało, zatrząsk, czyszczenie bufora, jeśli region go pobrał, oraz `break` jako skok do bloku wyjścia. Potem test w `tests/build.rs` ma konkretny kod wyjścia. Dla `while` z `break` przy wartości 3 jest to 3.

Na końcu dokument. `docs/language.md`, a jeśli ruszasz pamięć, także `docs/memory-model.md`. Plik `TODO.md` prosi, żeby dokument nadążał za kontrolą przed generowaniem kodu. Ta książka opisuje jedną rewizję i nie zaktualizuje się sama. Przy zmianie semantyki aktualizuj co najmniej `docs/language.md`.

Konstrukcja, która jest tylko skrótem składniowym, może skończyć się w parserze zamianą na istniejące drzewo. W tym repozytorium prawie nic się tak nie dzieje. Wyrażeniowe move i regionalne move są osobnymi węzłami. Nowy skrót lepiej zostawić osobnym węzłem, jeśli ma własny komunikat.

23.5 Braki, które są już zapisane

Plik `TODO.md` wymienia między innymi rozcięcie kodu generowania, żeby emiter funkcji nie był jednocześnie obiektem przechodzącym po drzewie regionów, oraz kursor regionów zamiast makra, gdy makro dalej urośnie. Wymienia też test, że harmonogram pętli to jedno wejście, wiele czyszczeń i jedno wyjście, synchronizację kontroli przed generowaniem kodu ze sprawdzeniem oraz to, żeby błędy przejścia `region_walk` zawsze były zwykłym komunikatem, bez gubienia tekstu użytkownika w opakowaniu o niezgodności harmonogramu. Są tam jeszcze prośby o więcej testów par wejście-wyjście na zagnieżdżonym warunku i na miejscach przeznaczenia napisów oraz o to, żeby nie commitować luźnych plików `smoke.bork` w korzeniu repozytorium.

Komunikat o niezgodności harmonogramu wokół liczb zmiennoprzecinkowych jest przykładem dwóch z tych punktów. Informacja, którą powinien zobaczyć użytkownik, utonęła w opakowaniu: dodawanie liczb zmiennoprzecinkowych nie jest obsługiwane. Kontrola przed generowaniem kodu w ogóle nie powiedziała, że liczba zmiennoprzecinkowa nie jest tłumaczona.

23.6 Czego nie robić w pierwszej poprawce

Nie zaczynaj od LLVM. Dopisz test parsera albo sprawdzania typów, który na czerwono nazywa zachowanie, i doprowadź `frontend::check` do tego tekstu. Odmowę w kontroli przed generowaniem kodu dodaj w tym samym patchu, jeśli emisji nie będzie. Brak takiej odmowy przy nowej składni zamienia przyszłą awarię kompilatora w coś, co użytkownik zobaczy dopiero jako panikę.

Nie zmieniaj `peel_blocks` tylko w jednym z dwóch miejsc.

Nie naprawiaj problemów z argumentami przez odrzucenie wszystkich napisów w kontroli, jeśli w `main` napisy działają. To obciążałoby legalne listingi. Najpierw ustal, czy wartość jest liczbą, floatem, czy deskryptorem, i dodaj test wykonawczy dla ścieżki `codegenu`.

23.7 Podsumowanie

- Polecenie `cargo test --workspace` nie uruchamia LLVM. Opcja `codegen` uruchamia testy w `tests/build.rs`.
- Wydruk regionów pokazuje błąd analizy własności, awaria `into_int_value`

pokazuje błąd emisji, a niezgodność harmonogramu pokazuje błąd przejścia `region_walk`.

- Nowa konstrukcja idzie warstwami: gramatyka, drzewo składni, sprawdzanie typów, analiza własności, ucieczka i wyniesienie, przejście po drzewie, kontrola, emisja, test binarki.
- Jeśli emisji nie ma, odmowa w kontroli przed generowaniem kodu jest obowiązkowa.
- Plik `TOD0.md` jest listą znanych braków. Sprawdź ją, zanim opiszesz różnicę dokumentu i kodu jako własne odkrycie.

24 Rozdział 22. Ćwiczenia

24.1 Ten rozdział obejmuje

- ćwiczenia na samo sprawdzenie programu, bez generowania kodu
- ćwiczenia na bork `build` i na kod wyjścia procesu
- ćwiczenia czytania kodu kompilatora
- odpowiedzi, które da się zestawić z komunikatem albo z plikiem

Odpowiedzi są w tym samym rozdziale, pod treścią zadań. Najpierw uruchom kompilator. Komunikaty z części pierwszej wolno potraktować jako ściągę.

24.2 Część A. Samo sprawdzenie

A1. Napisz funkcję `main`, która zwraca `i32` i nie ma ani jednego `return`. Jaki jest dokładny początek komunikatu, czyli faza i pierwsze słowa?

A2. Zadeklaruj `var s = "hi"` i `var t = s`. Popraw program najmniejszą zmianą tak, żeby sprawdzenie przeszło, a nazwy `s` nie dało się użyć niżej.

A3. Poniższy program jest poprawny i odejmuje trzy od zera. Zamień `0 - 3` na `-3` i nazwij fazę błędu.

```
fun main(): i32 {  
    val n = 0 - 3  
    return n  
}
```

A4. Program `val a: [i32; 2] = [1, 2, 3]` daje więcej niż jeden komunikat. Ile ich jest, jakiej są fazy i czego dotyczą? Nie poprawiaj programu.

A5. Napisz pętlę `for` po zakresie typu `i64`. Jakiego typu granic wymaga sprawdzanie?

A6. Weź fragment z dokumentacji języka, dopisz brakującą deklarację `out` i spraw, żeby sprawdzenie przeszło na dwa sposoby: przez stałe `val` oraz przez `move`.

```
var left = "L"
var right = "R"
{
    var piece = concat(left, right)
    out = move piece
}
```

A7. W funkcji zwracającej `String` napisz `return concat("a", "b")`. Czy komunikat ma numer linii?

24.3 Część B. Zbudowany program

Uruchom `bork build` i powstały proces.

B1. Ile wynosi kod wyjścia sumy `for (i in 0..5)`, gdy w ciele jest `total = total + i`, a `total` zaczyna się od zera?

B2. Pętla `while` podbija `i` od zera i robi `break` przy `i == 3`, zanim zwiększy licznik. Co zwraca `main`?

B3. W funkcji `main` bez zadeklarowanego typu wyniku jest samo `println("hi")`. Co jest na standardowym wyjściu i jaki jest kod wyjścia?

B4. Program zwraca `1 / 0`. Czy `bork build` kończy się błędem? Co robi proces?

B5. Program ma `val a = [10, 20, 30]` i odczyt `a[9]`. Czy sprawdzenie przechodzi?

B6. Weź listing 20.1 i zmień zakres na `0..4`. Jaki kod wyjścia przewidujesz, zanim uruchomisz program, i dlaczego?

B7. Program to `fun main(): i64 { return 1 }`. Czy pada sprawdzenie, czy dopiero budowanie, i jaka jest faza?

24.4 Część C. Kod kompilatora

C1. Wskaż plik i funkcję, które ustawiają kolejność wypisu: najpierw komunikaty analizy własności, potem komunikaty typów.

C2. Gdzie jest pojemność 4096? Podaj dwa pliki. Który wykonuje się w procesie użytkownika?

C3. Dlaczego `--dump-arenas` przy urwanym `fun oops` (nie drukuje drzewa)?

C4. Pole przechwycenia bloku `move` bywa puste albo bywa pustą listą. Który wariant zgaduje nazwy?

C5. Dlaczego reprezentacja pośrednia nie ma pola z numerem regionu? Skąd generator kodu bierze region?

C6. Która funkcja w `src/sema/policy.rs` zwraca współdzielenie?

C7. Frontend akceptuje `1.0 < 2.0`, ale generator kieruje operandy `float` do ścieżki całkowitoliczbowej i panikuje. Który typ wyniku widzi dispatcher i po jakich typach powinien rozpoznać rodzaj porównania?

C8. Masz nowy operator dwuargumentowy, którego LLVM jeszcze nie tłumaczy. Wymień warstwy z rozdziału 21 w kolejności i zaznacz, która jest obowiązkowa, żeby nie było awarii kompilatora.

24.5 Odpowiedzi do części A

A1. Początek komunikatu to `type: function must return a value of type i32 on all paths`. Zakres bywa pusty, więc numer linii może zniknąć z prefiksu.

A2. Zapis `var t = move s` przenosi własność. Przy gołym `s` komunikat każe użyć `move s`, żeby przenieść własność.

A3. Zapis `return -3` jest fazą `parse` i mówi o nieoczekiwanym tokenie. Zapis `0 - 3` jest odejmowaniem. Dla `main` zwracającego `i32` sprawdzenie jest zadowolone. Kod wyjścia procesu dla wartości ujemnej owija się w zakresie `i32`.

A4. Są dwie diagnostyki fazy `type`. Literał ma 3 elementy, a oczekiwano 2. Inicjalizator ma typ `[i32; 3]`, a oczekiwano `[i32; 2]`.

A5. Komunikat mówi, że granice zakresu muszą mieć typ `i32`.

A6. Na zewnątrz bloku deklarujesz `var out` z jakimś napisem początkowym, na przykład `"prefix"`. Albo `left` i `right` są stałymi `val`, albo wywołanie ma postać `concat(move left, move right)`. Wynikowy napis to `LR`, a nie `prefixLR`, bo `concat` zastępuje deskryptor, a nie dokleja znaki do starego bufora zmiennej `out`.

A7. Nie. Analiza ucieczki stawia pusty zakres. Zostaje komunikat fazy `ownership` o zwracaniu wyniku `concat`, bez numeru linii.

24.6 Odpowiedzi do części B

B1. Kod wyjścia to 10.

B2. Kod wyjścia to 3.

B3. Standardowe wyjście to hi i nowa linia. Kod to 0, bo main bez typu zwraca unit, a generowanie kodu zamienia to na i32 równe zero.

B4. Budowanie się udaje. Proces kończy się sygnałem przerwania, w powłoce kodem 134.

B5. Sprawdzenie przechodzi, ale proces jest przerywany, bo indeks nie jest częścią typu, natomiast częścią typu jest długość.

B6. Suma $0 + 1 + 2 + 3$ wynosi 6, bo zakres $0..4$ jest otwarty z prawej strony.

B7. Sprawdzenie przechodzi. Budowanie zgłasza fazę codegen i mówi, że main zwracające i64 nie jest jeszcze obsługiwane.

24.7 Odpowiedzi do części C

C1. Robi to `frontend::check` w `src/frontend.rs`. Sprawdzanie typów wykonuje się wcześniej i zwraca komunikaty. Analiza własności zwraca własne błędy. Wektor buduje się najpierw z nich, a dopiero potem dopisuje błędy typów.

C2. Stała `ARENA_CAPACITY` jest w `crates/bork_runtime/src/lib.rs` i ten plik wykonuje się w procesie użytkownika. Tę samą pojemność opisuje `src/arena.rs`, ale to tylko model w kompilatorze, którego analiza własności nie woła.

C3. Przy błędzie składni pole raportu w wyniku sprawdzenia jest puste. Wiersz poleceń drukuje drzewo tylko wtedy, gdy raport jest obecny.

C4. Brak listy zgaduje nazwy. Pusta lista to jawne `move ( )` i oznacza, że nic nie jest przenoszone.

C5. Komentarz w `src/hir/mod.rs` mówi, że regiony żyją w raporcie z analizy własności, a generator kodu bierze je we wspólnym przejściu `region_walk`, zsynchronizowanym z reprezentacją pośrednią.

C6. Współdzielenie zwraca `classify_use`, w gałęzi wiązania stałego, po sprawdzeniu, że typ nie jest kopiowalny i że region nie jest ten sam.

C7. Typ porównywanego operandu (f32 albo f64) oraz typ wyniku (bool). `emit_binary` wybiera obecnie ścieżkę float na podstawie `expr_ty`, czyli typu wyniku; dla porównania widzi bool i schodzi do ścieżki całkowitoliczbowej. Poprawka powinna rozpoznawać typ operandów i dodać test codegenu dla f32 oraz f64.

C8. Warstwy z rozdziału 21 idą w tej kolejności. Najpierw jest gramatyka i drzewo

składni, potem sprawdzanie typów, potem analiza własności, gdy operator przenosi nazwy albo je współdzieli, potem kontrola czasu życia napisu i wyniesienie alokacji, gdy operator obchodzi się z napisem albo tablicą, potem wspólne przejście reprezentacji pośredniej i raportu regionów, gdy operator otwiera region, a na końcu kontrola przed generowaniem kodu, emisja i test w `tests/build.rs`. Obowiązkowa, dopóki emisja nie ma gałęzi dla tego operatora, jest właśnie kontrola przed generowaniem kodu. Bez niej dopasowanie wpadnie albo w komunikat o braku wsparcia, albo w ścieżkę liczby i w złe rzutowanie wartości LLVM, czyli w awarię kompilatora zamiast w diagnostykę.

24.8 Podsumowanie

- Ćwiczenia A kończą się na tekście komunikatu. Ćwiczenia B kończą się na kodzie procesu.
- Ćwiczenia C wskazują pliki, a nie ogólniki.
- Zakres $0..n$ jest otwarty z prawej strony. Suma $0..5$ to 10. Suma $0..4$ to 6.
- Kontrola przed generowaniem kodu nie zastępuje rzutowania argumentu wywołania. Stąd osobna odpowiedź C7.

25 Dodatek A. Zestawienie składni

Ten dodatek zbiera składnię, którą przyjmuje czytanie programu w tej rewizji kompilatora. Gwiazdka przy wierszu znaczy, że sprawdzenie program przyjmuje, ale `bork build` nadal może go odrzucić albo ujawnić błąd kompilatora. Szczegóły są w dodatku C. Zdania wokół zestawienia mówią, jak czytać skrót. Sam skrót jest po to, żeby nie szukać reguły w całym rozdziale.

25.1 Program i parametry

Funkcja ma nazwę, listę parametrów w nawiasach i ciało w nawiasach klamrowych. Typ wyniku, jeśli jest, stoi po dwukropku przed ciałem. Brak typu wyniku znaczy `unit`.

```
fun nazwa(parametry) { instrukcje }
fun nazwa(parametry): Typ { instrukcje }
```

Parametr zapisuje się jako `nazwa: Typ`, `val nazwa: Typ` albo `var nazwa: Typ`. Goły parametr, bez `val` i bez `var`, jest stały. Instrukcje rozdziela przejście do nowego wiersza, a nie średnik. Komentarz zaczyna się od `//` i trwa do końca wiersza.

25.2 Typy

Typy proste to i8, i16, i32, i64, u8, u16, u32, u64, f32, f64, bool i unit. Aliasy Int, Long, Byte, Float i Double znaczą odpowiednio i32, i64, u8, f32 i f64. Napis to String. Tablica o znanej długości to [T; N]. Ograniczony widok na niekopiowane T zapisuje się &T. Wartość, która może być pusta, to T?. Typ funkcji to (A, B) -> R. Może też sam być pusty, na przykład ((A) -> R)?.

Kopiuwalne są tylko typy proste, które nie mogą być puste. Napis, tablica, typ funkcji, pożyczka &T i wartość pusta kopiuwalne nie są.

25.3 Instrukcje

```
val nazwa = wyrażenie
val nazwa: Typ = wyrażenie
var nazwa = wyrażenie
var nazwa: Typ = wyrażenie
nazwa = wyrażenie
nazwa[indeks] = wyrażenie
{ instrukcje }
for (nazwa in lo..hi) { instrukcje }
while (warunek) { instrukcje }
break
continue
return
return wyrażenie
move { instrukcje }
move () { instrukcje }
move (a, b) { instrukcje }
wyrażenie
```

Pętla for idzie po zakresie lo..hi typu i32 i nie wykonuje ciała dla wartości hi. Pętla while wymaga warunku typu bool. break i continue są legalne tylko wewnątrz pętli.

25.4 Wyrażenia

Najmocniej wiążą się wywołanie, indeks, wycinek, odczyt pola i przyrostkowe !!, potem negacja !, mnożenie i dzielenie, dodawanie i odejmowanie, porównania, zakres .., koniunkcja && oraz alternatywa ||. Najslabiej wiąże się ?: . Operator ? : jest prawostronnie łączny. Pozostałe operatory dwuargumentowe są lewostronnie łączne.

```
nazwa
liczba
liczba.liczba
"napis"      \n \r \t \\ \"
true false
[e, e, e]
nazwa[i]
nazwa[lo..hi]  lo i hi są literałami i32
nazwa.pole
```

```
nazwa?.pole
wyrażenie!!
wyrażenie ? : wyrażenie
Some(wyrażenie) None
move nazwa
&nazwa
promote nazwa
f(a, b)
f(a) { x -> instrukcje } *
f(a) move (s) { x -> instrukcje } *
```

Jednoargumentowego minusa nie ma. Liczbę ujemną zapisuje się odejmowaniem, na przykład $0 - 1$.

Warunek `if (warunek) { } else { }` jest wyrażeniem. Obie gałęzie są blokami. Bez `else`, gdy nikt nie oczekuje typu, wynikiem jest `unit`.

25.5 Własność nazw w skrócie

Gdy typ jest kopiowalny, wystarcza goła nazwa. Gdy stała z regionu zewnętrznego nie jest kopiowana, goła nazwa oznacza współdzielenie. Zmienną niekopiowaną można przenieść przez `move nazwa` albo pożyczyć przez `&nazwa`, jeśli kontekst oczekuje `&T`. Pożyczka nie jest przeniesieniem własności. Zapis do zmiennej zewnętrznej ma postać `outer = wyrażenie`, `outer = move inner` albo `outer = promote held`. Zwrot napisu to `return nazwa` na głębokości funkcji albo `return literału`. Zwrot wyniku `concat` oraz zwrot przeniesienia z regionu wewnętrznego są odrzucane.

25.6 Funkcje wbudowane i polecenia

`print` i `println` przyjmują `i32`, `i64` albo `String` i zwracają `unit`, mimo że ich nominalna sygnatura mówi o `i32`. `concat` bierze dwa napisy i zwraca napis.

```
bork plik.bork
bork --dump-arenas plik.bork
bork build plik.bork
bork build -o wyjście plik.bork
cargo test --workspace
cargo test --workspace --features codegen
```

Kod zero oznacza sukces, kod jeden błąd programu lub odmowę codegeny, a kod dwa złe wywołanie albo brak narzędzia. Kod 101 występuje w historycznych wynikach awarii kompilatora, a kod 134 w powłoce oznacza przerwanie procesu użytkownika przez `abort`.

Fazy komunikatów to `parse`, `ownership`, `type` i `codegen`.

26 Dodatek B. Słownik pojęć

Każde hasło jest zdaniem albo krótkim akapitem. Angielski termin z kodu podaję raz, obok polskiego, i dalej trzymam polski, chyba że w zdaniu chodzi o nazwę w źródle.

Analiza ucieczki. Przejście w pliku `src/escape.rs`, które sprawdza, w którym regionie leży pamięć napisu, i odrzuca użycie, które przeżyłoby zwolnienie bufora tego regionu. Komunikat ma fazę `ownership`. W kodzie funkcja licząca głębokość nazywa się `place`.

Analiza własności. Przejście w katalogu `src/sema`, które buduje drzewo regionów i decyduje, czy nazwę wolno skopiować, współdzielić albo trzeba przenieść. Po angielsku `semantic analysis`. Nie przydziela buforów.

Pożyczka (Borrow). Jawny, regionowy widok zapisany jako `&nazwa` i opisany typem `&T`. Nie kopiuje wartości ani nie zabiera jej właścicielowi. Działa dla ograniczonego zestawu kontekstów; nie można zwracać pożyczek ani przechowywać ich w polach.

Arena. Bufor jednego regionu, o pojemności 4096 bajtów, z przesuwającym się wskaźnikiem. W źródłach ta sama nazwa oznacza jeszcze węzeł drzewa regionów, `ArenaNode`. To nie jest ten sam obiekt. Rozdział 2 i rozdział 14 rozdzielają te znaczenia.

Drzewo składni. Nietypowany wynik parsera, w kodzie `AST`, od angielskiego `abstract syntax tree`. Definicja jest w `src/ast.rs`.

Głębokość. Liczba regionów między funkcją a miejscem, w którym leży pamięć napisu. Zero to region funkcji. Analiza ucieczki porównuje tę głębokość z głębokością odbiorcy.

Kontrola przed generowaniem kodu. Zejście po reprezentacji pośredniej w `src/codegen/gate.rs`, zanim powstanie moduł LLVM. Odrzuca konstrukcje, których emisja nie tłumaczy, komunikatem fazy `codegen`. Nie łapie każdego miejsca, w którym emisja kończy się awarią.

Kopiowanie. Przekroczenie granicy regionu przez wartość typu prostego, która nie może być pusta. Źródło zostaje żywe. W wydruku drzewa znacznik to `Copy`.

Miejsce przeznaczenia. Odbiorca wyniku, czyli cel przypisania, powrót albo wynik `concat`. Steruje tym, w której arenie powstanie bufor wyniku. W kodzie bywa nazywane `sink`.

Promocja. Zapis `promote nazwa` po prawej stronie przypisania do zmiennej z

regionu zewnętrznego. Kopiuje treść napisu do bufora tej zmiennej i unieważnia źródło.

Przeniesienie. Zapis `move`, po którym źródłowa nazwa jest martwa. Bajty w starej arenie zostają niedostępne aż do wyczyszczenia bufora. W wydruku drzewa znacznik to `Moved`.

Pula buforów. Lista wolnych aren w bibliotece wykonawczej. Zwrot bufora oddaje go na listę. Wyczyszczenie wskaźnika bufora nie oddaje go na listę. Pobranie bierze bufor z listy albo alokuje nowy.

Reprezentacja pośrednia. Drzewo programu po sprawdzeniu typów. Po angielsku `high-level intermediate representation`, w kodzie `HIR`. Niesie typ i rodzaj użycia nazwy. Nie niesie numeru regionu.

Region. Fragment programu i czas życia pamięci z nim związany: ciało funkcji, blok, gałąź warunku, ciało pętli albo funkcja dopisana na końcu wywołania, o ile zdjęcie opakowań nie skleiło go z otoczeniem.

Rodzaj użycia. Adnotacja na nazwie w reprezentacji pośredniej: użycie lokalne, współdzielenie, kopia, przeniesienie albo promocja. W kodzie `UseKind`. Nie zastępuje analizy własności.

Sprawdzenie. Funkcja `frontend::check`: czytanie składni, sprawdzanie typów, analiza własności, a przy braku błędów także oznaczenie buforów, wyniesienie alokacji i analiza ucieczki.

Typ nieznany. Typ w reprezentacji pośredniej, który nie ma odpowiednika w składni. Tłumi część dalszych komunikatów o typie. Nie jest kopiowalny. W kodzie `Unknown`.

Wartość nullable. Typ z dopiskiem `?`. Codegen obsługuje napisy i typy proste poza `unit`; nullable tablice odrzuca sprawdzanie typów, a typy funkcji i `unit?` nie mają reprezentacji.

Współdzielenie. Odczyt stałej, której nie wolno skopiować, z regionu zewnętrznego. Region wewnętrzny nie kopiuje treści napisu i nie unieważnia nazwy. W wydruku drzewa znacznik to `Shared`.

Wyniesienie alokacji. Rozpoznanie dwóch sąsiednich instrukcji i ustawienie pola `alloc_in_binding`, żeby napis powstał od razu w buforze zmiennej docelowej. Plik `src/hoist.rs`.

Zakres źródłowy. Para przesunięć bajtowych w pliku. W kodzie `Span`. Część komunikatów go nie ma. Wiersz poleceń pomija wtedy numer linii.

Zdjęcie opakowań. Usunięcie bloków, które zawierają tylko jeden wewnętrzny blok, żeby nie tworzyć regionu na same nawiasy. W kodzie `peel_blocks`. Licznik zdjętych nawiasów ląduje w `compacted_braces`.

codegen_push. Flaga na węźle regionu. Wygenerowany kod woła `bork_arena_push` tylko wtedy, gdy flaga jest ustawiona. Korzeń funkcji ma ją zawsze.

decl_tys. Wektor typów deklaracji w kolejności źródła, przekazany ze sprawdzania typów do analizy własności.

loop_move_ban. Zbiór nazw, których nie wolno przenieść w ciele pętli, bo były widoczne na wejściu do pętli.

unit. Typ funkcji bez zadeklarowanego wyniku. W składni pisze się małymi literami. Nie nazywa się `Unit`.

27 Dodatek C. Czego kompilator jeszcze nie potrafi

Ten dodatek wymienia ograniczenia aktualnego kodu i rozbieżności między kodem a dokumentacją. Awarie przywołane niżej są wynikami historycznymi, chyba że opis wyraźnie mówi, że zostały ponownie sprawdzone. Nic z tej listy nie jest propozycją naprawy.

27.1 Rzeczy świadomie niezaimplementowane

Aktualny kompilator ma ograniczone pożyczki `&T`: dla niekopiowanych typów, w parametrach i stałych widokach, z jawnym `&nazwa` przy wywołaniu. Nie ma pełnego borrow checkera, `&mut`, zwrotu referencji ani przechowywania referencji w polach. Działają reborrow przy bezpośrednim wywołaniu wewnątrz `if`/pętli oraz odczyt i zapis elementów przez `&[T; N]`.

Pozostałe ograniczenia z `src/escape.rs`, `docs/language.md` i `docs/memory-model.md`: nie ma areny wyniku po stronie wywołującego, więc zwrot wyniku `concat` i napisu z regionu wewnętrznego jest odrzucany; `promote` nie działa przy `return`. Wyniesienie alokacji rozpoznaje tylko określone, sąsiednie instrukcje. `Nullable String?` i typy proste poza `unit` mają `codegen` dla `Some`, `None`, `!!`, `?`: oraz `==/!=`. `Nullable` tablice odrzuca sprawdzanie typów, a nullable `unit` i typy funkcji nie mają reprezentacji. Generowanie kodu nadal nie tłumaczy funkcji dopisanej na końcu wywołania ani wywołań pośrednich. Porównania floatów przechodzą sprawdzanie typów, ale obecnie panikują w `codegenie`; float jako argument funkcji

jest odrzucany. `main` nie przyjmuje parametrów i nie zwraca typu innego niż `i32` lub `unit`. Nie ma modułów, struktur, typów ogólnych ani wyjątków.

Plik `TODO.md` wymienia prace nad samym kompilatorem, a nie nad semantyką języka. Są to rozcięcie gościa generowania kodu, kursor regionów zamiast makra, test zgodności kontroli przed generowaniem kodu ze sprawdzeniem, więcej testów par wejście-wyjście oraz uporządkowanie planów w `docs/superpowers/plans/`.

27.2 Gdzie dokument mówi co innego niż kod

Przykład `concat(left, right)` przy dwóch zmiennych napisowych w `docs/language.md` jest pokazany jako wzorzec. Sprawdzenie odrzuca go komunikatem, że `left` nie jest kopiowalne i trzeba je przenieść do bloku. Zdanie o kolejności operatorów w tym samym dokumencie pomija `&&`, `||` i `!`. Gramatyka te piętra ma.

Notatka projektowa z 23 września 2026 o reprezentacji pośredniej i LLVM mówi o LLVM 18 i o kolejności „najpierw własność, potem typy”. Kod używa LLVM 23, a sprawdzanie typów wykonuje się przed analizą własności. Specyfikacja serwera edytora mówi o komunikatach parsera. Kod woła pełne `frontend::check`. Specyfikacja pierwszej wersji regionów mówi o braku pełnego sprawdzania typów. Sprawdzanie typów jest. Nagłówek w README mówi o diagnostyce składni, a akapit niżej i kod robią pełne sprawdzenie. Specyfikacja parsera mówi o wyniku `Unit` i o braku zakresów źródłowych. W kodzie typ nazywa się `unit` i zakresy są. Dokument języka mówi, że dzielenie minimalnej liczby całkowitej przez minus jeden przerywa program. Strażnik w `guard_int_div` ten przypadek obsługuje, ale literału ujemnego nie da się zapisać, bo nie ma jednoargumentowego minusa.

27.3 Awarie i ostre krawędzie, które zostały uruchomione

Kompilator był złożony z opcją `codegen`, na LLVM 23.1.2 i rustc 1.98.1.

W historycznym przebiegu przekazanie napisu do funkcji użytkownika kończyło się awarią `into_int_value` (kod 101). Aktualny listing uruchomiłem ponownie z binarką `codegen: build` zakończył się kodem 0, program wypisał `hello` i zwrócił 0. Stary wiersz wyników pozostaje historyczny.

Arytmetyka floatów ma ścieżkę emisji. Porównanie `f64`, np. `x > 1.0`, przechodzi sprawdzanie typów, ale wybór ścieżki codegenu odbywa się według typu wyniku `bool`, więc porównanie trafia do emisji całkowitoliczbowej. Ponowne uruchomienie kończy się paniką `FloatValue/IntValue` (kod 101). Float jako argument funkcji jest osobno odrzucany przez codegen.

Indeks poza zakresem i dzielenie przez zero kompilują się. Proces użytkownika kończy się sygnałem przerwania, w powłoce kodem 134, bez tekstu z Borka.

Przepełnienia bufora nie uruchamiano dużym literałem w tej książce, bo krótki przykład trudno przepchnąć ponad 4096 bajtów bez pętli, która alokuje, a pętla czyści bufor przy obiegu. Kod w bibliotece wykonawczej przerywa się wtedy tekstem `arena overflow`, z podaną liczbą bajtów i pojemnością 4096. Test jednostkowy biblioteki to zamyka i nie jest to komunikat kompilacji.

Zwrot wyniku `concat` i część błędów analizy ucieczki nie mają zakresu źródłowego. Wiersz poleceń pomija wtedy linię i kolumnę.

Błąd zgłoszony przez regułę gramatyki, na przykład zły cel przypisania albo za duża liczba, wskazuje koniec pliku.

Parametry funkcji dopisanej na końcu wywołania mają w analizie własności typ nieznany, więc wydruk drzewa pokazuje współdzielenie dla parametrów, które sprawdzanie typów uważa za kopiowalne liczby. Sprawdzenie przechodzi. To myli przy czytaniu `--dump-arenas`, ale nie jest błędem użytkownika.

Komunikat analizy ucieczki o gałęzi warunku mówi o napisie. Warunek w kodzie obejmuje każdy typ trzymany w buforze regionu, także tablicę.

To, że `u32` albo alias `Byte` nie może być wynikiem `main` zwracającego `i32`, nie jest ograniczeniem szerokich liczb całkowitych w ogóle. Funkcja pomocnicza na `i64` i porównanie w `main` działają. Na sprawdzonym programie z odejmowaniem kod wyjścia wyniósł 7. Ograniczenie `main` dotyczy typu wyniku `main`, a nie lokalnej wartości `i64`.

Są dwie stałe 4096: w `src/arena.rs` i w bibliotece wykonawczej. Analiza własności nie woła żadnej. Łatwo poprawić zły plik.

Opakowanie o wewnętrznej niezgodności harmonogramu potrafi ukryć zwykły komunikat o braku wsparcia w tekście, który brzmi jak błąd wewnętrzny kompilatora. Czasem niezgodność naprawdę dotyczy dzieci drzewa regionów. Czasem jest tylko przedrostkiem.

Specyfikacje w `docs/superpowers` opisują świat sprzed tablic, sprzed pętli `while` i sprzed LLVM 23. Plik `TOD0.md` każe je przenieść, gdy wchłoną się w dokument albo w zamknięte poprawki. Nie są mapą bieżącego kodu.

27.4 Co jest pokryte testami

Aktualny korpus w `programs/` zawiera pozytywne i negatywne przypadki pożyczek, `reborrow` w pętlach oraz programy do sprawdzania i budowania; CI uruchamia go przez `tests/programs.rs`. W tej aktualizacji zbudowałem i uruchomiłem sześć przykładów nullable z `programs/build/conditionals/`, w tym `f32?`; krótki dodatkowy test objął `Some`, `?:`, `!!` oraz `==/!=` dla `f32?` i `f64?`.

Ponownie sprawdziłem też przekazanie napisu do funkcji: budowanie i wykonanie przeszły. Porównanie floatów nadal panikuje, a float jako argument funkcji jest odrzucany.

27.5 Kolejność zaufania

Gdy komentarz, specyfikacja i kod się różnią, sprawdzaj najpierw test wykonawczy w `tests/build.rs` lub korpus `programs/`, potem `frontend::check`, kod fazy, `docs/language.md`, a na końcu `docs/superpowers`. O braku pełnego borrow checkera można mówić tylko z zastrzeżeniem, że ograniczone &T już istnieje. Wiersz z `concat(left, right)` nadal wymaga uzgodnienia z aktualną diagnostyką.